

Unit -1

Database System Applications and Introduction to Database Design

Database System Applications:

DBMS contains information about a particular enterprise

1. Collection of interrelated data
2. Set of programs to access the data
3. An environment that is both *convenient* and *efficient* to use

Database Applications:

1. Banking: all transactions
2. Airlines: reservations, schedules
3. Universities: registration, grades
4. Sales: customers, products, purchases
5. Online retailers: order tracking, customized recommendations
6. Manufacturing: production, inventory, orders, supply chain
7. Human resources: employee records, salaries, tax deductions
8. Databases touch all aspects of our lives

What Is a DBMS?

- ☐ A very large, integrated collection of data.
- ☐ Models real-world enterprise. Entities (e.g., students, courses) Relationships (e.g., Madonna is taking CS564)
- ☐ A Database Management System (DBMS) is a software package designed to store and manages databases.

Why Use a DBMS?

- ☐ Data independence and efficient access.
- ☐ Reduced application development time.
- ☐ Data integrity and security.
- ☐ Uniform data administration.
- ☐ Concurrent access, recovery from crashes.

Why Study Databases??

- ☐ Shift from computation to information at the “low end”: scramble mess!) at the “high end”: scientific application
- ☐ Datasets increasing in diversity and volume. Digital libraries, interactive video, Human Genome project, EOS project ... need for DBMS exploding
- ☐ DBMS encompasses most of CS OS, languages, theory, AI, multimedia, logic

Purpose of Database Systems:

In the early days, database applications were built directly on top of file systems

Drawbacks of using file systems to store data:

- ☐ Data redundancy and inconsistency
- ☐ Multiple file formats, duplication of information in different files
- ☐ Difficulty in accessing data
- ☐ Need to write a new program to carry out each new task
- ☐ Data isolation — multiple files and formats
- ☐ Integrity problems
- ☐ Hard to add new constraints or change existing ones
- ☐ Atomicity of updates
- ☐ Failures may leave database in an inconsistent state with partial updates carried out
- ☐ Example: Transfer of funds from one account to another should either complete or not happen at all
- ☐ Concurrent access by multiple users
- ☐ Concurrent accessed needed for performance
- ☐ Uncontrolled concurrent accesses can lead to inconsistencies
- ☐ Example: Two people reading a balance and updating it at the same time
- ☐ Security problems
- ☐ Hard to provide user access to some, but not all, data

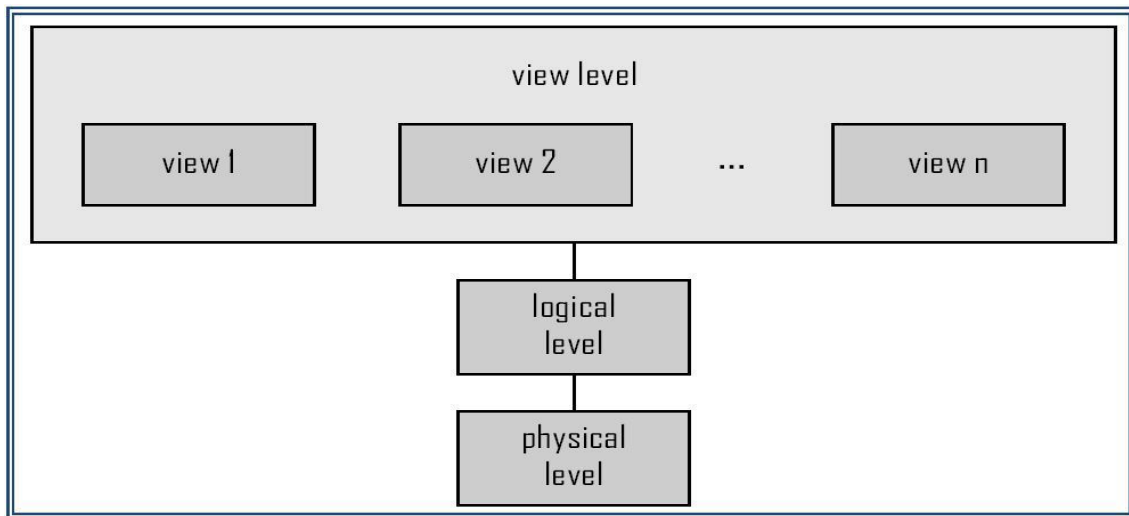
Database systems offer solutions to all the above problems

Files vs. DBMS:

- ☐ Application must stage large datasets between main memory and secondary storage (e.g., buffering, page-oriented access, 32-bit addressing, etc.)
 - ☐ Special code for different queries
 - ☐ Must protect data from inconsistency due to multiple concurrent users
 - ☐ Crash recovery
 - ☐ Security and access control

VIEW OF DATA:

An architecture for a database system :



DATA ABSTRACTION:

□ Major purpose of dbms is to provide users with abstract view of data i.e. the system hides certain details of how the data are stored and maintained. Since database system users are not computer trained, developers hide the complexity from users through 3 levels of abstraction, to simplify user's interaction

Levels of Abstraction

1) Physical level of data abstraction:

describes how a record (e.g., customer) is stored. This is the lowest level of abstraction which

describes how data are actually stored.

2) Logical level of data abstraction:

This level hides what data are actually stored in the database and what relationship exists among them. describes data stored in database, and the relationships among the data.

type customer = record

customer_id:string; customer_name:string;

customer_street:string; customer_city :

string;

end;

3) View Level of data abstraction:

View provides security mechanism to prevent user from accessing certain parts of database. application programs hide

details of data types. Views can also hide information (such as an employee's salary) for security purposes.

Summary

- DBMS used to maintain, query large datasets.
- Benefits include recovery from system crashes, concurrent access, quick application development, data integrity and security.
- Levels of abstraction give data independence.
- A DBMS typically has a layered architecture.
- DBAs hold responsible jobs and are well-paid!
- DBMS R&D is one of the broadest, most exciting areas in CS.

Instances and Schemas:

Similar to types and variables in programming languages

Schema –the logical structure of the database

- Example: The database consists of information about a set of customers and accounts and the relationship between them) Analogous to type information of a variable in a program.

Physical schema: database design at the physical level

Logical schema: database design at the logical level

Instance –the actual content of the database at a particular point in time analogous to the value of a variable.

Physical Data Independence –the ability to modify the physical schema without changing the logical schema.

Applications depend on the logical schema.

In general, the interfaces between the various levels and components should be well defined so that changes in some parts do not seriously influence others.

Data Models:

A collection of tools for describing

- Data
- Data relationships
- Data semantics
- Data constraints
- Relational model

- Entity-Relationship data model (mainly for database design)
- Object-based data models (Object-oriented and Object-relational)
- Semi structured data model (XML)
- Other older models:

o Network model

o Hierarchical model

1. A data model is a collection of concepts for describing data.
2. A schema is a description of a particular collection of data, using the a given data model.
3. The relational model of data is the most widely used model today.

Main concept: relation, basically a table with rows and columns

Every relation has a schema, which describes the columns, or fields.

Example: University Database

Conceptual schema:

Students(sid: string, name: string, login: string, age: integer, gpa:real)

Courses(cid: string, cname:string, credits:integer)

Enrolled(sid:string, cid:string, grade:string)

Physical schema:

Relations stored as unordered files.

Index on first column of Students.

External Schema (View):

Course_info(cid:string,enrollment:integer)

Data Independence:

Applications insulated from how data is structured and stored.

Logical data independence: Protection from changes in *logical* structure of data.

Physical data independence: Protection from changes in *physical* structure of data.

Data base Languages:

Data Manipulation Language (DML):

- Language for accessing and manipulating the data organized by the appropriate data Model.
- DML also known as query language
- Two classes of languages

Procedural –user specifies what data is required and how to get those data

Declarative (nonprocedural) –user specifies what data is required without specifying how to get those data.

Data Definition Language (DDL):

- Specification notation for defining the database schema

Example: **create table** *account*(

account_number **char**(10),

branch_name **char**(10),

balance **integer**)

- DDL compiler generates a set of tables stored in a *data dictionary*
- Data dictionary contains metadata (i.e., data about data)
 - Database schema
 - Data *storage and definition* language
- Specifies the storage structure and access methods used
 - Integrity constraints
- Domain constraints
- Referential integrity (e.g. *branch_name* must correspond to a valid

branch in the *branch* table)

- Authorization

Data Base Access from Application Programs:

- SQL: widely used non-procedural language
 - Example: Find the name of the customer with customer-id 192-83-7465

```
select      customer.customer_name
from        Customer
where       customer.customer_id = '192-83-7465'
```

– Example: Find the balances of all accounts held by the customer with customer-Id 192-83-7465.

```
select account.balance
from      depositor, account
where depositor.customer_id = '192-83-7465' and
depositor.account_number = account.account_number
```

SQL: widely used non-procedural language

Example: Find the name of the customer with customer-id 192-83-7465

```
select customer.customer_name from customer
where customer.customer_id= '192-83-7465'
```

Example: Find the balances of all accounts held by the customer with customer-id 192-83-7465

```
select account.balance
from      depositor, account
where depositor.customer_id = '192-83-7465' and
depositor.account_number = account.account_number
```

SQL

- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL
 - Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database.

Transaction Management:

- A **transaction** is a collection of operations that performs a single logical function in a database application
- **Transaction-management component** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.

SQL

- Application programs generally access databases through one of
 - Language extensions to allow embedded SQL

- Application program interface (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database.

Transaction Management:

- A **transaction** is a collection of operations that performs a single logical function in a database application

Transaction-management component ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.

- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

Data storage and Querying:

- Storage management
- Query processing
- Transaction processing

Storage Management

- Storage manager is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- **The storage manager is responsible to the following tasks:**

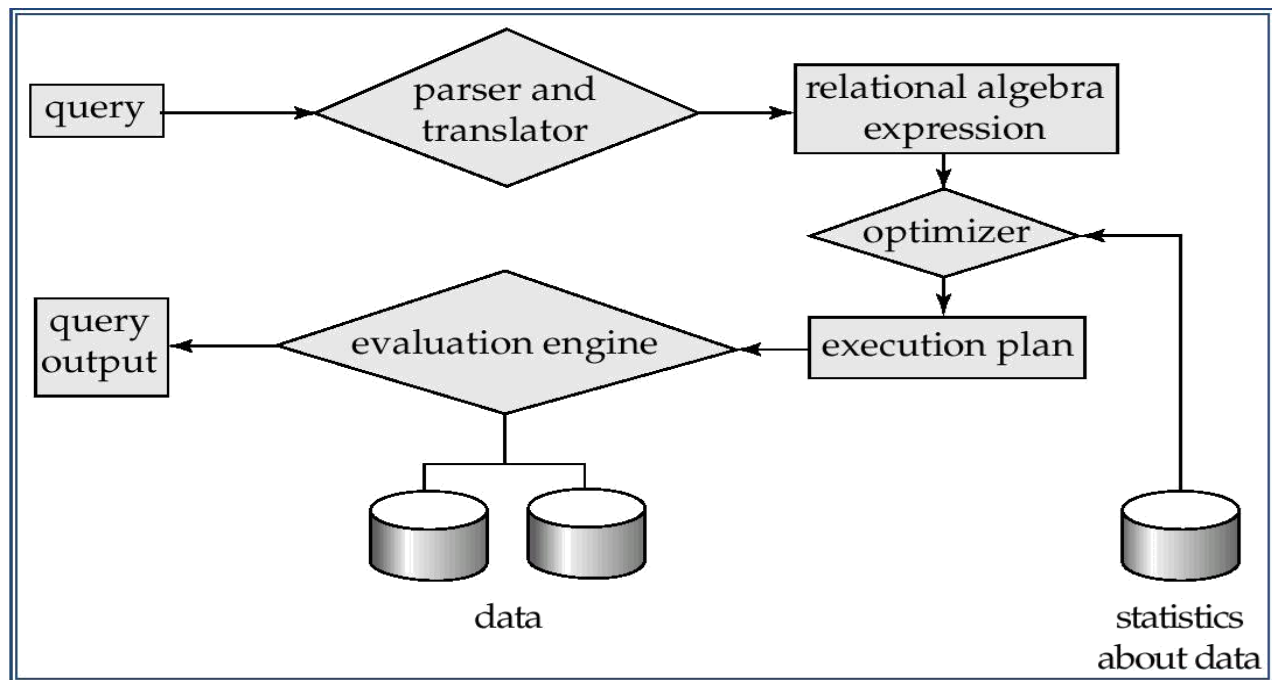
- Interaction with the file manager
- Efficient storing, retrieving and updating of data

- **Issues:**

- Storage access
- File organization
- Indexing and hashing

Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation



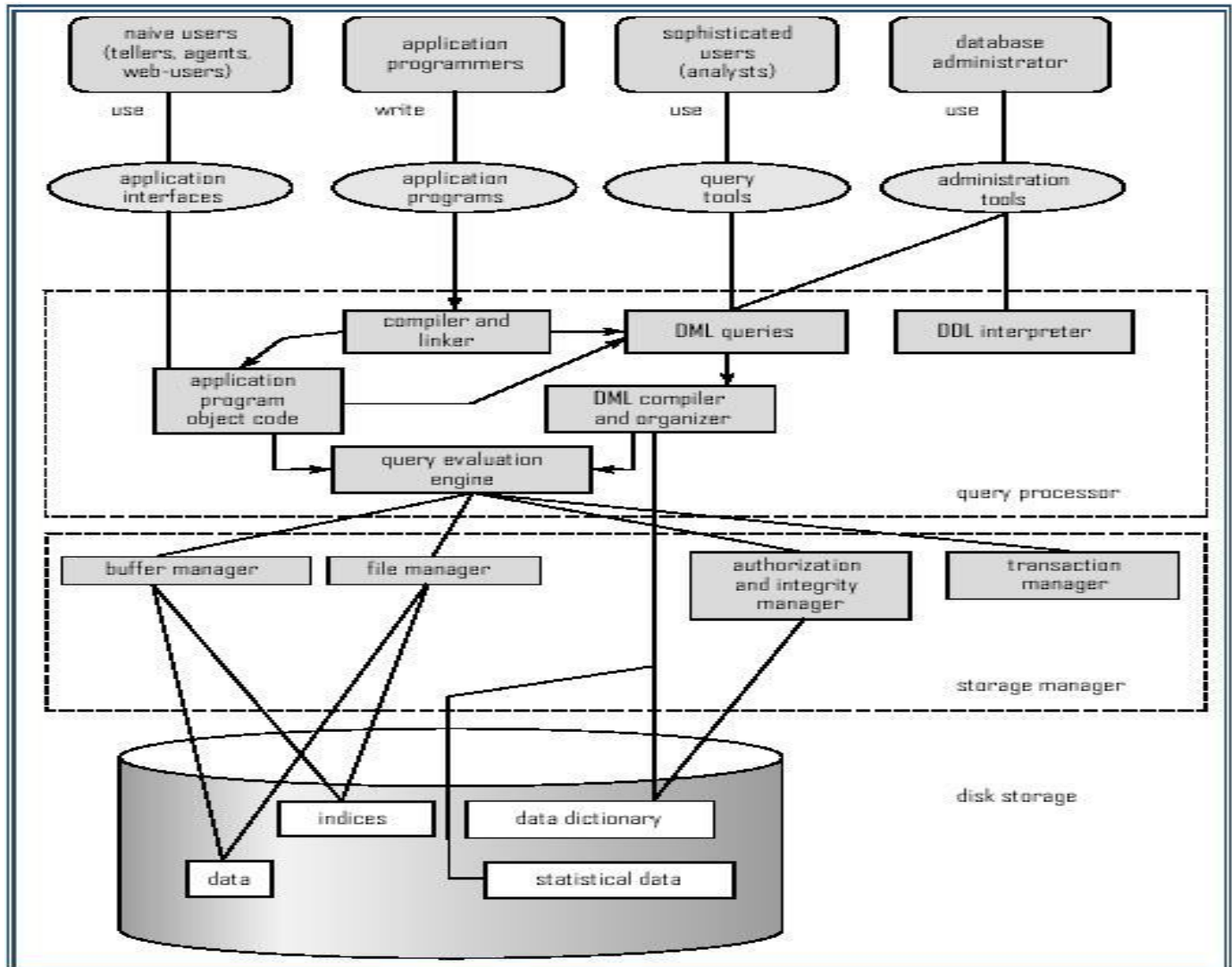
- Alternative ways of evaluating a given query
 - Equivalent expressions
 - Different algorithms for each operation
- Cost difference between a good and a bad way of evaluating a query can be enormous
- Need to estimate the cost of operations
 - Depends critically on statistical information about relations which the database must maintain
 - Need to estimate statistics for intermediate results to compute cost of complex expressions

Database Architecture:

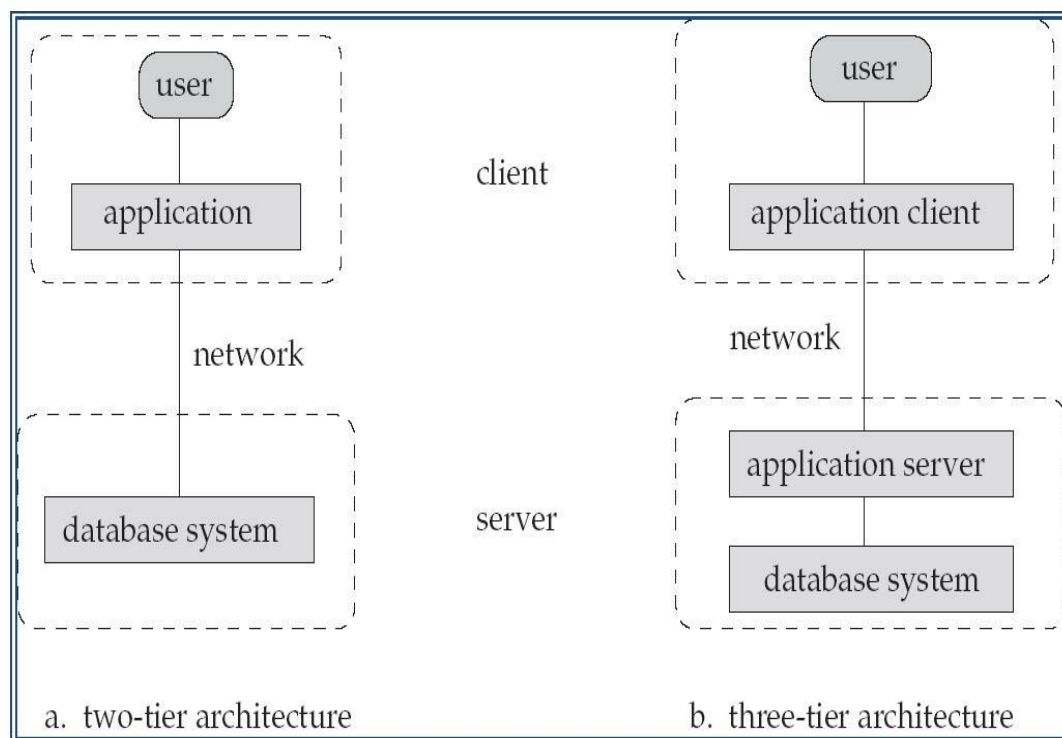
The architecture of a database systems is greatly influenced by the underlying computer system on which the database is running:

- Centralized

- Client-server
- Parallel (multiple processors and disks)
- Distributed

Structure of DBMS:

Database Application Architectures:



Database Users and Administrators:

Database Users

Users are differentiated by the way they expect to interact with the system

- **Application programmers** –interact with system through DML calls
- **Sophisticated users** –form requests in a database query language
- **Specialized users** –write specialized database applications that do not fit into the traditional data processing framework
- **Naïve users** –invoke one of the permanent application programs that have been written previously
 - Examples, people accessing database over the web, bank tellers, clerical staff

Database Administrator

- Coordinates all the activities of the database system

- has a good understanding of the enterp

- **Database administrator's duties include:**

- Storage structure and access method definition
- Schema and physical organization modification
- Granting users authority to access the database
- Backing up data
- Monitoring performance and responding to changes
- Database tuning.

History of Database Systems:

- 1950s and early 1960s:

- Data processing using magnetic tapes for storage
- Tapes provide only sequential access
- Punched cards for input

- Late 1960s and 1970s:

- Hard disks allow direct access to data
- Network and hierarchical data models in widespread use
- Ted Codd defines the relational data model
- Would win the ACM Turing Award for this work
- IBM Research begins System R prototype

UC Berkeley begins Ingres prototype

- High-performance (for the era) transaction processing
- 1980s:
- Research relational prototypes evolve into commercial systems

- SQL becomes industry standard
- Parallel and distributed database systems
- Object-oriented database systems
- 1990s:
 - Large decision support and data-mining applications
 - Large multi-terabyte data warehouses
 - Emergence of Web commerce
- 2000s:
 - XML and XQuery standards
 - Automated database administration
 - Increasing use of highly parallel database systems
 - Web-scale distributed data storage systems

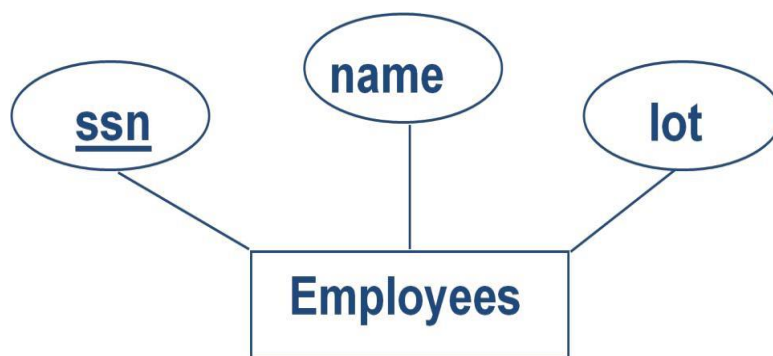
Introduction to Database Design:

- Conceptual design: (*ER Model is used at this stage.*)
 - What are the *entities* and *relationships* in the enterprise?
 - What information about these entities and relationships should we store in the database?
 - What are the *integrity constraints* or *business rules* that hold?

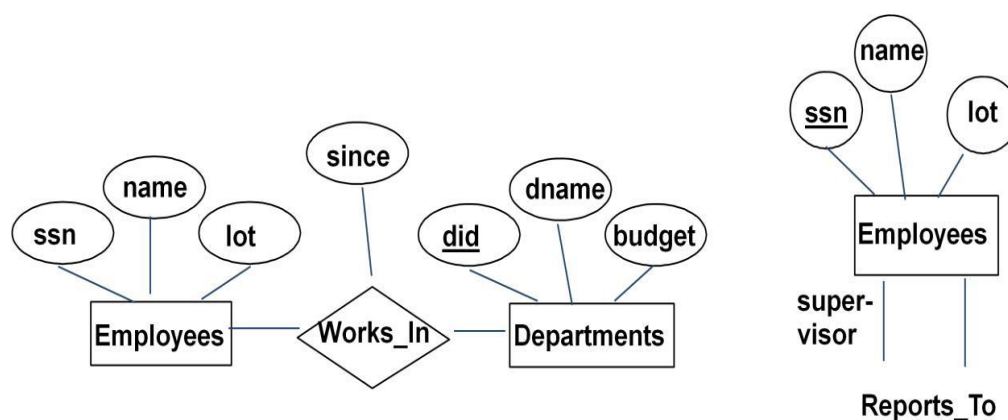
A database 'schema' in the ER Model can be represented by *ER diagrams*).

- Can map an ER diagram into a relational schema.

ER Diagrams:



- **Entity**: Real-world object distinguishable from other objects. An entity is described (in DB) using a set of *attributes*.
- **Entity Set**: A collection of similar entities. E.g., all employees.
 - All entities in an entity set have the same set of attributes. (Until we consider ISA hierarchies, anyway!)
 - Each entity set has a *key*.
 - Each attribute has a *domain*.



- **Relationship**: Association among two or more entities. E.g., Attishoo works in Pharmacy department.
- **Relationship Set**: Collection of similar relationships.
 - An n-ary relationship set *R* relates *n* entity sets *E*₁ ... *E*_{*n*}; each relationship in *R*

involves entities $e_1, E_1, \dots, e_n, E_n$

- Same entity set could participate in different relationship sets, or in different “roles” in same set.

Modeling:

- A *database* can be modeled as:
 - a collection of entities,
 - relationship among entities.

Entities and Entity Sets:

- An **entity** is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- Entities have *attributes*
 - Example: people have *names* and *addresses*
- An **entity set** is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays

Example: Entity Sets customer and loan

321-12-3123	Jones	Main	Harrison
019-28-3746	Smith	North	Rye
677-89-9011	Hayes	Main	Harrison
555-55-5555	Jackson	Dupont	Woodside
244-66-8800	Curry	North	Rye
963-96-3963	Williams	Nassau	Princeton
335-57-7991	Adams	Spring	Pittsfield
<i>customer</i>			

L-17	1000
L-23	2000
L-15	1500
L-14	1500
L-19	500
L-11	900
L-16	1300
<i>loan</i>	

Attributes:

- An entity is represented by a set of attributes, that is descriptive properties possessed by all members of an entity set.

- **Domain** –the set of permitted values for each attribute

- **Attribute types:**

– *Simple* and *composite* attributes.

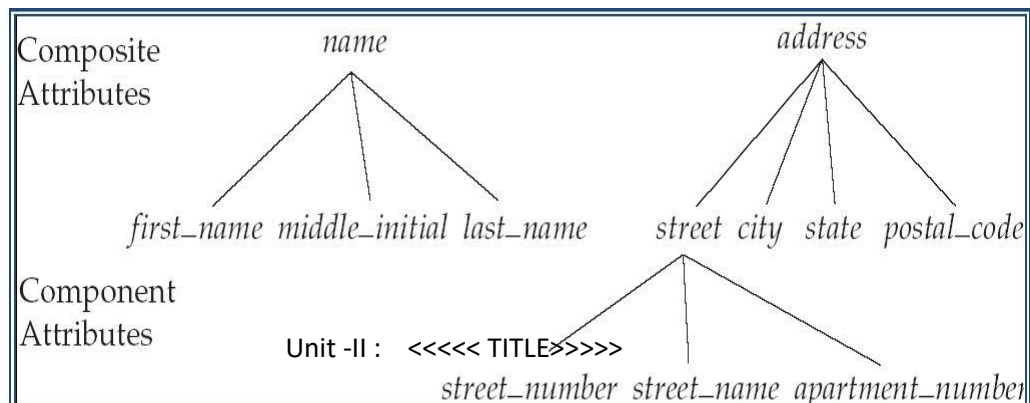
– *Single-valued* and *multi-valued* attributes

- Example: multivalued attribute: *phone_numbers*

– *Derived* attributes

- Can be computed from other attributes
- Example: age, given date_of_birth

- **Composite Attributes**



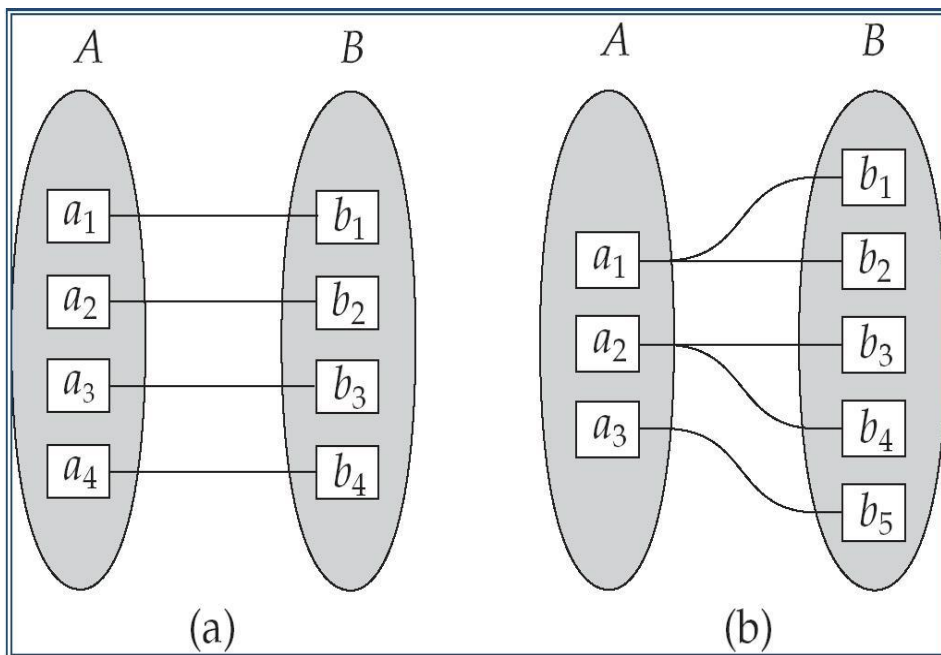
Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one

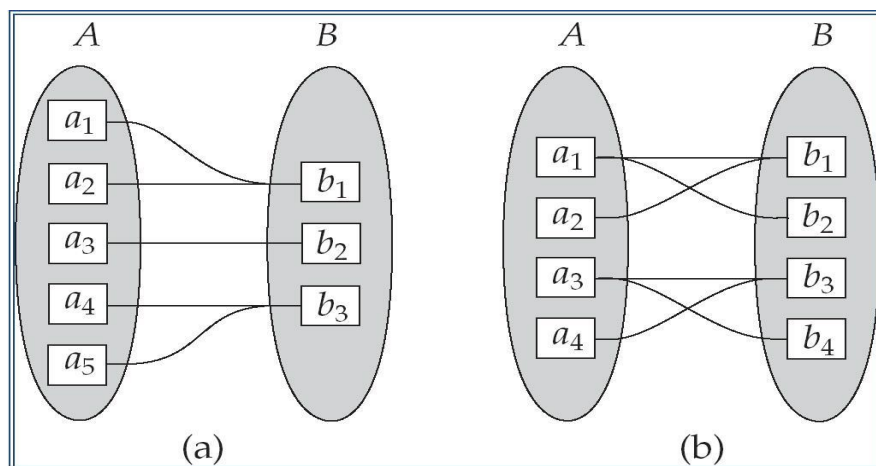
- One to many
- Many to one
- Many to many

Mapping Cardinalities:

Note: Some elements in A and B may not be mapped to any elements in the other set

**Mapping Cardinalities**

Note: Some elements in A and B may not be mapped to any elements in the other set



Relationships and Relationship Sets

- A **relationship** is an association among several entities

A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from

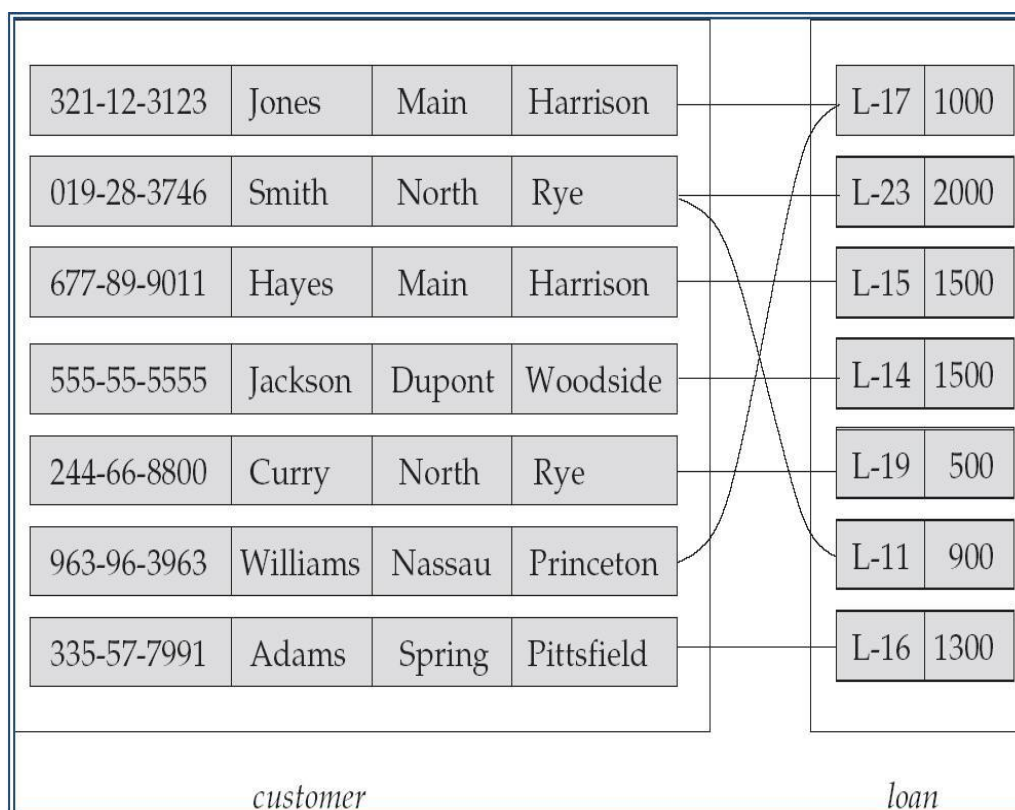
entity sets

- $\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$ where $(e_1, e_2, \dots, e_n)$ is a relationship

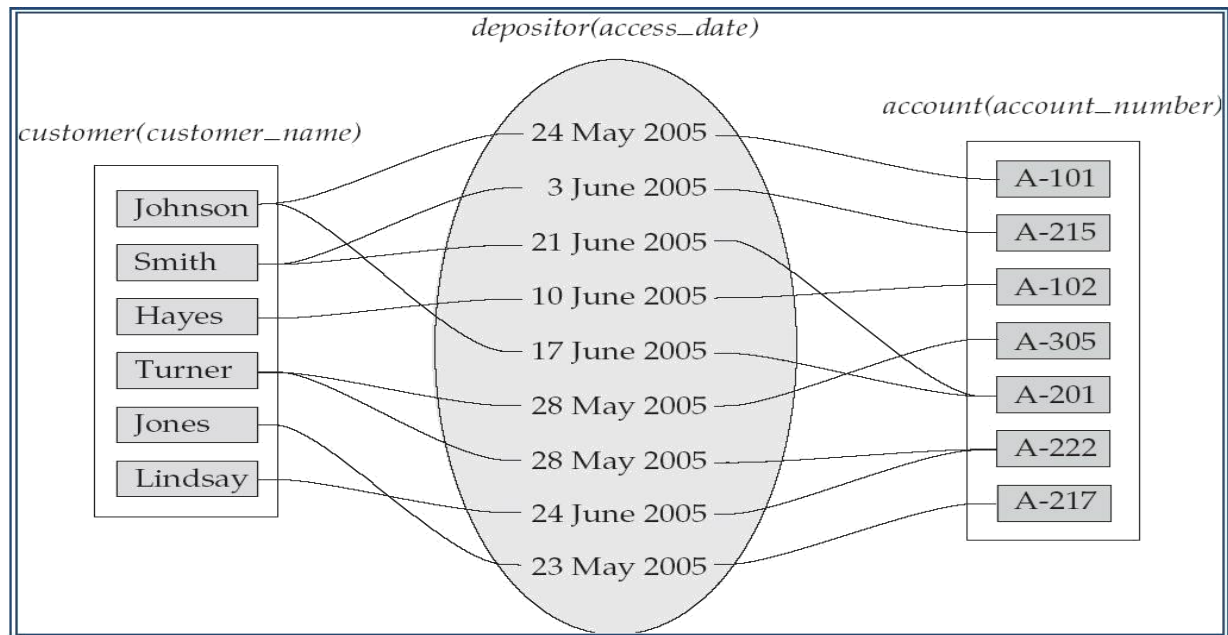
– Example:

(Hayes, A-102) $\in$ *depositor*

Relationship Set *borrower*



An **attribute** can also be property of a relationship set.



- For instance, the *depositor* relationship set between entity sets *customer* and *account* may have the attribute *access-date*

Degree of a Relationship Set

- Refers to number of entity sets that participate in a relationship set.
- Relationship sets that involve two entity sets are **binary** (or degree two). Generally, most relationship sets in a database system are binary.
- Relationship sets may involve more than two entity sets.

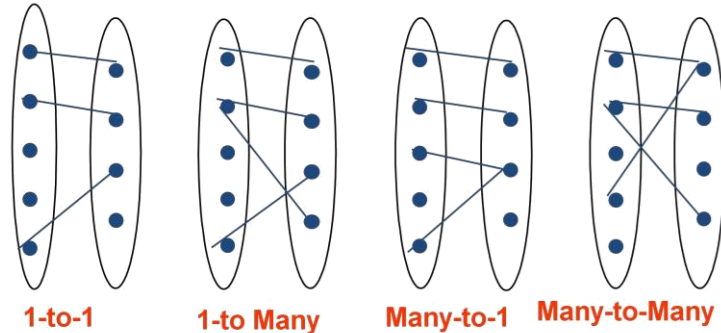
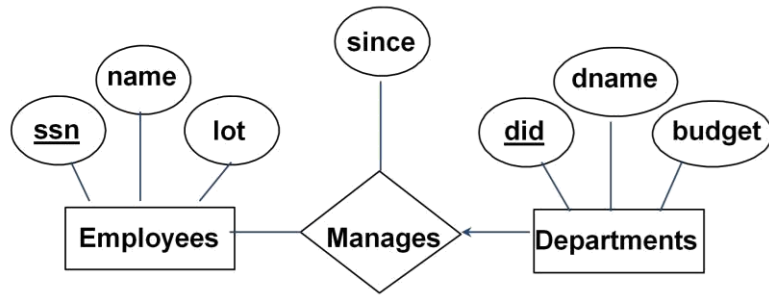
Example: Suppose employees of a bank may have jobs (responsibilities) at multiple branches, with different jobs at different branches. Then there is a ternary relationship set between entity sets *employee*, *job*, and *branch*

- Relationships between more than two entity sets are rare. Most relationships are binary.

Additional features of the ER model

Key Constraints

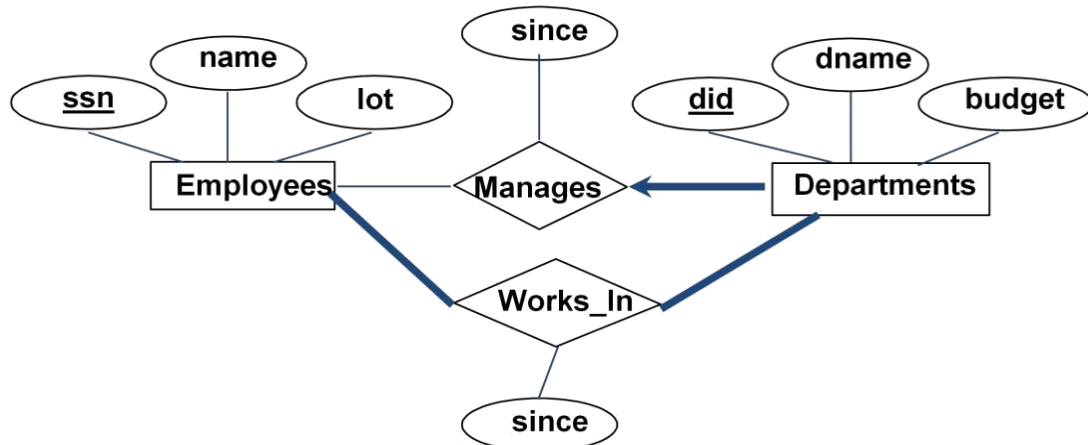
- Consider Works_In:
An employee can work in many departments; a dept can have many employees.
- In contrast, each dept has at most one manager, according to the key constraint on Manages.



Slide No:L4-1

Participation Constraints

- Does every department have a manager?
 - If so, this is a participation constraint: the participation of Departments in Manages is said to be total (vs. partial).
 - Every Departments entity must appear in an instance of the Manages relationship.



Slide No:L4-2

Weak Entities

- A *weak entity* can be identified uniquely only by considering the primary key of another (*owner*) entity.

Owner entity set and weak entity set must participate in a one-to-many relationship set (one owner, many weak entities).

Weak entity set must have total participation in this *identifying* relationship set.

Weak Entity Sets

- An entity set that does not have a primary key is referred to as a **weak entity set**.
- The existence of a weak entity set depends on the existence of a **identifying entity set**

it must relate to the identifying entity set via a total, one-to-many relationship set from the identifying to the weak entity set

Identifying relationship depicted using a double diamond

- The **discriminator** (*or partial key*) of a weak entity set is the set of attributes that distinguishes among all the entities of a weak entity set.
- The primary key of a weak entity set is formed by the primary key of the strong entity set on which the weak entity set is existence dependent, discriminator.
- depict a weak entity set by double rectangles.
- underline the discriminator of a weak entity set with a dashed line.
- *payment_number* –discriminator of the *payment* entity set
- Primary key for *payment* –(*loan_number*, *payment_number*)
- Note: the primary key of the strong entity set is not explicitly stored with the weak

entity set, since it is implicit in the identifying relationship.

- If *loan_number* were explicitly stored, *payment* could be made a strong entity, but then the relationship between *payment* and *loan* would be duplicated by an implicit relationship defined by the attribute *loan_number* common to *payment* and *loan*

More Weak Entity Set Examples

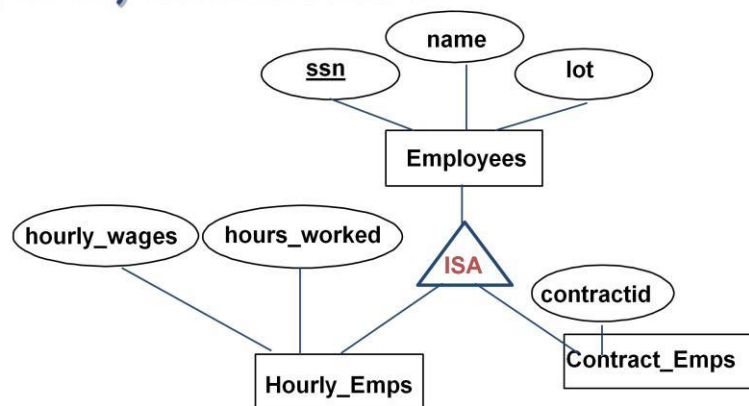
- In a university, a *course* is a strong entity and a *course_offering* can be modeled as a weak entity
- The discriminator of *course_offering* would be *semester* (including year) and *section_number* (if there is more than one section)
- If we model *course_offering* as a strong entity we would model *course_number* as an attribute.

Then the relationship with *course* would be implicit in the *course_number* attribute

ISA ('is a') Hierarchies

ISA ('is a') Hierarchies

- ❖ As in C++, or other PLs, attributes are inherited.
- ❖ If we declare A **ISA** B, every A entity is also considered to be a B entity.

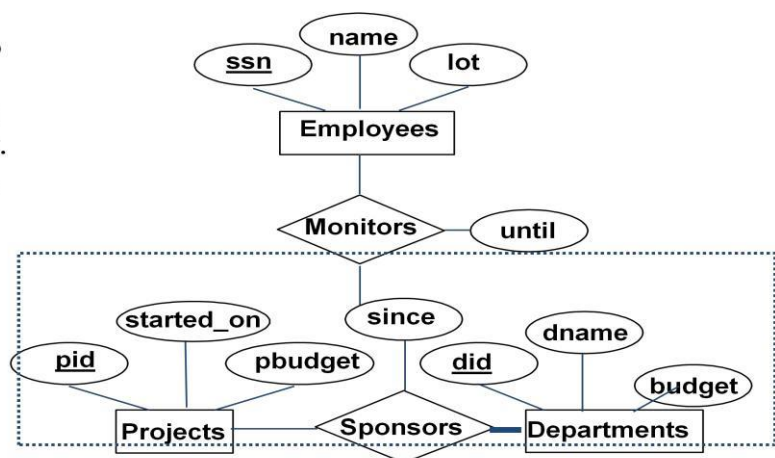


- **Overlap constraints:** Can Joe be an Hourly_Emps as well as a Contract_Emps entity? (*Allowed/disallowed*)
- **Covering constraints:** Does every Employees entity have to be an Hourly_Emps or a Contract_Emps entity? (*Yes/no*)
- Reasons for using ISA:
 - To add descriptive attributes specific to a subclass.
 - To identify entities that participate in a relationship.

Aggregation

Aggregation

- Used when we have to model a relationship involving (entity sets and) a *relationship set*.
 - Aggregation* allows us to treat a relationship set as an entity set for purposes of participation in (other) relationships.



✉ *Aggregation vs. ternary relationship:*

- ❖ Monitors is a distinct relationship, with a descriptive attribute.
- ❖ Also, can say that each sponsorship is monitored by at most one employee.

Slide No:L5-2

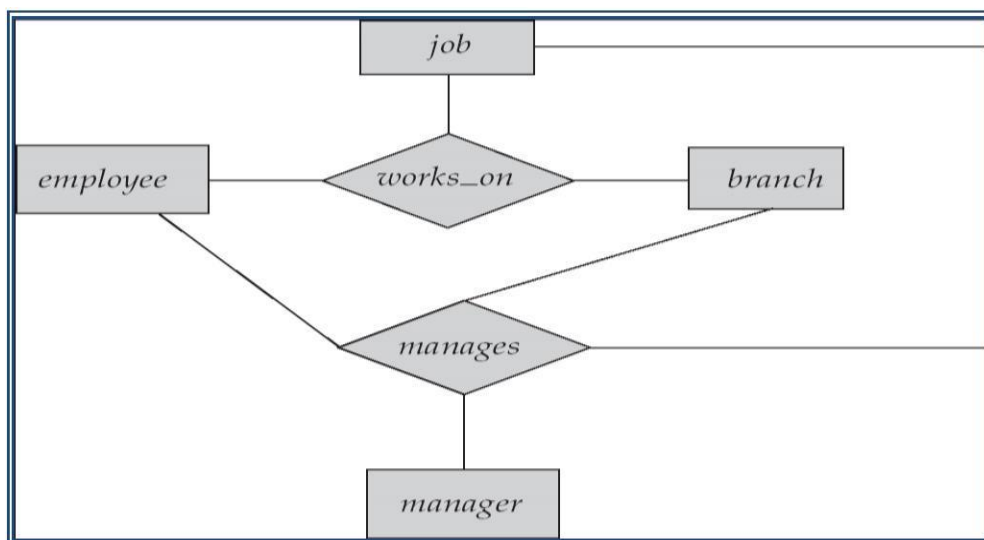
- Relationship sets *works_on* and *manages* represent overlapping information
- Every *manages* relationship corresponds to a *works_on* relationship
- However, some *works_on* relationships may not correspond to any *manages* relationships .

So we can't work on relationship the

Eliminate this redundancy via *aggregation*

- Treat relationship as an abstract entity
- Allows relationships between relationships
- Abstraction of relationship into new entity
- Without introducing redundancy, the following diagram represents:
 - An employee works on a particular job at a particular branch
 - An employee, branch, job combination may have an associated manager

E-R Diagram with Aggregation



Conceptual Design with ER Model

- Design choices:
 - Should a concept be modeled as an entity or an attribute?
 - Should a concept be modeled as an entity or a relationship?

- Identifying relationships: Binary or ternary? Aggregation?
- Constraints in the ER Model:
 - A lot of data semantics can (and should) be captured.
 - But some constraints cannot be captured in ER diagrams.

Entity vs. Attribute

- Should *address* be an attribute of Employees or an entity (connected to Employees by a relationship)?
- Depends upon the use we want to make of address information, and the semantics of the data:

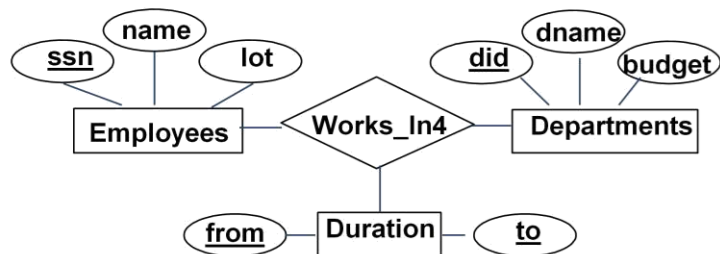
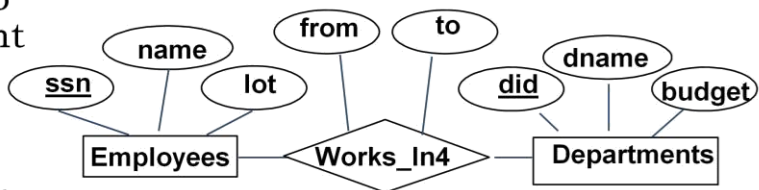
If we have several addresses per employee, *address* must be an entity

(since attributes cannot be set-valued).

If the structure (city, street, etc.) is important, e.g., we want to retrieve employees in a given city, *address* must be modeled as an entity (since attribute values are atomic).

Entity vs. Attribute (Contd.)

- Works_In4 does not allow an employee to work in a department for two or more periods.
- Similar to the problem of wanting to record several addresses for an employee: We want to record *several values of the descriptive attributes for each instance of this relationship*. Accomplished by introducing new entity set, Duration.



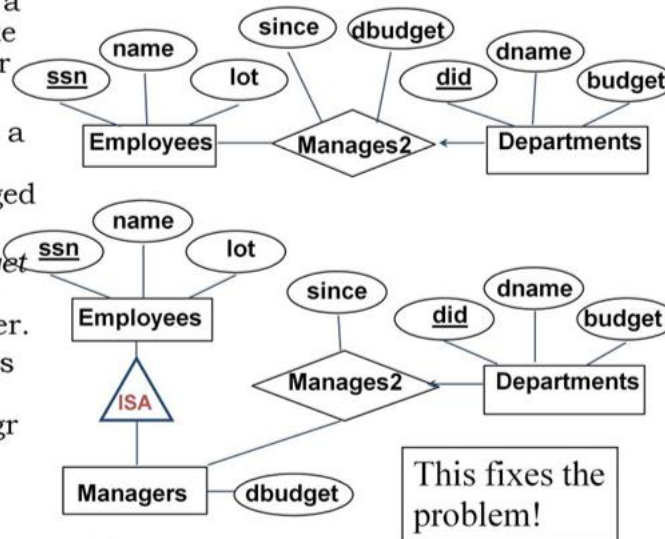
Slide No:L6-3

Entity vs. Relationship

Entity vs. Relationship

- First ER diagram OK if a manager gets a separate discretionary budget for each dept.
- What if a manager gets a discretionary budget that covers *all* managed depts?

- **Redundancy:** *dbudget* stored for each dept managed by manager.
- **Misleading:** Suggests *dbudget* associated with department-mgr combination.

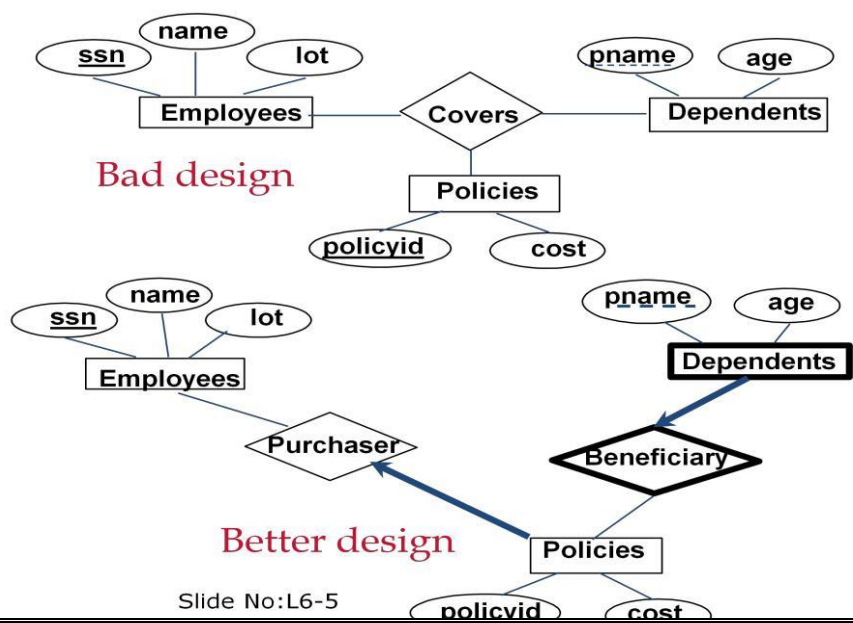


Slide No:L6-4

Binary vs. Ternary Relationships

Binary vs. Ternary Relationships

- If each policy is owned by just 1 employee, and each dependent is tied to the covering policy, first diagram is inaccurate.
- What are the additional constraints in the 2nd diagram?



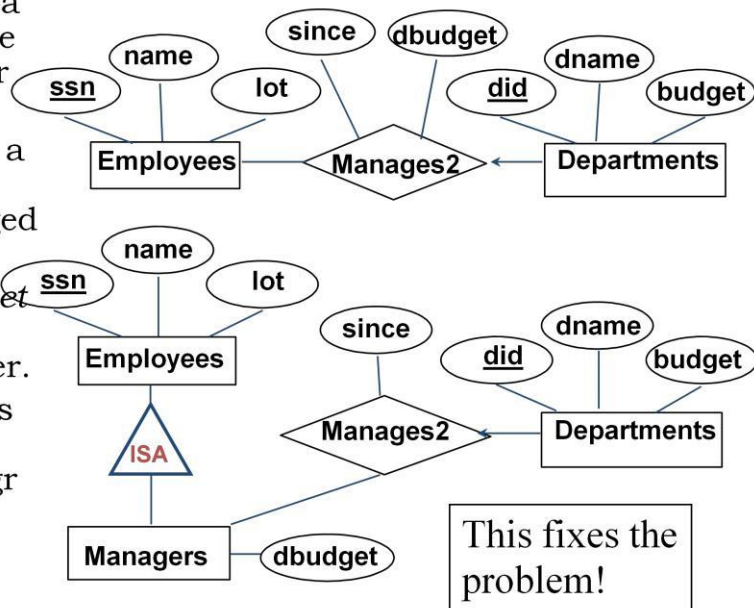
Slide No:L6-5

- Previous example illustrated a case when two binary relationships were better than one ternary relationship.
- An example in the other direction: a ternary relation *Contracts* relates entity sets *Parts*, *Departments* and *Suppliers*, and has descriptive attribute *qty*. No combination of binary relationships is an adequate substitute:
 - S “can-supply” P, D “needs”-with P,S doesand notD “dim
 has agreed to buy P from S.
 - How do we record *qty*?

Entity vs. Relationship

- First ER diagram OK if a manager gets a separate discretionary budget for each dept.
- What if a manager gets a discretionary budget that covers *all* managed depts?

- **Redundancy:** *dbudget* stored for each dept managed by manager.
- **Misleading:** Suggests *dbudget* associated with department-mgr combination.



Slide No:L6-4

Unit -2

Introduction to The Relational Model

Relational Database: Definitions

- *Relational database*: a set of *relations*
- *Relation*: made up of 2 parts:
 - *Instance* : a *table*, with rows and columns.
 - $\#Rows = \text{cardinality}$, $\#fields = \text{degree} / \text{arity}$.
 - *Schema* : specifies name of relation, plus name and type of each column.
- E.G. Students (*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: real).
- Can think of a relation as a *set* of rows or *tuples* (i.e., all rows are distinct).

Example Instance of Students Relation

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Cardinality = 3, degree = 5, all rows distinct

Do all columns in a relation instance have to be distinct?

Relational Query Languages

- A major strength of the relational model: supports simple, powerful *querying* of data.
- Queries can be written intuitively, and the DBMS is responsible for efficient evaluation.

–

The key: precise semantics

for relational queries.

—
extensively re-order operations, and still ensure that the
answer does not change.

Allows the optimizer to

The SQL Query Language

The SQL Query Language

```
SELECT *
FROM Students S
WHERE S.age=18
```

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@ee	18	3.2

- To find just names and logins, replace the first line:

```
SELECT S.name, S.login
```

Querying Multiple Relations

- What does the following query compute?

```
SELECT S.name, E.cid
FROM Students S, Enrolled E
WHERE S.sid=E.sid AND E.grade="A"
```

Given the following instances
of Enrolled and Students:

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

sid	cid	grade
53831	Carnatic101	C
53831	Reggae203	B
53650	Topology112	A
53666	History105	B

we get:

S.name	E.cid
Smith	Topology112

Creating Relations in SQL

- Creates the Students relation. Observe that the type of each field is specified, and enforced by the DBMS whenever tuples are added or modified.

```
CREATE TABLE Students (sid: CHAR(20), name: CHAR(20), login: CHAR(10), age: INTEGER,
gpa: REAL)
```

- As another example, the Enrolled table holds information about courses that students

take.

```
CREATE TABLE Enrolled (sid: CHAR(20), cid: CHAR(20), grade: CHAR(2))
```

2.2. Integrity Constraints (ICs) over Relations:

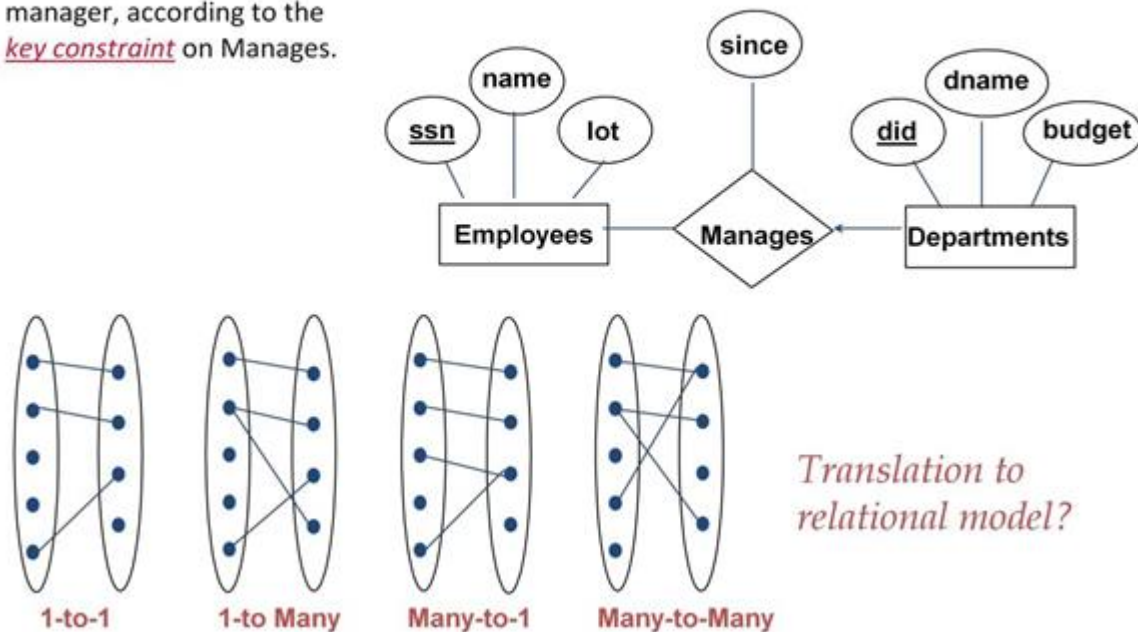
- IC: condition that must be true for *any* instance of the database; e.g., domain constraints.
- ICs are specified when schema is defined.
- ICs are checked when relations are modified.
- A legal instance of a relation is one that satisfies all specified ICs.
- DBMS should not allow illegal instances.
- If the DBMS checks ICs, stored data is more faithful to real-world meaning.
 - Avoids data entry errors, too!

Primary Key Constraints

- A set of fields is a key for a relation if :
 1. No two distinct tuples can have same values in all key fields, and
 2. This is not true for any subset of the key.
 - Part 2 false? A *superkey*.
 - If there's >1 key for a relation, one *primary key*.
- E.g., *sid* is a key for Students. (What about *name*?) The set {*sid*, *gpa*} is a superkey.

Review: Key Constraints

- Each dept has at most one manager, according to the key constraint on Manages.



Primary and Candidate Keys in SQL

- Possibly many candidate keys (specified using UNIQUE), one of which is chosen as the *primary key*.

Foreign Keys, Referential Integrity

- Foreign key: Set of fields in one relation that is used to 'refer' to relation. (Must correspond to primary key of the second relation.) Like a 'logical pointer'.
- E.g. *sid* is a foreign key referring to Students:
 - Enrolled(*sid*: string, *cid*: string, *grade*: string)
 - If all foreign key constraints are enforced, referential integrity is achieved, i.e., no dangling references.
 - Can you name a data model w/o referential integrity?
- Links in HTML!

Foreign Keys in SQL

- Only students listed in the Students relation should be allowed to enroll for courses.

2.3.ENFORCING INTEGRITY CONSTRAINTS:

- Consider Students and Enrolled; *sid* in Enrolled is a foreign key that references Students.
- What should be done if an Enrolled tuple with a non-existent student id is inserted? (*Reject it!*)
- What should be done if a Students tuple is deleted?
 - Also delete all Enrolled tuples that refer to it.
 - Disallow deletion of a Students tuple that is referred to.
 - Set *sid* in Enrolled tuples that refer to it to a *default sid*.
 - (In SQL, also: Set *sid* in Enrolled tuples that refer to it to a special value *null*, denoting *'unknown'* or *'inapplicable'*.)
- Similar if primary key of Students tuple is updated.

Referential Integrity in SQL

- SQL/92 and SQL:1999 support all 4 options on deletes and updates.
- Default is NO ACTION (*delete/update is rejected*)
- CASCADE (also delete all tuples that refer to deleted tuple)
- SET NULL / SET DEFAULT (sets foreign key value of referencing tuple)

Where do ICs Come From?

- ICs are based upon the semantics of the real-world enterprise that is being described in the database relations.
- We can check a database instance to see if an IC is violated, but we can NEVER infer that an IC is true by looking at an instance.
- An IC is a statement about *all possible* instances!
- From example, we know *name* is not a key, but the assertion that *sid* is a key is given to us.
- Key and foreign key ICs are the most common; more general ICs supported too.

24.QUERYING RELATIONAL DATA:

Formal Relational Query Languages

- Two mathematical Query Languages form the and for implementation:
 - Relational Algebra: More operational, very useful for representing execution plans.
 - Relational Calculus: Lets users describe what they want, rather than how to compute it. (Non-operational, declarative.)

Preliminaries:

- A query is applied to relation instances, and the result of a query is also a relation instance.
- Schemas of input relations for a query are fixed (but query will run regardless of instance!)
- The schema for the result of a given query is also fixed! Determined by definition of query language constructs.
- Positional vs. named-field notation:
 - Positional notation easier for formal definitions, named-field notation more readable.
 - Both used in SQL

Example Instances

- “Sailors” and “Reserves” relations for our examples.
- We’ll use positional or named field notation, assume that names of fields in query results are ‘inherited’ from names of fields in query input relations.

R1

<u>sid</u>	<u>bid</u>	<u>day</u>
22	101	10/10/96
58	103	11/12/96

S1

<u>sid</u>	<u>sname</u>	<u>rating</u>	<u>age</u>
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

<u>sid</u>	<u>sname</u>	<u>rating</u>	<u>age</u>
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

- “Sailors” and “Reserves” relations for our
- We’ll use positional or named field notation results are ‘inherited’ from names of fields.

2.5. LOGICAL DATABASE DESIGN:**Logical DB Design: ER to Relational**

- Entity sets to tables:



```

CREATE TABLE Employees
(ssn CHAR(11),
name CHAR(20),
lot INTEGER,
PRIMARY KEY (ssn))
  
```

- Entity sets to tables:

Relationship Sets to Tables

- In translating a relationship set to a relation, attributes of the relation must include:
 - Keys for each participating entity set (as foreign keys).
- This set of attributes forms a *superkey* for the relation.
- All descriptive attributes.

Review: Key Constraints

- Each dept has at most one manager, according to the key constraint on Manages.

Translating ER Diagrams with Key Constraints

- Map relationship to a table:
 - Note that did is the key now!
 - Separate tables for Employees and Departments.
- Since each department has a unique manager, we could instead combine Manages and Departments.

```
CREATE TABLE Manages(  
  ssn CHAR(11),  
  did INTEGER,  
  since DATE,  
  PRIMARY KEY (did),  
  FOREIGN KEY (ssn) REFERENCES Employees,  
  FOREIGN KEY (did) REFERENCES Departments)
```

```
CREATE TABLE Dept_Mgr(  
  did INTEGER,  
  dname CHAR(20),  
  budget REAL,  
  ssn CHAR(11),  
  since DATE,  
  PRIMARY KEY (did),  
  FOREIGN KEY (ssn) REFERENCES Employees)
```

Review: Participation Constraints

- Does every department have a manager?
 - If so, this is a participation constraint: the participation of Departments in Manages is said to be *total* (vs. *partial*).
 - Every *did* value in Departments table must appear in a row of the Manages table (with a non-null *ssn* value!)
- Does every department have a manager?
 - If so, this is a participation constraint: the participation of Departments in Manages is said to be *total* (vs. *partial*).
 - Every *did* value in Departments table must appear in a row of the Manages table (with a non-null *ssn* value!)

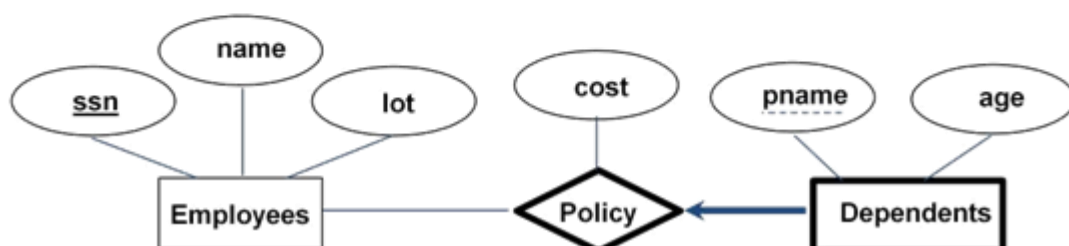
Participation Constraints in SQL

- We can capture participation constraints involving one entity set in a binary relationship, but little else (without resorting to CHECK constraints).

```
CREATE TABLE Dept_Mgr(
  did INTEGER,
  dname CHAR(20),
  budget REAL,
  ssn CHAR(11) NOT NULL,
  since DATE,
  PRIMARY KEY (did),
  FOREIGN KEY (ssn) REFERENCES Employees,
  ON DELETE NO ACTION)
```

Review: Weak Entities

- A **weak entity** can be identified uniquely only by considering the primary key of another (*owner*) entity.
 - Owner entity set and weak entity set must participate in a one-to-many relationship set (1 owner, many weak entities).
 - Weak entity set must have total participation in this *identifying* relationship set.



Translating Weak Entity Sets

- Weak entity set and identifying relationship set are translated into a single table.
- When the owner entity is deleted, all owned weak entities must also be deleted.

```
CREATE TABLE Dep_Policy (
  pname CHAR(20),
  age INTEGER,
  cost REAL,
  ssn CHAR(11) NOT NULL,
  PRIMARY KEY (pname, ssn),
  FOREIGN KEY (ssn) REFERENCES Employees,
  ON DELETE CASCADE)
```

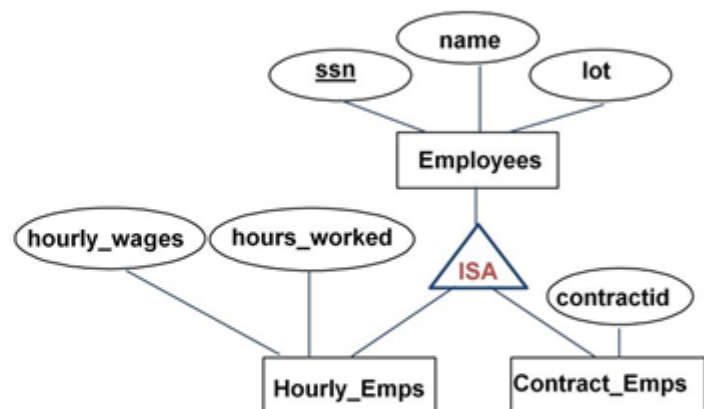
Review: ISA Hierarchies

- *Overlap constraints:* Can Joe be an Hourly_Emps as well as a Contract_Emps entity?(Allowed/disallowed)
- *Covering constraints:* Does every Employees entity also have to be an Hourly_Emps ora Contract_Emps entity? (Yes/no)

Review: ISA Hierarchies

❖ As in C++, or other PLs, attributes are inherited.

❖ If we declare A **ISA** B, every A entity is also considered to be a B entity.



- *Overlap constraints:* Can Joe be an Hourly_Emps as well as a Contract_Emps entity? (Allowed/disallowed)
- *Covering constraints:* Does every Employees entity also have to be an Hourly_Emps or a Contract_Emps entity? (Yes/no)

Translating ISA Hierarchies to Relations

- *General approach:*

- 3 relations: Employees, Hourly_Emps and Contract_Emps.

Hourly_Emps: Every employee is recorded in Employees. For hourly emps, extra info recorded in Hourly_Emps (*hourly_wages*, *hours_worked*, *ssn*); must delete Hourly_Emps tuple if referenced Employees tuple is deleted).

- Queries involving all employees easy, those involving just Hourly_Emps

require a join to get some attributes.

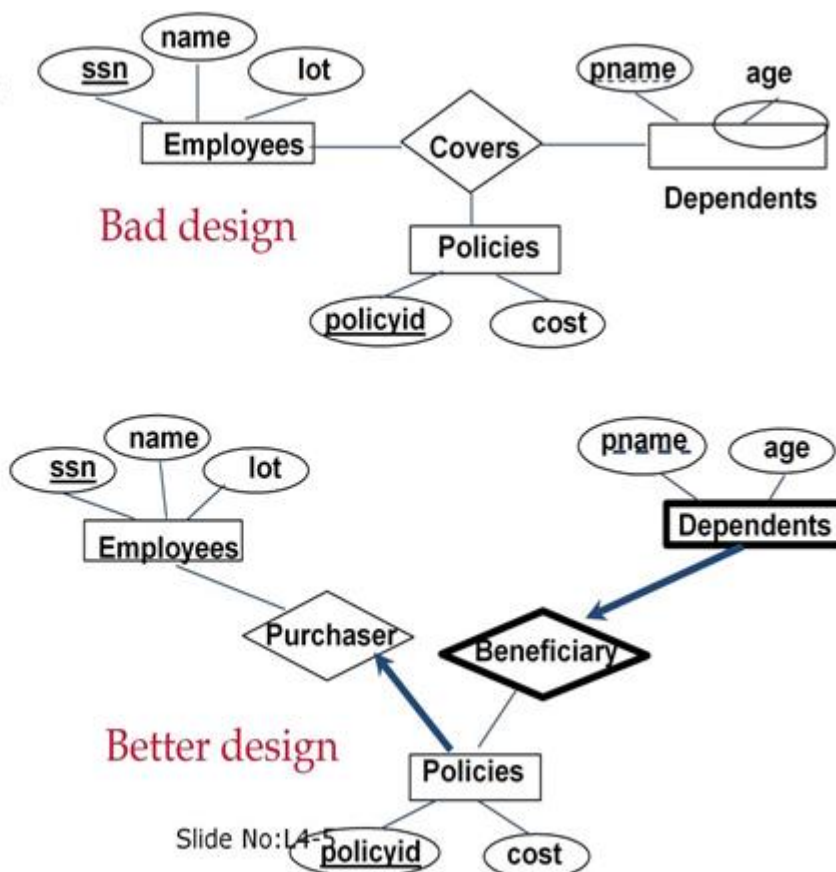
- Alternative: Just Hourly_Emps and Contract_Emps.

- *Hourly_Emps*: *ssn*, *name*, *lot*, *hourly_wages*, *hours_worked*.

- Each employee must be in one of these two subclasses.

Review: Binary vs. Ternary Relationships

- What are the additional constraints in the 2nd diagram?



Binary vs. Ternary Relationships (Contd.)

- The key constraints allow us to combine Purchaser with Policies and Beneficiary with Dependents.

```

CREATE TABLE Policies (
  policyid INTEGER,
  cost REAL,
  ssn CHAR(11) NOT NULL,
  PRIMARY KEY (policyid).
  FOREIGN KEY (ssn) REFERENCES Employees,
    ON DELETE CASCADE)

CREATE TABLE Dependents (
  pname CHAR(20),
  age INTEGER,
  policyid INTEGER,
  PRIMARY KEY (pname, policyid).
  FOREIGN KEY (policyid) REFERENCES Policies,
    ON DELETE CASCADE)

```

2.6.Introduction To Views:

Views and Security

- Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s).

– Given YoungStudents, but not Students or Enrolled, we can find students s who have are enrolled, but not the cid 's of the courses they are enrolled in.

- View Definition
 - A relation that is not of the conceptual model but is made visible to a relation".is called **a view**.

the expression defining v_1

- A view relation v_1 is said to *depend on* view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be *recursive* if it depends on itself.

- View Expansion

- A way to define the meaning of views defined in terms of other views.

• Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations.

- View expansion of an expression repeats the following replacement step:

repeat

Find any view relation v_i in e_1

Replace the view relation v_i by the expression defining v_i

until no more view relations are present in e_1

- As long as the view definitions are not recursive, this loop will terminate
- With Clause
- The **with** clause provides a way of defining a temporary view whose definition is available only to the query in which the **with** clause occurs.
- Find all accounts with the maximum balance

with *max_balance(value)* **as**

select max (*balance*)

from *account*

select *account_number*

from *account, max_balance*

where *account.balance = max_balance.value*

- Complex Queries using With Clause
- Find all branches where the total account deposit is greater than the average of the total account deposits at all branches.
- Update of a View
- Create a view of all loan data in the *loan* relation, hiding the *amount* attribute

create view *loan_branch* **as select**

loan_number, branch_name **from** *loan*

- Add a new tuple to *loan_branch*

insert into *loan_branch*

values ('L-37', 'Perryridge')

This insertion must be represented by the insertion of the tuple ('L-37', 'Perryridge',

null) into the *loan* relation

2.7.Destroying and Altering Tables and Views:

- Destroys the relation Students. The schema information *and* the tuples are deleted.
- **Adding and Deleting Tuples**
- Can insert a single tuple using:

What if Policies is a weak entity set?

- Views :A view is just a relation, but we store a *definition*, rather than a set of tuples.

RELATIONAL ALGEBRA

Relational algebra is one of the two formal query languages associated with the relational model. Queries in algebra are composed using a collection of operators. A fundamental property is that every operator in the algebra accepts (one or two) relation instances as arguments and returns a relation instance as the result.

Each relational query describes a step-by-step procedure for computing the desired answer, based on the order in which operators are applied in the query.

Selection and Projection

Relational algebra includes operators to *select* rows from a relation (σ) and to *project* columns (π). These operations allow us to manipulate data in a single relation. Consider the instance of the Sailors relation shown in Figure 4.2, denoted as S_2 . We can retrieve rows corresponding to expert sailors by using the σ operator. The expression,

$\sigma_{rating > 8}(S_2)$

The selection operator σ specifies the tuples to retain through a *selection condition*. In general, the selection condition is a boolean combination (i.e., an expression using the logical connectives $\wedge$ and $\vee$) of *terms* that have the form *attribute* op *constant* or *attribute1* op *attribute2*, where op is one of the comparison operators $<$, $<=$, $=$, $>=$, or $>$.

The projection operator π allows us to extract columns from a relation; for example, we can find out all sailor names and ratings by using π . The expression $\pi_{name, rating}(S_2)$

Suppose that we wanted to find out only the ages of sailors. The expression

$\pi_{age}(S_2)$

a single tuple with *age* = 35.0 appears in the result of the projection. This follows from the definition of a relation as a *set* of tuples. However, our discussion of relational algebra and calculus assumes that duplicate elimination is always done so that relations are always sets of tuples.

Set Operations

The following standard operations on sets are also available in relational algebra: *union* ($\cup$), *intersection* ($\cap$), *set-difference* ($-$), and *cross-product* ($\times$).

- Union: $R \cup S$ returns a relation instance containing all tuples that occur in *either* relation instance R or relation instance S (or both). R and S must be *union-compatible*, and the schema of the result is defined to be identical to the schema of R .
- Intersection: $R \cap S$ returns a relation instance containing all tuples that occur in *both* R and S . The relations R and S must be union-compatible, and the schema of the result is defined to be identical to the schema of R .
- Set-difference: $R - S$ returns a relation instance containing all tuples that occur in R but not in S . The relations R and S must be union-compatible, and the schema of the result is defined to be identical to the schema of R .
- Cross-product: $R \times S$ returns a relation instance whose schema contains all the fields of R (in the same order as they appear in R) followed by all the fields of S (in the same order as they appear in S). The result of $R \times S$ contains one tuple $\langle r, s \rangle$ (the concatenation of tuples r and s) for each pair of tuples $r \in R, s \in S$. The cross-product operation is sometimes called Cartesian product.

<i>sid</i>	<i>sname</i>	<i>rating</i>	<i>age</i>
31	Lubbe	8	55.5
58	Rusty	10	35.0

Figure 4.9 $S1 \cap S2$

<i>sid</i>	<i>sname</i>	<i>rating</i>	<i>age</i>
22	Dustin	7	45.0

Figure 4.10 $S1 - S2$

The result of the cross-product $S1 \times R1$ is shown in Figure 4.11. The fields in $S1 \times R1$ have the same domains as the corresponding fields in $R1$ and $S1$. In Figure 4.11 *sid* is listed in parentheses to emphasize that it is not an inherited field name; only the corresponding domain is inherited.

<i>(sid)</i>	<i>sname</i>	<i>rating</i>	<i>age</i>	<i>(sid)</i>	<i>bid</i>	<i>day</i>
22	Dustin	7	45.0	22	101	10/10/96
22	Dustin	7	45.0	58	103	11/12/96
31	Lubber	8	55.5	22	101	10/10/96
31	Lubber	8	55.5	58	103	11/12/96
58	Rusty	10	35.0	22	101	10/10/96
58	Rusty	10	35.0	58	103	11/12/96

Figure 4.11

 $S1 \times R1$

Renaming

We introduce a renaming operator ρ for this purpose. The expression $\rho(R(F), E)$ takes an arbitrary relational algebra expression E and returns an instance of a (new) relation called R . R contains the same tuples as the result of E , and has the same schema as E , but some fields are renamed. The field names in relation R are the same as in E , except for fields renamed in the *renaming list* F .

For example, the expression $\rho(C(1 \rightarrow sid1, 5 \rightarrow sid2), S1 \times R1)$ returns a relation that contains the tuples shown in Figure 4.11 and has the following schema: $C(sid1: integer, sname: string, rating: integer, age: real, sid2: integer, bid: integer, day: dates)$.

Joins

The *join* operation is one of the most useful operations in relational algebra and is the most commonly used way to combine information from two or more relations. Although a join can be defined as a cross-product followed by selections and projections, joins arise much more frequently in practice than plain cross-products. Joins have received a lot of attention, and there are several variants of the join operation.

Condition Joins

The most general version of the join operation accepts a *join condition* c and a pair of relation instances as arguments, and returns a relation instance. The *join condition* is identical to a *selection condition* in form. The operation is defined as follows:

$$R \Join_c S = \sigma_c(R \times S)$$

Thus $\Join$ is defined to be a cross-product followed by a selection. Note that the condition c can (and typically *does*) refer to attributes of both R and S .

<i>(sid)</i>	<i>sname</i>	<i>rating</i>	<i>age</i>	<i>(sid)</i>	<i>bid</i>	<i>day</i>
22	Dustin	7	45.0	58	103	11/12/96
31	Lubber	8	55.5	58	103	11/12/96

Figure 4.12 $S1 \Join_{S1.sid < R1.sid} R1$

Equijoin

A common special case of the join operation $R \Join S$ is when the *join condition* consists solely of equalities (connected by $\wedge$) of the form $R.name1 = S.name2$, that is, equalities between two fields in R and S . In this case, obviously, there is some redundancy in retaining both attributes in the result.

Natural Join

A further special case of the join operation $R \bowtie S$ is an equijoin in which equalities are specified on *all* fields having the same name in R and S . In this case, we can simply omit the join condition; the default is that the join condition is a collection of equalities on all common fields.

Division

The division operator is useful for expressing certain kinds of queries, for example: “Find the names of sailors who have reserved all boats.” Understanding how to use the basic operators of the algebra to define division is a useful exercise.

(Q1) Find the names of sailors who have reserved boat 103.

This query can be written as follows:

$$\pi_{\text{name}}((\sigma_{\text{bid}=103} \text{Reserves}) \bowtie \text{Sailors})$$

We first compute the set of tuples in Reserves with *bid* = 103 and then take the natural join of this set with Sailors. This expression can be evaluated on instances of Reserves and Sailors. Evaluated on the instances R_2 and S_3 , it yields a relation

(Q2) Find the names of sailors who have reserved a red boat.

$$\pi_{\text{name}}((\sigma_{\text{color}='red'} \text{Boats}) \bowtie \text{Reserves} \bowtie \text{Sailors})$$

This query involves a series of two joins. First we choose (tuples describing) red boats.

(Q3) Find the colors of boats reserved by Lubber.

$$\pi_{\text{color}}((\sigma_{\text{name}='Lubber'} \text{Sailors}) \bowtie \text{Reserves} \bowtie \text{Boats})$$

This query is very similar to the query we used to compute sailors who reserved red boats. On instances B_1 , R_2 , and S_3 , the query will return the colors green and red.

(Q4) Find the names of sailors who have reserved at least one boat.

$$\pi_{\text{name}}(\text{Sailors} \bowtie \text{Reserves})$$

(Q5) Find the names of sailors who have reserved a red or a green boat.

$$\rho(\text{Tempboats}, (\sigma_{\text{color}='red'} \text{Boats}) \cup (\sigma_{\text{color}='green'} \text{Boats}))$$

$$\pi_{\text{name}}(\text{Tempboats} \bowtie \text{Reserves} \bowtie \text{Sailors})$$

(Q6) Find the names of sailors who have reserved a red and a green boat

$$\rho(\text{Tempboats2}, (\sigma_{\text{color}='red'} \text{Boats}) \cap (\sigma_{\text{color}='green'} \text{Boats}))$$

$$\pi_{\text{name}}(\text{Tempboats2} \bowtie \text{Reserves} \bowtie \text{Sailors})$$

However, this solution is incorrect —it instead tries to compute sailors who have re-served a boat that is both red and green.

$$\rho(\text{Tempred}, \pi_{\text{sid}}((\sigma_{\text{color}='red'} \text{Boats}) \bowtie \text{Reserves}))$$

$$\rho(\text{Tempgreen}, \pi_{\text{sid}}((\sigma_{\text{color}='green'} \text{Boats}) \bowtie \text{Reserves}))$$

$$\pi_{\text{name}}((\text{Tempred} \cap \text{Tempgreen}) \bowtie \text{Sailors})$$

(Q7) Find the names of sailors who have reserved at least two boats.

$$\rho(\text{Reservations}, \pi_{\text{sid}, \text{sname}, \text{bid}}(\text{Sailors} \bowtie \text{Reserves}))$$

$$\rho(\text{Reservationpairs}(1 \rightarrow \text{sid1}, 2 \rightarrow \text{sname1}, 3 \rightarrow \text{bid1}, 4 \rightarrow \text{sid2},$$

$$5 \rightarrow \text{sname2}, 6 \rightarrow \text{bid2}), \text{Reservations} \times \text{Reservations})$$

$$\pi_{\text{sname1}}(\sigma(\text{sid1} = \text{sid2}) \cap (\text{bid1} = \text{bid2}) \text{Reservationpairs})$$

(Q8) Find the sids of sailors with age over 20 who have not reserved a red boat.

$$\pi_{\text{sid}}(\sigma_{\text{age} > 20} \text{Sailors}) - \pi_{\text{sid}}((\sigma_{\text{color} = 'red'} \text{Boats}) \bowtie \text{Reserves} \bowtie \text{Sailors})$$

This query illustrates the use of the set-difference operator. Again, we use the fact that *sid* is the key for Sailors.

(Q9) Find the names of sailors who have reserved all boats.

The use of the word *all* (or *every*) is a good indication that the division operation might be applicable:

$$\rho(\text{Tempids}, (\pi_{\text{sid}, \text{bid}} \text{Reserves}) / (\pi_{\text{bid}} \text{Boats}))$$

$$\pi_{\text{name}}(\text{Tempids} \bowtie \text{Sailors})$$

(Q10) Find the names of sailors who have reserved all boats called Interlake.

$$\rho(\text{Tempids}, (\pi_{\text{sid}, \text{bid}} \text{Reserves}) / (\pi_{\text{bid}}(\sigma_{\text{name} = 'Interlake'} \text{Boats})))$$

$$\pi_{\text{name}}(\text{Tempids} \bowtie \text{Sailors})$$

RELATIONAL CALCULUS

Relational calculus is an alternative to relational algebra. In contrast to the algebra, which is procedural, the calculus is nonprocedural, or *declarative*, in that it allows us to describe the set of answers without being explicit about how they should be computed.

Tuple Relational Calculus

A tuple variable is a variable that takes on tuples of a particular relation schema as values. That is, every value assigned to a given tuple variable has the same number and type of fields.

(Q11) Find all sailors with a rating above 7.

$$\{S \mid S \in \text{Sailors} \wedge S.\text{rating} > 7\}$$

with respect to the given database instance, *F* evaluates to (or simply 'is') true if one of the following holds:

- *F* is an atomic formula $R \in \text{Rel}$, and *R* is assigned a tuple in the instance of relation *Rel*.

- F is a comparison $R.a \text{ op } S.b$, $R.a \text{ op constant}$, or $\text{constant op } R.a$, and the tuples assigned to R and S have field values $R.a$ and $S.b$ that make the comparison true.
- F is of the form $\neg p$, and p is not true; or of the form $p \wedge q$, and both p and q are true; or of the form $p \vee q$, and one of them is true, or of the form $p \Rightarrow q$ and q is true whenever⁴ p is true.
- F is of the form $\exists R(p(R))$, and there is some assignment of tuples to the free variables in $p(R)$, including the variable R ,⁵ that makes the formula $p(R)$ true.
- F is of the form $\forall R(p(R))$, and there is some assignment of tuples to the free variables in $p(R)$ that makes the formula $p(R)$ true no matter what tuple is assigned to R .

(Q12) Find the names and ages of sailors with a rating above 7.

$$\{P \mid \exists S \in \text{Sailors}(S.\text{rating} > 7 \wedge P.\text{name} = S.\text{sname} \wedge P.\text{age} = S.\text{age})\}$$

This query illustrates a useful convention: P is considered to be a tuple variable with exactly two fields, which are called *name* and *age*, because these are the only fields of P that are mentioned and P does not range over any of the relations in the query; that is, there is no subformula of the form $P \in \text{Relname}$.

(Q13) Find the sailor name, boat id, and reservation date for each reservation

$$\{P \mid \exists R \in \text{Reserves} \quad \exists S \in \text{Sailors} \\ (R.\text{sid} = S.\text{sid} \wedge P.\text{bid} = R.\text{bid} \wedge P.\text{day} = R.\text{day} \wedge P.\text{sname} = S.\text{sname})\}$$

(Q1) Find the names of sailors who have reserved boat 103.

$$\{P \mid \exists S \in \text{Sailors} \exists R \in \text{Reserves}(R.\text{sid} = S.\text{sid} \wedge R.\text{bid} = 103 \wedge P.\text{sname} = S.\text{sname})\}$$

This query can be read as follows: “Retrieve all sailor tuples for which there exists a tuple in Reserves, having the same value in the *sid* field, and with *bid* = 103.”

(Q2) Find the names of sailors who have reserved a red boat.

$$\{P \mid \exists S \in \text{Sailors} \exists R \in \text{Reserves}(R.\text{sid} = S.\text{sid} \wedge P.\text{sname} = S.\text{sname} \\ \wedge \exists B \in \text{Boats}(B.\text{bid} = R.\text{bid} \wedge B.\text{color} = \text{'red'}))\}$$

This query can be read as follows: "Retrieve all sailor tuples S for which there exist tuples R in Reserves and B in Boats such that $S.sid = R.sid$, $R.bid = B.bid$, and $B.color = 'red'$."

(Q7) Find the names of sailors who have reserved at least two boats. $\{P \mid \exists S \in \text{Sailors} \exists R1 \in \text{Reserves} \exists R2 \in \text{Reserves} (S.sid = R1.sid \wedge R1.sid = R2.sid \wedge R1.bid \neq R2.bid \wedge P.sname = S.sname)\}$

(Q9) Find the names of sailors who have reserved all boats.

$$\{P \mid \exists S \in \text{Sailors} \quad \forall B \in \text{Boats} \\ (\exists R \in \text{Reserves}(S.sid = R.sid \wedge R.bid = B.bid \wedge P.sname = S.sname))\}$$

(Q14) Find sailors who have reserved all red boats.

$$\{S \mid S \in \text{Sailors} \in \forall B \in \text{Boats} \\ (B.color = 'red' \rightarrow (\exists R \in \text{Reserves}(S.sid = R.sid \wedge R.bid = B.bid)))\}$$

Domain Relational Calculus

A domain variable is a variable that ranges over the values in the domain of some attribute (e.g., the variable can be assigned an integer if it appears in an attribute

whose domain is the set of integers). A DRC query has the form $\{ \langle x_1, x_2, \dots, x_n \rangle \mid p(\langle x_1, x_2, \dots, x_n \rangle) \}$, where each x_i is either a domain variable or a constant and $p(\langle x_1, x_2, \dots, x_n \rangle)$ denotes a DRC formula whose only free variables are the variables among the x_i , $1 \leq i \leq n$.

The result of this query is the set of all tuples $\langle x_1, x_2, \dots, x_n \rangle$ for which the formula evaluates to true.

(Q1) Find the names of sailors who have reserved boat 103.

$$\{ \langle N \rangle \mid \exists I, T, A (\langle I, N, T, A \rangle \in \text{Sailors} \\ \wedge \exists Ir, Br, D (\langle Ir, Br, D \rangle \in \text{Reserves} \wedge Ir = I \wedge Br = 103)) \}$$

(Q2) Find the names of sailors who have reserved a red boat.

$$\{ \langle N \rangle \mid \exists I, T, A (\langle I, N, T, A \rangle \in \text{Sailors} \\ \wedge \exists Ir, Br, D (\langle Ir, Br, D \rangle \in \text{Reserves} \wedge \exists \langle Br, BN, 'red' \rangle \in \text{Boats})) \}$$

(Q7) Find the names of sailors who have reserved at least two boats.

$$\{ \langle N \rangle \mid \exists I, T, A (\langle I, N, T, A \rangle \in \text{Sailors} \wedge \exists Br1, Br2, D1, D2 (\langle I, Br1, D1 \rangle \in \text{Reserves} \wedge \langle I, Br2, D2 \rangle \in \text{Reserves} \wedge Br1 \neq Br2)) \}$$

(Q9) Find the names of sailors who have reserved all boats.

$$\{ \langle N \rangle \mid \exists I, T, A (\langle I, N, T, A \rangle \in \text{Sailors} \wedge \\ \forall B, BN, C (\langle B, BN, C \rangle \in \text{Boats} \rightarrow$$

$$(\exists \langle Ir, Br, D \rangle \in \text{Reserves} (I = Ir \wedge Br = B))) \}$$

UNIT 3

SQL: QUERIES, CONSTRAINTS, TRIGGERS

THE FORM OF A BASIC SQL QUERY₁

This section presents the syntax of a simple SQL query and explains its meaning through a *conceptual evaluation strategy*. A conceptual evaluation strategy is a way to evaluate the query that is intended to be easy to understand, rather than efficient. A DBMS would typically execute a query in a different and more efficient way.

<i>Sid</i>	<i>sname</i>	<i>rating</i>	<i>age</i>
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Figure 5.1 An Instance *S 3* of Sailors

<i>sid</i>	<i>bid</i>	<i>day</i>
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Figure 5.2 An Instance *R2* of Reserves

<i>bid</i>	<i>bname</i>	<i>color</i>
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

(Q15) Find the names and ages of all sailors.

```
SELECT DISTINCT S.sname, S.age FROM Sailors S
```

The answer to this query with and without the keyword DISTINCT on instance S3 of Sailors is shown in Figures 5.4 and 5.5. The only difference is that the tuple for Horatio appears twice if DISTINCT is omitted; this is because there are two sailors called Horatio and age 35.

(Q11) Find all sailors with a rating above 7.

```
SELECT S.sid, S.sname, S.rating, S.age FROM Sailors AS S WHERE S.rating > 7
```

(Q16) Find the sids of sailors who have reserved a red boat.

```
SELECT R.sid FROM Boats B, Reserves R WHERE B.bid = R.bid AND B.color = 'red'
```

(Q2) Find the names of sailors who have reserved a red boat.

```
SELECT S.sname FROM Sailors S, Reserves R, Boats B WHERE S.sid = R.sid AND  
R.bid = B.bid AND B.color = 'red'
```

(Q3) Find the colors of boats reserved by Lubber.

```
SELECT B.color FROM Sailors S, Reserves R, Boats B WHERE S.sid = R.sid  
AND R.bid = B.bid AND S.sname = 'Lubber'
```

(Q4) Find the names of sailors who have reserved at least one boat.

```
SELECT S.sname FROM Sailors S, Reserves R WHERE S.sid = R.sid
```

Expressions and Strings in the SELECT Command

SQL supports a more general version of the select-list than just a list of columns. Each item in a select-list can be of the form expression AS column name, where expression is any arithmetic or string expression over column names (possibly prefixed by range variables) and constants.

(Q5) Compute increments for the ratings of persons who have sailed two different boats on the same day.

```
SELECT S.sname, S.rating+1 AS rating FROM Sailors S, Reserves R1, Reserves R2
WHERE S.sid = R1.sid AND S.sid = R2.sid AND R1.day = R2.day AND R1.bid <>
R2.bid
```

Also, each item in a *qualification* can be as general as *expression1 = expression2*.

```
SELECT S1.sname AS name1, S2.sname AS name2 FROM Sailors S1, Sailors
S2 WHERE 2*S1.rating = S2.rating-1.
```

(Q6) Find the ages of sailors whose name begins and ends with B and has at least three characters.

```
SELECT S.age FROM Sailors S WHERE S.sname LIKE 'B %B'
```

The only such sailor is Bob, and his age is 63.5.

UNION, INTERSECT, AND EXCEPT

SQL provides three set-manipulation constructs that extend the basic query form pre-sented earlier. Since the answer to a query is a multiset of rows, it is natural to consider the use of operations such as union, intersection, and difference. SQL supports these operations under the names UNION, INTERSECT, and EXCEPT.⁴ SQL also provides other set operations: IN (to check if an element is in a given set), op ANY, op ALL (to compare a value with the elements in a given set, using comparison operator op), and EXISTS (to check if a set is empty). IN and EXISTS can be prefixed by NOT, with the obvious modification to their meaning. We cover UNION, INTERSECT, and EXCEPT in this section. Consider the following query:

(Q1) Find the names of sailors who have reserved both a red and a green boat.

```
SELECT S.sname FROM Sailors S, Reserves R1, Boats B1, Reserves R2, Boats
B2 WHERE S.sid = R1.sid AND R1.bid = B1.bid AND S.sid = R2.sid AND R2.bid
= B2.bid AND B1.color='red' AND B2.color = 'green'
```

(Q2) Find the sids of all sailors who have reserved red boats but not green boats.

```
SELECT S.sid FROM Sailors S, Reserves R, Boats B
WHERE S.sid = R.sid AND R.bid = B.bid AND B.color = 'red' EXCEPT
SELECT S2.sid FROM Sailors S2, Reserves R2, Boats B2 WHERE
S2.sid = R2.sid AND R2.bid = B2.bid AND B2.color = 'green'
```

NESTED QUERIES

A nested query is a query that has another query embedded within it; the embedded query is called a subquery.

(Q1) Find the names of sailors who have reserved boat 103.

```
SELECT S.sname
FROM   Sailors S
WHERE  S.sid IN ( SELECT R.sid
                  FROM   Reserves R
                  WHERE  R.bid = 103 )
```

(Q2) Find the names of sailors who have reserved a red boat.

```
SELECT  S.sname
FROM    Sailors S
WHERE   S.sid IN ( SELECT R.sid
                  FROM    Reserves R
                  WHERE   R.bid IN ( SELECT B.bid
                                    FROM   Boats B
                                    WHERE  B.color = 'red' ) )
```

(Q3) Find the names of sailors who have not reserved a red boat.

```
SELECT  S.sname
FROM    Sailors S
WHERE   S.sid NOT IN ( SELECT R.sid
                     FROM    Reserves R
                     WHERE   R.bid IN ( SELECT B.bid
                                       FROM   Boats B
                                       WHERE  B.color = 'red' ) )
```

Correlated Nested Queries

In the nested queries that we have seen thus far, the inner subquery has been completely independent of the outer query:

(Q1) Find the names of sailors who have reserved boat number 103.

```
SELECT S.sname
FROM   Sailors S
WHERE  EXISTS ( SELECT *
                FROM   Reserves R
                WHERE  R.bid = 103
                    AND R.sid = S.sid )
```

Set-Comparison Operators

(Q1) Find sailors whose rating is better than some sailor called Horatio.

```
SELECT S.sid
FROM   Sailors S
WHERE  S.rating > ANY ( SELECT S2.rating
                       FROM   Sailors S2
                       WHERE  S2.sname = 'Horatio' )
```

(Q2) Find the sailors with the highest rating .

```
SELECT S.sid
FROM   Sailors S
WHERE  S.rating >= ALL ( SELECT S2.rating
                       FROM   Sailors S2 )
```

More Examples of Nested Queries

(Q1) Find the names of sailors who have reserved both a red and a green boat.

```
SELECT S.sname
FROM   Sailors S, Reserves R, Boats B
WHERE  S.sid = R.sid AND R.bid = B.bid AND B.color = 'red'
      AND S.sid IN ( SELECT S2.sid
                    FROM   Sailors S2, Boats B2, Reserves R2
                    WHERE  S2.sid = R2.sid AND R2.bid = B2.bid
                        AND B2.color = 'green' )
```

Q9) Find the names of sailors who have reserved all boats.

```
SELECT S.sname
FROM   Sailors S
WHERE  NOT EXISTS (( SELECT B.bid
                     FROM   Boats B )
                  EXCEPT
                  (SELECT R.bid
                   FROM   Reserves R
                   WHERE  R.sid = S.sid ))
```

AGGREGATE OPERATORS

We now consider a powerful class of constructs for computing *aggregate values* such as MIN and SUM.

1. COUNT ([DISTINCT] A): The number of (unique) values in the A column.
2. SUM ([DISTINCT] A): The sum of all (unique) values in the A column.
3. AVG ([DISTINCT] A): The average of all (unique) values in the A column.
4. MAX (A): The maximum value in the A column.
5. MIN (A): The minimum value in the A column.

(Q1) Find the average age of all sailors.

```
SELECT AVG (S.age)
FROM   Sailors S
```

(Q2) Find the average age of sailors with a rating of 10.

```
SELECT AVG (S.age)
FROM   Sailors S
WHERE  S.rating = 10
```

```
SELECT  S.sname,  MAX  (S.age)
FROM   Sailors S
```

Q3) Count the number of sailors.

```
SELECT COUNT (*)
FROM   Sailors S
```

The GROUP BY and HAVING Clauses

we want to apply aggregate operations to each of a number of groups of rows in a relation, where the number of groups depends on the relation instance (i.e., is not known in advance).

(Q31) Find the age of the youngest sailor for each rating level.

```
SELECT MIN (S.age)
FROM   Sailors S
WHERE  S.rating = i
```

(Q32) Find the age of the youngest sailor who is eligible to vote (i.e., is at least 18 years old) for each rating level with at least two such sailors.

```
SELECT  S.rating, MIN (S.age) AS minage
GROUP BY S.rating
HAVING  COUNT (*) > 1
```

More Examples of Aggregate Queries

(Q3) For each red boat, find the number of reservations for this boat.

```
SELECT B.bid, COUNT (*) AS sailorcount
FROM Boats B, Reserves R
WHERE R.bid = B.bid AND B.color = 'red'
GROUP BY B.bid
```

```
SELECT B.bid, COUNT (*) AS sailorcount
FROM Boats B, Reserves R
WHERE R.bid = B.bid
GROUP BY B.bid
HAVING B.color = 'red'
```

(Q4) Find the average age of sailors for each rating level that has at least two sailors.

```
SELECT  S.rating, AVG (S.age) AS avgage
FROM    Sailors S
GROUP BY S.rating
HAVING  COUNT (*) > 1
```

(Q5) Find the average age of sailors who are of voting age (i.e., at least 18 years old) for each rating level that has at least two sailors.

```
SELECT  S.rating, AVG ( S.age ) AS avgage
FROM    Sailors S
WHERE   S.age >= 18
GROUP BY S.rating
HAVING  1 < ( SELECT COUNT (*)
```

FROM Sailors S2 WHERE S.rating = S2.rating

(Q6) Find the average age of sailors who are of voting age (i.e., at least 18 years old) for each rating level that has at least two such sailors.

```
SELECT  S.rating, AVG ( S.age ) AS avgage
FROM    Sailors S
WHERE   S. age > 18
GROUP BY S.rating

HAVING 1 < ( SELECT COUNT (*)
              FROM  Sailors S2
              WHERE S.rating = S2.rating AND S2.age >= 18 )
```

The above formulation of the query reflects the fact that it is a variant of Q35. The answer to Q36 on instance S3 is shown in Figure 5.16. It differs from the answer to Q35 in that there is no tuple for rating 10, since there is only one tuple with rating 10 and $age \geq 18$.

```
SELECT  S.rating, AVG ( S.age ) AS avgage
FROM    Sailors S
WHERE   S. age > 18
GROUP BY S.rating
HAVING  COUNT (*) > 1
```

This formulation of Q36 takes advantage of the fact that the WHERE clause is applied before grouping is done; thus, only sailors with $age > 18$ are left when grouping is done. It is instructive to consider yet another way of writing this query:

```
SELECT Temp.rating, Temp.avgage
FROM ( SELECT S.rating, AVG ( S.age ) AS
        avgage, COUNT (*) AS
        ratingcount
      FROM Sailors S WHERE S. age > 18 GROUP BY S.rating ) AS Temp
WHERE Temp.ratingcount > 1
```

NULL VALUES

we have assumed that column values in a row are always known. In practice column values can be unknown. For example, when a sailor, say Dan, joins a yacht club, he may not yet have a rating assigned. Since the definition for the Sailors table has a *rating* column, what row should we insert for Dan? What is needed here is a special value that denotes *unknown*.

SQL provides a special column value called *null* to use in such situations. We use *null* when the column value is either *unknown* or *inapplicable*. Using our Sailor table definition, we might enter the row $(98, \text{Dan}, \text{null}, 39)$ to represent Dan. The presence of *null* values complicates many issues, and we consider the impact of *null* values on SQL in this section.

Comparisons Using Null Values

Consider a comparison such as $\text{rating} = 8$. If this is applied to the row for Dan, is this condition true or false? Since Dan's rating is unknown, it is reasonable to say that this comparison should evaluate to the value unknown.

SQL also provides a special comparison operator IS NULL to test whether a column value is *null*; for example, we can say rating IS NULL , which would evaluate to true on the row representing Dan. We can also say $\text{rating IS NOT NULL}$, which would evaluate to false on the row for Dan.

Logical Connectives AND, OR, and NOT

Now, what about boolean expressions such as $\text{rating} = 8 \text{ OR } \text{age} < 40$ and $\text{rating} = 8 \text{ AND } \text{age} < 40$? Considering the row for Dan again, because $\text{age} < 40$, the first expression evaluates to true regardless of the value of *rating*, but what about the second? We can only say unknown.

INTRODUCTION TO VIEWS

A view is a table whose rows are not explicitly stored in the database but are computed as needed from a view definition. Consider the Students and Enrolled relations.

```
CREATE VIEW B-Students (name, sid, course)
AS SELECT S.sname, S.sid, E.cid
FROM   Students S, Enrolled E
WHERE  S.sid = E.sid AND E.grade = 'B'
```

This view can be used just like a base table, or explicitly stored table, in defining new queries or views.

DESTROYING/ALTERING TABLES AND VIEWS

If we decide that we no longer need a base table and want to destroy it (i.e., delete all the rows *and* remove the table definition information), we can use the `DROP TABLE` command. For example, `DROP TABLE Students RESTRICT` destroys the `Students` table unless some view or integrity constraint refers to `Students`; if so, the command fails. If the keyword `RESTRICT` is replaced by `CASCADE`, `Students` is dropped and any referencing views or integrity constraints are (recursively) dropped as well; one of these two keywords must always be specified. A view can be dropped using the `DROP VIEW` command, which is just like `DROP TABLE`.

`ALTER TABLE` modifies the structure of an existing table. To add a column called *maiden-name* to `Students`, for example, we would use the following command:

```
ALTER TABLE Students
  ADD COLUMN maiden-name CHAR(10)
```

TRIGGERS

A trigger is a procedure that is automatically invoked by the DBMS in response to specified changes to the database, and is typically specified by the DBA. A database that has a set of associated triggers is called an active database. A trigger description contains three parts:

Event: A change to the database that activates the trigger.

Condition: A query or test that is run when the trigger is activated.

Action: A procedure that is executed when the trigger is activated and its condition is true.

A trigger *action* can examine the answers to the query in the condition part of the trigger, refer to old and new values of tuples modified by the statement activating the trigger, execute new queries, and make changes to the database.

The examples shown in Figure 5.19, written using Oracle 7 Server syntax for defining triggers, illustrate the basic concepts behind triggers. (The SQL:1999 syntax for these triggers is similar; we will see an example using SQL:1999 syntax shortly.) The trigger called *init count* initializes a counter variable before every execution of an INSERT statement that adds tuples to the Students relation. The trigger called *incr count* increments the counter for each inserted tuple that satisfies the condition *age < 18*.

```
CREATE TRIGGER init count BEFORE INSERT ON Students /* Event */
  DECLARE
    count INTEGER;
  BEGIN                                     /* Action */
    count := 0;
  END

CREATE TRIGGER incr count AFTER INSERT ON Students /* Event */
  WHEN (new.age < 18) /* Condition; 'new' is just-inserted tuple */
  FOR EACH ROW
  BEGIN /* Action; a procedure in Oracle's PL/SQL syntax */ count :=
    count + 1;
  END
```

(identifying the modified table, Students, and the kind of modifying statement, an INSERT), and the third field is the number of inserted Students tuples with *age < 18*. (The trigger in Figure 5.19 only computes the count; an additional trigger is required to insert the appropriate tuple into the statistics table.)

```
CREATE TRIGGER set count AFTER INSERT ON Students /* Event */
  REFERENCING NEW TABLE AS InsertedTuples
  FOR EACH STATEMENT
  INSERT                                     /* Action */
    INTO StatisticsTable(ModifiedTable, ModificationType, Count) SELECT
    'Students', 'Insert', COUNT *
  FROM InsertedTuples I
  WHERE I.age < 18
```

INTRODUCTION TO SCHEMA REFINEMENT:

The Evils of Redundancy

- *Redundancy* is at the root of several problems associated with relational schemas:
 - redundant storage, insert/delete/update anomalies
- Integrity constraints, in particular *functional dependencies*, can be used to identify schemas with such problems and to suggest refinements.
- Main refinement technique: *decomposition* (replacing ABCD with, say, AB and BCD, or ACD and ABD).
- Decomposition should be used judiciously:
 - Is there reason to decompose a relation?
 - What problems (if any) does the decomposition cause?

Problems Caused by Redundancy:

- Storing the same information **redundantly**, that is, in more than one place within a database, can lead to several problems:
- **Redundant storage:** Some information is stored repeatedly.
- **Update anomalies:** If one copy of such repeated data is updated, an inconsistency is created unless all copies are similarly updated.
- **Insertion anomalies:** It may not be possible to store some information unless some other information is stored as well.
- **Deletion anomalies:** It may not be possible to delete some information without losing some other information as well.
- Consider a relation obtained by translating a variant of the Hourly Emps entity set

NORMALIZATION:

SCHEMA REFINEMENT

We now present an overview of the problems that schema refinement is intended to address and a refinement approach based on decompositions. Redundant storage of information is the root cause of these problems. Although decomposition can eliminate redundancy, it can lead to problems of its own and should be used with caution.

Problems Caused by Redundancy:

Storing the same information redundantly, that is, in more than one place within a database, can lead to several problems:

Redundant storage: Some information is stored repeatedly.

Update anomalies: If one copy of such repeated data is updated, an inconsistency is created unless all copies are similarly updated.

Insertion anomalies: It may not be possible to store some information unless some other information is stored as well.

Deletion anomalies: It may not be possible to delete some information without losing some other information as well.

Consider a relation obtained by translating a variant of the Hourly Emps entity set :

Hourly Emps(ssn, name, lot, rating, hourly wages, hours worked)

If we delete all tuples with a given rating value (e.g., we delete the tuples for Smethurst and Guldu) we lose the association between that rating value and its hourly wage value (a deletion anomaly).

Ideally, we want schemas that do not permit redundancy, but at the very least we want to be able to identify schemas that do allow redundancy. Even if we choose to accept a schema with some of these drawbacks, perhaps owing to performance considerations, we want to make an informed decision.

Use of Decompositions

Intuitively, redundancy arises when a relational schema forces an association between attributes that is not natural. Functional dependencies (and, for that matter, other ICs) can be used to identify such situations and to suggest refinements to the schema.

We can deal with the redundancy in Hourly Emps by decomposing it into two relations:

Hourly Emps2(ssn, name, lot, rating, hours worked)

Wages(rating, hourly wages)

Unless we are careful, decomposing a relation schema can create more problems than it solves. Two important questions must be asked repeatedly:

1. Do we need to decompose a relation?
2. What problems (if any) does a given decomposition cause?

If a relation schema is in one of these normal forms, we know that certain kinds of problems cannot arise. Considering the normal form of a given relation schema can help us to decide whether or not to decompose it further. If we decide that a relation schema must be decomposed further, we must choose a particular decomposition (i.e., a particular collection of smaller relations to replace the given relation).

FUNCTIONAL DEPENDENCIES

A functional dependency (FD) is a kind of IC that generalizes the concept of a *key*. Let R be a relation schema and let X and Y be nonempty sets of attributes in R . We

say that an instance r of R satisfies the FD $X \twoheadrightarrow Y$ ¹ if the following holds for every pair of tuples t_1 and t_2 in r :

If $t_1.X = t_2.X$, then $t_1.Y = t_2.Y$.

A primary key constraint is a special case of an FD. The attributes in the key play the role of X , and the set of all attributes in the relation plays the role of Y . Note, however, that the definition of an FD does not require that the set X be minimal; the additional minimality condition must be met for X to be a key. If $X \twoheadrightarrow Y$ holds, where

Y is the set of all attributes, and there is some subset V of X such that $V \twoheadrightarrow Y$ holds, then X is a *super key*; if V is a strict subset of X , then X is not a key.

In the rest of this chapter, we will see several examples of FDs that are not key constraints.

REASONING ABOUT FUNCTIONAL DEPENDENCIES

The discussion up to this point has highlighted the need for techniques that allow us to carefully examine and further refine relations obtained through ER design (or, for that matter, through other approaches to conceptual design).

Given a set of FDs over a relation schema R , there are typically several additional

FDs that hold over R whenever all of the given FDs hold. As an example, consider:

Workers(ssn, name, lot, did, since)

15.4.1 Closure of a Set of FDs

The set of all FDs implied by a given set F of FDs is called the closure of F and is

denoted as F^+ . An important question is how we can infer, or compute, the closure of a given set F of FDs. The answer is simple and elegant. The following three rules, called Armstrong's Axioms, can be applied repeatedly to infer all FDs implied by a set F of FDs. We use X , Y , and Z to denote *sets* of attributes over a relation schema R :

- Reflexivity: If $X \supseteq Y$, then $X \twoheadrightarrow Y$.
- Augmentation: If $X \twoheadrightarrow Y$, then $XZ \twoheadrightarrow YZ$ for any Z .
- Transitivity: If $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, then $X \twoheadrightarrow Z$.

Armstrong's Axioms are sound in that they generate only FDs in F^+ when applied to a set F of FDs. They are complete in that repeated application of these rules will generate all FDs in the closure F^+ . (We will not prove these claims.) It is convenient to use some additional rules while reasoning about F^+ :

- Union: If $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z$, then $X \twoheadrightarrow YZ$.
- Decomposition: If $X \twoheadrightarrow YZ$, then $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z$.

These additional rules are not essential; their soundness can be proved using Armstrong's Axioms.

use a more elaborate version of the Contracts relation:

Contracts (contractid, supplierid, projectid, deptid, partid, qty, value)

The following ICs are known to hold:

1. The contract id C is a key: $C \twoheadrightarrow CSJDPQV$.
2. A project purchases a given part using a single contract: $JP \twoheadrightarrow C$.
3. A department purchases at most one part from a supplier: $SD \twoheadrightarrow P$.

NORMAL FORMS

Given a relation schema, we need to decide whether it is a good design or whether we need to decompose it into smaller relations. Such a decision must be guided by an understanding of what problems, if any, arise from the current schema. To provide such guidance, several normal forms have been proposed. If a relation schema is in one of these normal forms, we know that certain kinds of problems cannot arise.

The normal forms based on FDs are first normal form (1NF), second normal form (2NF), third normal form (3NF), and Boyce-Codd normal form (BCNF).

First Normal Form (1NF)

For a table to be in the First Normal Form, it should follow the following 4 rules:

Rules :

The first normal form expects you to follow a few simple rules while designing your database, and they are:

Rule 1: Single Valued Attributes

Each column of your table should be single valued which means they should not contain multiple values. We will explain this with help of an example later, let's see the other rules for now.

Rule 2: Attribute Domain should not change

This is more of a "Common Sense" rule. In each column the values stored must be of the same kind or type.

For example: If you have a column dob to save date of births of a set of people, then you cannot or you must not save 'names' of some of them in that column along with 'date of birth' of others in that column. It should hold only 'date of birth' for all the records/rows.

Rule 3: Unique name for Attributes/Columns

This rule expects that each column in a table should have a unique name. This is to avoid confusion at the time of retrieving data or performing any other operation on the stored data.

If one or more columns have same name, then the DBMS system will be left confused.

Rule 4: Order doesn't matters

This rule says that the order in which you store the data in your table doesn't matter.

Example:

Although all the rules are self explanatory still let's take an example where we will create a table to store student data which will have student's roll no., their name and the name of subjects they have opted for.

Here is our table, with some sample data added to it.

roll_no	name	subject
101	Akon	OS, CN
103	Ckon	Java
102	Bkon	C, C++

Our table already satisfies 3 rules out of the 4 rules, as all our column names are unique, we have stored data in the order we wanted to and we have not inter-mixed different type of data in columns.

But out of the 3 different students in our table, 2 have opted for more than 1 subject. And we have stored the subject names in a single column. But as per the 1st Normal form each column.

must contain atomic value.

How to solve this Problem?

It's very simple, because all we have to do is break the values into atomic values.

Here is our updated table and it now satisfies the First Normal Form.

roll_no	name	subject
101	Akon	OS
101	Akon	CN
103	Ckon	Java
102	Bkon	C
102	Bkon	C++

by doing so, although a few values are getting repeated but values for the subject column are now atomic for each record/row.

Using the First Normal Form, data redundancy increases, as there will be many columns with same data in multiple rows but each row as a whole will be unique.

Second normal form (2NF)

A table is said to be in 2NF if both the following conditions hold:

- Table is in 1NF (First normal form)
- No non-prime attribute is dependent on the proper subset of any candidate key of table.

An attribute that is not part of any candidate key is known as non-prime attribute.

Example: Suppose a school wants to store the data of teachers and the subjects they teach. They create a table that looks like this: Since a teacher can teach more than one subjects, the table can have multiple rows for a same teacher.

teacher_id	subject	teacher_age
111	Maths	38
111	Physics	38

222	Biology	38
333	Physics	40
333	Chemistry	40

Candidate Keys: {teacher_id, subject}

Non prime attribute: teacher_age

The table is in 1 NF because each attribute has atomic values. However, it is not in 2NF because non prime attribute teacher_age is dependent on teacher_id alone which is a proper subset of candidate key. This violates the rule for 2NF as the rule says “**no** non-prime attribute is dependent on the proper subset of any candidate key of the table”.

To make the table complies with 2NF we can break it in two tables like this:

teacher_details table:

teacher_id	teacher_age
111	38
222	38
333	40

teacher_subject table:

teacher_id	subject
111	Maths
111	Physics
222	Biology
333	Physics
333	Chemistry

Now the tables comply with Second normal form (2NF).

Third Normal form (3NF)

A table design is said to be in 3NF if both the following conditions hold:

- Table must be in 2NF
- [Transitive functional dependency](#) of non-prime attribute on any super key should be removed.

An attribute that is not part of any [candidate key](#) is known as non-prime attribute.

In other words 3NF can be explained like this: A table is in 3NF if it is in 2NF and for each functional dependency $X \rightarrow Y$ at least one of the following conditions hold:

- X is a [super key](#) of table
- Y is a prime attribute of table

An attribute that is a part of one of the candidate keys is known as prime attribute.

Example: Suppose a company wants to store the complete address of each employee, they create a table named employee_details that looks like this:

emp_id	emp_name	emp_zip	emp_state	emp_city	emp_district
1001	John	282005	UP	Agra	Dayal Bagh
1002	Ajeet	222008	TN	Chennai	M-City
1006	Lora	282007	TN	Chennai	Urrapakkam
1101	Lilly	292008	UK	Pauri	Bhagwan
1201	Steve	222999	MP	Gwalior	Ratan

Super keys: {emp_id}, {emp_id, emp_name}, {emp_id, emp_name, emp_zip}...so on

Candidate Keys: {emp_id}

Non-prime attributes: all attributes except emp_id are non-prime as they are not part of any candidate keys.

Here, emp_state, emp_city & emp_district dependent on emp_zip. And, emp_zip is dependent on emp_id that makes non-prime attributes (emp_state, emp_city & emp_district) transitively dependent on super key (emp_id). This violates the rule of 3NF.

To make this table complies with 3NF we have to break the table into two tables to remove the transitive dependency:

employee table:

emp_id	emp_name	emp_zip
1001	John	282005
1002	Ajeet	222008
1006	Lora	282007
1101	Lilly	292008
1201	Steve	222999

employee_zip table:

emp_zip	emp_state	emp_city	emp_district
282005	UP	Agra	Dayal Bagh
222008	TN	Chennai	M-City
282007	TN	Chennai	Urrapakkam
292008	UK	Pauri	Bhagwan
222999	MP	Gwalior	Ratan

These forms have increasingly restrictive requirements: Every relation in BCNF is also in 3NF, every relation in 3NF is also in 2NF, and every relation in 2NF is in 1NF. A relation is in first normal form if every field

contains only atomic values, that is, not lists or sets. This requirement is implicit in our definition of the relational model. Although some of the newer database systems are relaxing this requirement, in this chapter we will assume that it always holds. 2NF is mainly of historical interest. 3NF and BCNF are important from a database design standpoint.

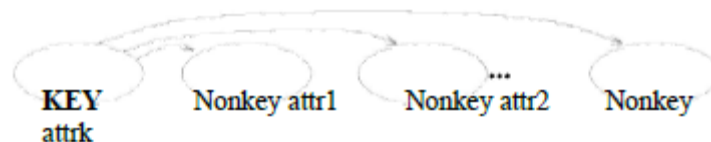
Boyce-Codd Normal Form(BCNF):

Let R be a relation schema, X be a subset of the attributes of R , and let A be an attribute of R .

R is in Boyce-Codd normal form if for every FD $X \rightarrow A$ that holds over R , one of the following statements is true:

$A \rightarrow X$; that is, it is a trivial FD, or X is a super key. Note that if we are given a set F of FDs, according to this definition, we must consider each dependency $X \rightarrow A$ in the closure F^+ to determine whether R is in BCNF.

However, we can prove that it is sufficient to check whether the left side of each dependency in F is a super key (by computing the attribute closure and seeing if it includes all attributes of R).



FDs in a BCNF Relation

BCNF ensures that no redundancy can be detected using FD information alone. It is thus the most desirable normal form (from the point of view of redundancy).

Thus, if a relation is in BCNF, every field of every tuple records a piece of information that cannot be inferred (using only FDs) from the values in all other fields in (all tuples of) the relation instance. Third Normal Form Let R be a relation schema, X be a subset of the attributes of R , and A be an attribute of R . R is in third normal form if for every FD $X \rightarrow A$ that holds over R , one of the following statements is true:

- $A \rightarrow X$; that is, it is a trivial FD, or
- X is a super key, or
- A is part of some key for R .

The definition of 3NF is similar to that of BCNF, with the only difference being the third condition. Every BCNF relation is also in 3NF. Partial dependencies are illustrated in Figure 15.9, and transitive dependencies are illustrated in Figure. Note that in Figure 15.10, the set X of attributes may or may not have some attributes in common with KEY ; the diagram should be interpreted as indicating only that X is not a subset of KEY .



Figure 15.9 Partial Dependencies

**Transitive Dependencies**

The motivation for 3NF is rather technical. By making an exception for certain dependencies involving key attributes, we can ensure that every relation schema can be decomposed into a collection of 3NF relations using only decompositions that have certain desirable properties

DECOMPOSITIONS:

As we have seen, a relation in BCNF is free of redundancy (to be precise, redundancy that can be detected using FD information), and a relation schema in 3NF comes close. If a relation schema is not in one of these normal forms, the FDs that cause a violation can give us insight into the potential problems. A decomposition of a relation schema R consists of replacing the relation schema by two (or more) relation schemas that each contain a subset of the attributes of R and together include all attributes in R . **Lossless-Join Decomposition** Let R be a relation schema and let F be a set of FDs over R .

A decomposition of R into two schemas with attribute sets X and Y is said to be a lossless-join decomposition with respect to F if for every instance r of R that satisfies the dependencies in F , $X(r) \bowtie Y(r) = r$. All decompositions used to eliminate redundancy must be lossless.

The following simple test is very useful:

Let R be a relation and F be a set of FDs that hold over R . The decomposition of R into relations with attribute sets R_1 and R_2 is lossless if and only if F^+ contains either the FD $R_1 \not\rightarrow R_2$ or the FD $R_2 \not\rightarrow R_1$.

Because SD is not a key the dependency $SD \rightarrow P$ causes a violation of BCNF. Let R be a relation schema that is decomposed into two schemas with attribute sets X and Y , and let F be a set of FDs over R .

The projection of F on X is the set of $\rightarrow$ FDs in the closure F^+ (not just F !) that involve only attributes in X . We will denote $\rightarrow$ the projection of F on attributes X as F_X .

Note that a dependency $U \rightarrow V$ in F is in F_X only if all the attributes in U and V are in X . The decomposition of relation schema R with FDs F into schemas with attribute sets X and Y is dependency-preserving if $(F_X \cup F_Y)^+ = F^+$.

That is, if we take the dependencies in F_X and F_Y and compute the closure of their union, we get back all dependencies in the closure of F . Therefore, we need to enforce only the dependencies in F_X and F_Y ; all FDs in F^+ are then sure to be satisfied. To enforce F_X , we need to examine only relation X (on inserts to that relation).

To enforce F_Y , we need to examine only relation Y .

NORMALIZATION

Having covered the concepts needed to understand the role of normal forms and de-compositions in database design, we now consider algorithms for converting relations to BCNF or 3NF. If a relation schema is not in BCNF, it is possible to obtain a lossless-join decomposition into a collection of BCNF relation schemas..

Unfortunately, there may not be any dependency-preserving decomposition into a collection of BCNF relation schemas. Decomposition into BCNF We now present an algorithm for decomposing a relation schema R into a collection of BCNF relation schemas:

1. Suppose that R is not in BCNF. Let $X \rightarrow A$ be a single attribute in R , and $X \rightarrow A$ be an FD that causes a violation of BCNF. Decompose R into $R \rightarrow A$ and $X \rightarrow A$.
2. If either $R \rightarrow A$ or $X \rightarrow A$ is not in BCNF, decompose them further by a recursive application of this algorithm. $R \rightarrow A$ denotes the set of attributes other than A in R , and $X \rightarrow A$ denotes the union of attributes in X and A .

Since $X \rightarrow A$ violates BCNF, it is not a trivial dependency; further, A is a single attribute. Therefore, A is not in X ; that is, $X \cap A$ is empty. Thus, each decomposition carried out in Step 1 is lossless-join.

The set of dependencies associated with $R \rightarrow A$ and $\pi_X A$ is the projection of F onto their attributes. If one of the new relations is not in BCNF, we decompose it further in Step.

Since a decomposition results in relations with strictly fewer attributes, this process will terminate, leaving us with a collection of relation schemas that are all in BCNF. Consider the Contracts relation with attributes CSJDPQV and key C. We are given FDs $JP \twoheadrightarrow C$ and $SD \twoheadrightarrow P$. By using the dependency $SD \twoheadrightarrow P$ to guide the decomposition, we get the two schemas SDP and CSJDQV. SDP is in BCNF.

Suppose that we also have the constraint that each project deals with a single supplier: $J \twoheadrightarrow S$. This means that the schema CSJDQV is not in BCNF. So we decompose it further into JS and CJDQV. $C \twoheadrightarrow JDQV$ holds over CJDQV; the only other FDs that hold are those obtained from this FD by augmentation, and therefore all FDs contain a key in the left side.

Thus, each of the Unfortunately, there may not be any dependency-preserving decomposition into a collection of BCNF relation schemas

Decomposition into BCNF

We now present an algorithm for decomposing a relation schema R into a collection of BCNF relation schemas:

1. Suppose that R is not in BCNF. Let $X \twoheadrightarrow A$ be a single attribute in R , and $X \not\rightarrow A$ be an FD that causes a violation of BCNF. Decompose R into $R \rightarrow A$ and $X \twoheadrightarrow A$.
2. If either $R \rightarrow A$ or $X \twoheadrightarrow A$ is not in BCNF, decompose them further by a recursive application of this algorithm.

$R \rightarrow A$ denotes the set of attributes other than A in R , and $X \twoheadrightarrow A$ denotes the union of attributes in X and A . Since $X \not\rightarrow A$ violates BCNF, it is not a trivial dependency; further, A is a single attribute. Therefore, A is not in X ; that is, $X \cap A$ is empty. Thus, each decomposition carried out in Step 1 is lossless-join.

The set of dependencies associated with $R \rightarrow A$ and $X \twoheadrightarrow A$ is the projection of F onto their attributes. If one of the new relations is not in BCNF, we decompose it further in Step 1. Since a decomposition results in relations with strictly fewer attributes, this process will terminate, leaving us with a collection of relation schemas that are all in BCNF.

Consider the Contracts relation with attributes $CSJDQV$ and key C . We are given FDs $JP \twoheadrightarrow C$ and $SD \twoheadrightarrow P$. By using the dependency $SD \twoheadrightarrow P$ to guide the decomposition, we get the two schemas SDP and $CSJDQV$. SDP is in BCNF. Suppose that we also have the constraint that each project deals with a single supplier: $J \twoheadrightarrow S$. This means that the schema $CSJDQV$ is not in BCNF. So we decompose it further into JS and $CJDQV$. $C \twoheadrightarrow JDQV$ holds over $CJDQV$; the only other FDs that hold are those obtained from this FD by augmentation, and therefore all FDs contain a key in the left side. Thus, each of the

schemas SDP, JS, and CJDQV is in BCNF, and this collection of schemas also represents a lossless-join decomposition of CSJDQV. The steps in this decomposition process can be visualized as a tree, as shown in Figure. The root is the original relation CSJDPQV, and the leaves are the BCNF relations that are the result of the decomposition algorithm, namely, SDP, JS, and CSDQV. Intuitively, each internal node is replaced by its children through a single decomposition step that is guided by the FD shown just below the node.

Redundancy in BCNF Revisited

The decomposition of $CSJDQV$ into SDP , JS , and $CJDQV$ is not dependency-preserving. Intuitively, dependency $JP \twoheadrightarrow C$ cannot be enforced without a join. One way to deal with this situation is to add a relation with attributes CJP .

This is a subtle point: Each of the schemas CJP , SDP , JS , and $CJDQV$ is in BCNF, yet there is some redundancy that can be predicted by FD information. In particular, if we join the relation instances for SDP and $CJDQV$ and project onto the attributes CJP , we must get exactly the instance stored in the relation with schema CJP .

Minimal Cover for a Set of FDs

A minimal cover for a set F of FDs is a set G of FDs such that:

1. Every dependency in G is of the form $X \twoheadrightarrow A$, where A is a single attribute.
2. The closure F^+ is equal to the closure G^+ .

3. If we obtain a set H of dependencies from G by deleting one or more dependencies, or by deleting attributes from a dependency in G , then $F^+ \neq H^+$.

Intuitively, a minimal cover for a set F of FDs is an equivalent set of dependencies that is *minimal* in two respects: (1) Every dependency is as small as possible; that is, each attribute on the left side is necessary and the right side is a single attribute.

(2) Every dependency in it is required in order for the closure to be equal to F^+ . As an example, let F be the set of dependencies:

$$A \twoheadrightarrow B, ABCD \twoheadrightarrow E, EF \twoheadrightarrow G, EF \twoheadrightarrow H, \text{ and } ACDF \twoheadrightarrow EG.$$

First, let us rewrite $ACDF \rightarrow EG$ so that every right side is a single attribute: $ACDF \rightarrow E$ and $ACDF \rightarrow G$.

Next consider $ACDF \rightarrow G$. This dependency is implied by the following FDs:

$A \rightarrow B$, $ABCD \rightarrow E$, and $EF \rightarrow G$.

Therefore, we can delete it. Similarly, we can delete $ACDF \rightarrow E$. Next consider $ABCD$

$\rightarrow E$. Since $A \rightarrow B$ holds, we can replace it with $ACD \rightarrow E$. (At this point, the reader should verify that each remaining FD is minimal and required.) Thus, a minimal cover for F is the set:

$A \rightarrow B$, $ACD \rightarrow E$, $EF \rightarrow G$, and $EF \rightarrow H$.

The preceding example suggests a general algorithm for obtaining a minimal cover of a set F of FDs:

1. Put the FDs in a standard form: Obtain a collection G of equivalent FDs with a single attribute on the right side (using the decomposition axiom).
2. Minimize the left side of each FD: For each FD in G , check each attribute in the left side to see if it can be deleted while preserving equivalence to F^+ .
3. Delete redundant FDs: Check each remaining FD in G to see if it can be deleted while preserving equivalence to F^+ .

Dependency-Preserving Decomposition into 3NF

Returning to the problem of obtaining a lossless-join, dependency-preserving decomposition into 3NF relations, let R be a relation with a set F of FDs that is a

minimal cover, and let $R_1, R_2, \dots, R_n$ be a lossless-join decomposition of R . For $1 \leq i \leq n$, suppose that each R_i is in 3NF and let F_i denote the projection of F onto the attributes of R_i . Do the following:

Identify the set N of dependencies in F that are not preserved, that is, not included in the closure of the union of F_i s.

- For each FD $X \rightarrow A$ in N , create a relation schema XA and add it to the decomposition of R .

Obviously, every dependency in F is preserved if we replace R by the R_i s plus the schemas of the form XA added in this step. The R_i s are given to be in 3NF. We can show that each of the schemas XA is in 3NF as follows: Since $X \rightarrow A$ is in the minimal cover F , $Y \rightarrow A$ does not hold for any Y that is a strict subset of X . Therefore, X is a key for XA .

As an optimization, if the set N contains several FDs with the same left side, say, $X \rightarrow$

$A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$, we can replace them with a single equivalent FD $X \rightarrow A_1 \dots$

A_n . Therefore, we produce one relation schema $XA_1 \dots A_n$, instead of several

schemas $XA_1, \dots, XA_n$, which is generally preferable.

Comparing this decomposition with the one that we obtained earlier in this section, we find that they are quite close, with the only difference being that one of them has $CDJPQV$ instead of CJP and $CJDQV$. In general, however, there could be significant differences.

Database designers typically use a conceptual design methodology (e.g., ER design) to arrive at an initial database design. Given this, the approach of repeated decompositions to rectify instances of redundancy is likely to be the most natural use of FDs and normalization techniques. However, a designer can also consider the alternative designs suggested by the synthesis approach.

Multivalued Dependencies

Suppose that we have a relation with attributes *course*, *teacher*, and *book*, which we denote as *CTB*. The meaning of a tuple is that teacher *T* can teach course *C*, and book

B is a recommended text for the course. There are no FDs; the key is *CTB*.

However, the recommended texts for a course are independent of the instructor.

The instance shown in Figure 15.13 illustrates this situation.

<i>course</i>	<i>teacher</i>	<i>book</i>
Physics101	Green	Mechanics
Physics101	Green	Optics
Physics101	Brown	Mechanics
Physics101	Brown	Optics
Math301	Green	Mechanics
Math301	Green	Vectors
Math301	Green	Geometry

BCNF Relation with Redundancy That Is Revealed by MVDs

There are three points to note here:

The relation schema *CTB* is in BCNF; thus we would not consider decomposing it further if we looked only at the FDs that hold over *CTB*.

There is redundancy. The fact that Green can teach Physics101 is recorded once per recommended text for the course. Similarly, the fact that Optics is a text for Physics101 is recorded once per potential teacher.

- The redundancy can be eliminated by decomposing *CTB* into *CT* and *CB*.

This table suggests another way to think about MVDs: If $X \twoheadrightarrow Y$ holds over R , then γ

$z(\chi=x(R)) = \gamma(\chi=x(R)) z(\chi=x(R))$ in every legal instance of R , for any value x that appears in the X column of R . In other words, consider groups of tuples in R with the same X -value, for each X -value. In each such group consider the projection onto the attributes YZ . This projection must be equal to the cross-product of the projections onto Y and Z . That is, for a given X -value, the Y -values and Z -values are independent. (From this definition it is easy to see that $X \twoheadrightarrow Y$ must hold whenever $X \rightarrow Y$ holds. If the FD $X \rightarrow Y$ holds, there is exactly one Y -value for a given X -value, and the conditions in the MVD definition hold trivially. The converse does not hold, as Figure 15.14 illustrates.)

Returning to our *CTB* example, the constraint that course texts are independent of instructors can be expressed as $C \twoheadrightarrow T$. In terms of the definition of MVDs, this constraint can be read as follows:

If (there is a tuple showing that) C is taught by teacher T , and (there is a tuple showing that) C has book B as text,
then (there is a tuple showing that) C is taught by T and has text B .

Given a set of FDs and MVDs, in general we can infer that several additional FDs and MVDs hold. A sound and complete set of inference rules consists of the three Armstrong Axioms plus five additional rules. Three of the additional rules involve only MVDs:

- **MVD Complementation:** If $X \twoheadrightarrow Y$, then $X \twoheadrightarrow R - XY$.
- **MVD Augmentation:** If $X \twoheadrightarrow Y$ and $W \twoheadrightarrow Z$, then $WX \twoheadrightarrow YZ$.
- **MVD Transitivity:** If $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z$, then $X \twoheadrightarrow (Z - Y)$.

As an example of the use of these rules, since we have $C \twoheadrightarrow T$ over CTB , MVD complementation allows us to infer that $C \twoheadrightarrow CT - B = CT$ as well, that is, $C \twoheadrightarrow B$. The remaining two rules relate FDs and MVDs:

- **Replication:** If $X \twoheadrightarrow Y$, then $X \rightarrow Y$.
- **Coalescence:** If $X \twoheadrightarrow Y$ and there is a W such that $W \cap Y$ is empty, $W \rightarrow Z$, and $Y \twoheadrightarrow Z$, then $X \rightarrow Z$.

Observe that replication states that every FD is also an MVD.
Fourth Normal Form

Fourth normal form is a direct generalization of BCNF. Let R be a relation schema, X and Y be nonempty subsets of the attributes of R , and F be a set of dependencies that includes both FDs and MVDs. R is said to be in fourth normal form (4NF) if for every MVD $X \twoheadrightarrow Y$ that holds over R , one of the following statements is true:

- $Y \subseteq X$ or $XY = R$, or
- X is a Superkey.

In reading this definition, it is important to understand that the definition of a key has not changed the key must uniquely determine all attributes through FDs alone. $X \twoheadrightarrow Y$ is a trivial MVD if $Y \subseteq X$ or $XY = R$; such MVDs always hold.

The relation CTB is not in 4NF because $C \twoheadrightarrow T$ is a nontrivial MVD and C is not a key. We can eliminate the resulting

redundancy by decomposing CTB into CT and CB ; each of these relations is then in 4NF.

To use MVD information fully, we must understand the theory of MVDs. However, the following result due to Date and Fagin identifies conditions detected using only FD information! under which we can safely ignore MVD information. That is, using MVD information in addition to the FD information will not reveal any redundancy. Therefore, if these conditions hold, we do not even need to identify all MVDs.

If a relation schema is in BCNF, and at least one of its keys consists of a single attribute, it is also in 4NF.

An important assumption is implicit in any application of the preceding result: *The set of FDs identified thus far is indeed the set of all FDs that hold over the relation.* This assumption is important because the result relies on the relation being in BCNF, which in turn depends on the set of FDs that hold over the relation.

Figure shows three tuples from an instance of $ABCD$ that satisfies the given MVD B

!! C. From the definition of an MVD, given tuples t_1 and t_2 , it follows

B	C	A	D	
b	c_1	a_1	d_1	tuple t_1
b	c_2	a_2	d_2	tuple t_2
b	c_1	a_2	d_2	tuple t_3

Three Tuples from a Legal Instance of $ABCD$

that tuple t_3 must also be included in the instance.

Consider tuples t_2 and t_3 . From the given FD $A \twoheadrightarrow BCD$ and the fact that these tuples have the same A -value, we can deduce that $c_1 = c_2$. Thus, we see that the FD $B \twoheadrightarrow C$ must hold over $ABCD$ whenever the FD $A \twoheadrightarrow BCD$ and the MVD $B \twoheadrightarrow\!\!\rightarrow C$ hold. If $B \twoheadrightarrow C$ holds, the relation $ABCD$ is not in BCNF (unless additional FDs hold that make B a key)!

Join Dependencies

A join dependency is a further generalization of MVDs. A join dependency (JD)

$\Join fR_1; \dots; R_n g$ is said to hold over a relation R if $R_1; \dots; R_n$ is a lossless-join decomposition of R .

An MVD $X \twoheadrightarrow\!\!\rightarrow Y$ over a relation R can be expressed as the join dependency $\Join fXY, X(R-Y)g$. As an example, in the CTB relation, the MVD $C \twoheadrightarrow\!\!\rightarrow T$ can be expressed as the join dependency $\Join fCT, CBg$.

Unlike FDs and MVDs, there is no set of sound and complete inference rules for JDs.

Fifth Normal Form

A relation schema R is said to be in fifth normal form (5NF) if for every JD $\Join fR_1; \dots; R_n g$ that holds over R , one of the following statements is true:

- $R_i = R$ for some i , or
- The JD is implied by the set of those FDs over R in which the left side is a key for R .

The second condition deserves some explanation, since we have not presented inference rules for FDs and JDs taken together. Intuitively, we must be able to show

that the decomposition of R into $\{R_1; \dots; R_n\}$ is lossless-join whenever the key dependencies (FDs in which the left side is a key for R) hold. $\{R_1; \dots; R_n\}$ is a trivial JD if $R_i = R$ for some i ; such a JD always holds.

The following result, also due to Date and Fagin, identifies conditions again, detected using only FD information under which we can safely ignore JD information.

If a relation schema is in 3NF and each of its keys consists of a single attribute, it is also in 5NF.

The conditions identified in this result are sufficient for a relation to be in 5NF, but not necessary. The result can be very useful in practice because it allows us to conclude that a relation is in 5NF *without ever identifying the MVDs and JDs that may hold over the relation*.

UNIT-IV

TRANSACTION MANAGEMENT**What is a Transaction?**

A transaction is an event which occurs on the database. Generally a transaction reads a value from the database or writes a value to the database. If you have any concept of Operating Systems, then we can say that a transaction is analogous to processes.

Although a transaction can both read and write on the database, there are some fundamental differences between these two classes of operations. A read operation does not change the image of the database in any way. But a write operation, whether performed with the intention of inserting, updating or deleting data from the database, changes the image of the database. That is, we may say that these transactions bring the database from an image which existed before the transaction occurred (called the **Before Image** or **BFIM**) to an image which exists after the transaction occurred (called the **After Image** or **AFIM**).

The Four Properties of Transactions

Every transaction, for whatever purpose it is being used, has the following four properties. Taking the initial letters of these four properties we collectively call them the **ACID Properties**. Here we try to describe them and explain them.

Atomicity: This means that either all of the instructions within the transaction will be reflected in the database, or none of them will be reflected.

Say for example, we have two accounts A and B, each containing Rs 1000/-. We now start a transaction to deposit Rs 100/- from account A to Account B.

Read A;

$A = A - 100;$

Write B;

Read B;

B = B + 100;

Write B;

Fine, is not it? The transaction has 6 instructions to extract the amount from A and submit it to B. The AFIM will show Rs 900/- in A and Rs 1100/- in B. Now, suppose there is a power failure just after instruction 3 (Write A) has been complete. What happens now? After the system recovers the AFIM will show Rs 900/- in A, but the same Rs 1000/- in B. It would be said that Rs 100/- evaporated in thin air for the power failure. Clearly such a situation is not acceptable. The solution is to keep every value calculated by the instruction of the transaction not in any stable storage (hard disc) but in a volatile storage (RAM), until the transaction completes its last instruction. When we see that there has not been any error we do something known as a COMMIT operation. Its job is to write every temporarily calculated value from the volatile storage on to the stable storage. In this way, even if power fails at instruction 3, the post recovery image of the database will show accounts A and B both containing Rs 1000/- , as if the failed transaction had never occurred.

Consistency: If we execute a particular transaction in isolation or together with other transaction, (i.e. presumably in a multi-programming environment), the transaction will yield the same expected result. To give better performance, every database management system supports the execution of multiple transactions at the same time, using CPU Time Sharing. Concurrently executing transactions may have to deal with the problem of sharable resources, i.e. resources that multiple transactions are trying to read/write at the same time. For example, we may have a table or a record on which two transaction are trying to read or write at the same time. Careful mechanisms are created in order to prevent mismanagement of these sharable resources, so that there should not be any change in the way a transaction performs.

A transaction which deposits Rs 100/- to account A must deposit the same amount whether it is acting alone or in conjunction with another transaction that may be trying

to deposit or withdraw some amount at the same time.

Isolation: In case multiple transactions are executing concurrently and trying to access a sharable resource at the same time, the system should create an ordering in their execution so that they should not create any anomaly in the value stored at the sharable resource.

There are several ways to achieve this and the most popular one is using some kind of locking mechanism. Again, if you have the concept of Operating Systems, then you should remember the semaphores, how it is used by a process to make a resource busy before starting to use it, and how it is used to release the resource after the usage is over. Other processes intending to access that same resource must wait during this time. Locking is almost similar. It states that a transaction must first lock the data item that it wishes to access, and release the lock when the accessing is no longer required. Once a transaction locks the data item, other transactions wishing to access the same data item must wait until the lock is released.

Durability: It states that once a transaction has been complete the changes it has made should be permanent.

As we have seen in the explanation of the Atomicity property, the transaction, if completes successfully, is committed. Once the COMMIT is done, the changes which the transaction has made to the database are immediately written into permanent storage. So, after the transaction has been committed successfully, there is no question of any loss of information even if the power fails. Committing a transaction guarantees that the AFIM has been reached. There are several ways Atomicity and Durability can be implemented. One of them is called Shadow Copy. In this scheme a database pointer is used to point to the BFIM of the database. During the transaction, all the temporary changes are recorded into a Shadow Copy, which is an exact copy of the original database plus the changes made by the transaction, which is the AFIM. Now, if the transaction is required to COMMIT, then the database pointer is updated to point to the AFIM copy, and the BFIM copy is discarded. On the other hand, if the transaction is not committed, then the database pointer is not updated. It

keeps pointing to the BFIM, and the AFIM is discarded. This is a simple scheme, but takes a lot of memory space and time to implement.

If you study carefully, you can understand that Atomicity and Durability is essentially the same thing, just as Consistency and Isolation is essentially the same thing.

Transaction States:

There are the following six states in which a transaction may exist:

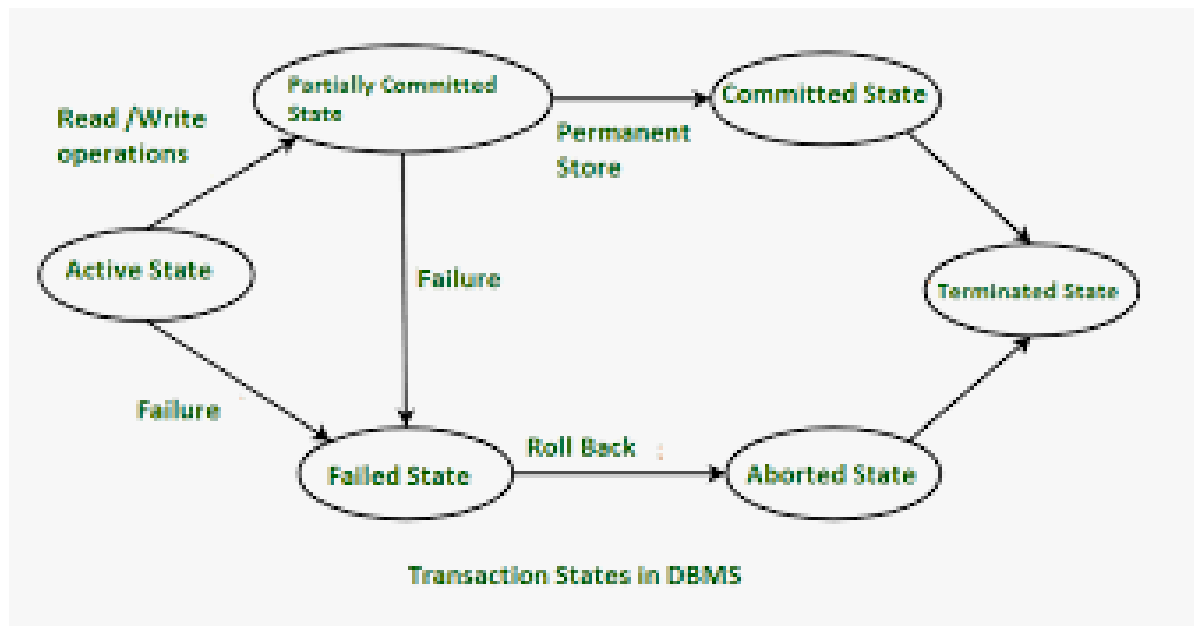
Active: The initial state when the transaction has just started execution.

Partially Committed: At any given point of time if the transaction is executing properly, then it is going towards its COMMIT POINT. The values generated during the execution are all stored in volatile storage. **Failed:** If the transaction fails for some reason. The temporary values are no longer required, and the transaction is set to ROLLBACK. It means that any change made to the database by this transaction up to the point of the failure must be undone. If the failed transaction has withdrawn Rs. 100/- from account A, then the ROLLBACK operation should add Rs 100/- to account A.

Aborted: When the ROLLBACK operation is over, the database reaches the BFIM. The transaction is now said to have been aborted.

Committed: If no failure occurs then the transaction reaches the COMMIT POINT. All the temporary values are written to the stable storage and the transaction is said to have been committed.

Terminated: Either committed or aborted, the transaction finally reaches this state. The whole process can be described using the following diagram:

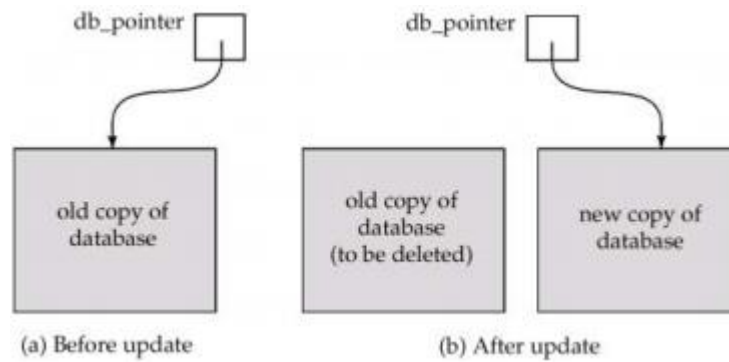


Implementation Of Atomicity And Durability

1. The recovery- management component of a database system implements the support for atomicity and durability.

E.g. the shadow-database scheme:

2. All updates are prefabricated on a shadow copy of the database
 - db_pointer is made to repair to the updated shadow copy after the transaction reaches inclined commit and all updated pages make been flushed to disk.
3. **db_pointer** ever points to the modern conformable repeat of the database.
 - In case transaction unsuccess, old consistent duplicate pointed to by db_pointer can be used, and the shade reduplicate can be deleted.
4. The shadow-database scheme:
 - Assumes that exclusive one transaction is live at a instant.
 - Assumes disks do not break
 - Useful for book editors, but
 1. extremely inefficient for macro databases
 2. Variant called shadow paging reduces copying of information, but is still not pragmatic for large databases
5. Does not interact concurrent transactions



Concurrent Execution:

A schedule is a collection of many transactions which is implemented as a unit. Depending upon how these transactions are arranged in within a schedule, a schedule can be of two types:

- Serial: The transactions are executed one after another, in a non-preemptive manner.

- Concurrent: The transactions are executed in a preemptive, time shared method. In Serial schedule, there is no question of sharing a single data item among many transactions, because not more than a single transaction is executing at any point of time. However, a serial schedule is inefficient in the sense that the transactions suffer for having a longer waiting time and response time, as well as low amount of resource utilization. In concurrent schedule, CPU time is shared among two or more transactions in order to run them concurrently. However, this creates the possibility that more than one transaction may need to access a single data item for read/write purpose and the database could contain inconsistent value if such accesses are not handled properly. Let us explain with the help of an example. Let us consider there are two transactions T1 and T2, whose instruction sets are given as following. T1 is the same as we have seen earlier, while T2 is a new transaction.

T1

Read A;

$A = A - 100;$

Write A;

Read B;

$B = B + 100;$

Write B;

T2

Read A;

$Temp = A * 0.1;$

Read C;

$C = C + Temp;$

Write C;

T2 is a new transaction which deposits to account C 10% of the amount in account A.

If we prepare a serial schedule, then either T1 will completely finish before T2 can begin, or T2 will completely finish before T1 can begin. However, if we want to create a concurrent schedule, then some Context Switching need to be made, so that some portion of T1 will be executed, then some portion of T2 will be executed and so on. For example say we have prepared the following concurrent schedule.

T1T2

Read A;

 $A = A - 100;$

Write A;

Read A;

 $Temp = A * 0.1;$

Read C;

 $C = C + Temp;$

Write C;

Read B;

 $B = B + 100;$

Write B;

No problem here. We have made some Context Switching in this Schedule, the first one after executing the third instruction of T1, and after executing the last statement of T2. T1 first deducts Rs 100/- from A and writes the new value of Rs 900/- into A. T2 reads the value of A, calculates the value of Temp to be Rs 90/- and adds the value to C. The remaining part of T1 is executed and Rs 100/- is added to B.

It is clear that a proper Context Switching is very important in order to maintain the Consistency and Isolation properties of the transactions. But let us take another example where a wrong Context Switching can bring about disaster. Consider the following example involving the same T1 and T2

<u>T1</u>	<u>T2</u>
Read A;	
A = A – 100;	
	Read A;
	Temp = A * 0.1;
	Read C;
	C = C + Temp;
	Write C;
Write A;	
Read B;	
B = B + 100;	
Write B;	

This schedule is wrong, because we have made the switching at the second instruction of T1. The result is very confusing. If we consider accounts A and B both containing Rs 1000/- each, then the result of this schedule should have left Rs 900/- in A, Rs 1100/- in B and add Rs 90 in C (as C should be increased by 10% of the amount in A). But in this wrong schedule, the Context Switching is being performed before the new value of Rs 900/- has been updated in A. T2 reads the old value of A, which is still Rs 1000/-, and deposits Rs 100/- in C. C makes an unjust gain of Rs 10/- out of nowhere.

Serializability:

When several concurrent transactions are trying to access the same data item, the instructions within these concurrent transactions must be ordered in some way so as there are no problem in accessing and releasing the shared data item. There are two aspects of serializability which are described here:

Conflict Serializability

Two instructions of two different transactions may want to access the same data item in order to perform a read/write operation. Conflict Serializability deals with detecting whether the instructions are conflicting in any way, and specifying the order in which these two instructions

will be executed in case there is any conflict. A **conflict** arises if at least one (or both) of the instructions is a write operation. The following rules are important in Conflict Serializability:

1. If two instructions of the two concurrent transactions are both for read operation, then they are not in conflict, and can be allowed to take place in any order.
2. If one of the instructions wants to perform a read operation and the other instruction wants to perform a write operation, then they are in conflict, hence their ordering is important. If the read instruction is performed first, then it reads the old value of the data item and after the reading is over, the new value of the data item is written. If the write instruction is performed first, then updates the data item with the new value and the read instruction reads the newly updated value.
3. If both the transactions are for write operation, then they are in conflict but can be allowed to take place in any order, because the transaction do not read the value updated by each other. However, the value that persists in the data item after the schedule is over is the one written by the instruction that performed the last write.

View Serializability:

This is another type of serializability that can be derived by creating another schedule out of an existing schedule, involving the same set of transactions. These two schedules would be called View Serializable if the following rules are followed while creating the second schedule out of the first. Let us consider that the transactions T1 and T2 are being serialized to create two different schedules

S1 and S2 which we want to be **View Equivalent** and both T1 and T2 wants to access the same data item.

1. If in S1, T1 reads the initial value of the data item, then in S2 also, T1 should read the initial value of that same data item.
2. If in S1, T1 writes a value in the data item which is read by T2, then in S2 also, T1 should

write the value in the data item before T2 reads it.

3. If in S_1 , T_1 performs the final write operation on that data item, then in S_2 also, T_1 should perform the final write operation on that data item.

Let us consider a schedule S in which there are two consecutive instructions, I and J , of transactions T_i and T_j , respectively ($i \neq j$). If I and J refer to different data items, then we can swap I and J without affecting the results of any instruction in the schedule. However, if I and J refer to the same data item Q , then the order of the two steps may matter. Since we are dealing with only read and write instructions, there are four cases that we need to consider:

- $I = \text{read}(Q), J = \text{read}(Q)$. The order of I and J does not matter, since the same value of Q is read by T_i and T_j , regardless of the order.
 - $I = \text{read}(Q), J = \text{write}(Q)$. If I comes before J , then T_i does not read the value of Q that is written by T_j in instruction J . If J comes before I , then T_i reads the value of Q that is written by T_j . Thus, the order of I and J matters.
 - $I = \text{write}(Q), J = \text{read}(Q)$. The order of I and J matters for reasons similar to those of the previous case.
4. $I = \text{write}(Q), J = \text{write}(Q)$. Since both instructions are write operations, the order of these instructions does not affect either T_i or T_j . However, the value obtained by the next $\text{read}(Q)$ instruction of S is affected, since the result of only the latter of the two write instructions is preserved in the database. If there is no other $\text{write}(Q)$ instruction after I and J in S , then the order of I and J directly affects the final value of Q in the database state that results from schedule S .

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

Fig: Schedule 3—showing only the read and write instructions.

item, and at least one of these instructions is a write operation. To illustrate the concept of conflicting instructions, we consider schedule 3 in Figure above. The $\text{write}(A)$ instruction of T_1 conflicts with the $\text{read}(A)$ instruction of T_2 . However, the $\text{write}(A)$ instruction of T_2 does not conflict with the $\text{read}(B)$ instruction of T_1 , because the two instructions access different data items.

Transaction Characteristics

Every transaction has three characteristics: *access mode*, *diagnostics size*, and *isolation level*.

The **diagnostics size** determines the number of error conditions that can be recorded.

If the **access mode** is READ ONLY, the transaction is not allowed to modify the database.

Thus, INSERT, DELETE, UPDATE, and CREATE commands cannot be executed. If we have to execute one of these commands, the access mode should be set to READ WRITE. For transactions with READ ONLY access mode, only shared locks need to be obtained, thereby increasing concurrency.

The **isolation level** controls the extent to which a given transaction is exposed to the actions of other transactions executing concurrently. By choosing one of four possible isolation level settings, a user can obtain greater concurrency at the cost of increasing the transaction's exposure to other transactions' uncommitted changes.

Isolation level choices are READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE. The effect of these levels is summarized in Figure given below.

In this context, *dirty read* and *unrepeatable read* are defined as usual. **Phantom** is defined to be the possibility that a transaction retrieves a collection of objects (in SQL terms, a collection of tuples) twice and sees different results, even though it does not modify any of these tuples itself.

In terms of a lock-based implementation, a SERIALIZABLE transaction obtains locks before reading or writing objects, including locks on sets of objects that it requires to be unchanged (see Section 19.3.1), and holds them until the end, according to Strict 2PL.

REPEATABLE READ ensures that T reads only the changes made by committed transactions, and that no value read or written by T is changed by any other transaction until T is complete.

However, T could experience the phantom phenomenon; for example, while T examines all Sailors records with $rating=1$, another transaction might add a new such Sailors record, which is missed by T .

A REPEATABLE READ transaction uses the same locking protocol as a SERIALIZABLE transaction, except that it does not do index locking, that is, it locks only individual objects, not sets of objects.

READ COMMITTED ensures that T reads only the changes made by committed transactions,

and that no value written by T is changed by any other transaction until T is complete. However, a value read by T may well be modified by another transaction while T is still in progress, and T is, of course, exposed to the phantom problem.

A READ COMMITTED transaction obtains exclusive locks before writing objects and holds these locks until the end. It also obtains shared locks before reading objects, but these locks are released immediately; their only effect is to guarantee that the transaction that last modified the object is complete. (This guarantee relies on the fact that *every* SQL transaction obtains exclusive locks before writing objects and holds exclusive locks until the end.)

A READ UNCOMMITTED transaction does not obtain shared locks before reading objects. This mode represents the greatest exposure to uncommitted changes of other transactions; so much so that SQL prohibits such a transaction from making any changes itself - a READ UNCOMMITTED transaction is required to have an access mode of READ ONLY. Since such a transaction obtains no locks for reading objects, and it is not allowed to write objects (and therefore never requests exclusive locks), it never makes any lock requests.

The SERIALIZABLE isolation level is generally the safest and is recommended for most transactions. Some transactions, however, can run with a lower isolation level, and the smaller number of locks requested can contribute to improved system performance.

For example, a statistical query that finds the average sailor age can be run at the READ COMMITTED level, or even the READ UNCOMMITTED level, because a few incorrect or missing values will not significantly affect the result if the number of sailors is large. The isolation level and access mode can be set using the SET TRANSACTION command. For example, the following command declares the current transaction to be SERIALIZABLE and READ ONLY:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE READ ONLY

When a transaction is started, the default is SERIALIZABLE and READ WRITE.

PRECEDENCE GRAPH:

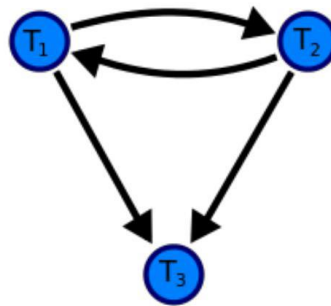
A precedence graph, also named conflict graph and serializability graph, is used in the context of concurrency control in databases.

The precedence graph for a schedule S contains:

A node for each committed transaction in S

An arc from T_i to T_j if an action of T_i precedes and conflicts with one of T_j 's actions.

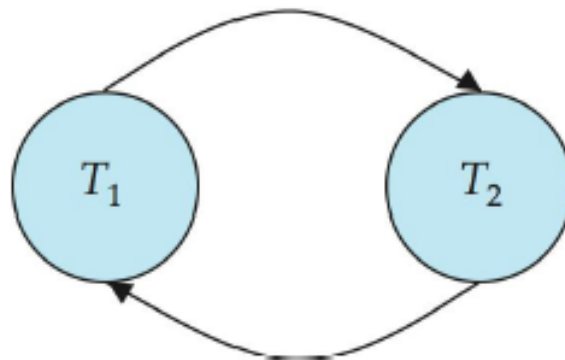
$$D = \begin{bmatrix} T1 & T2 & T3 \\ & R(B) & \\ R(C) & W(A) & \\ W(C) & & \\ R(D) & & W(B) \\ W(D) & & W(A) \end{bmatrix}$$



A precedence graph of the schedule D, with 3 transactions. As there is a cycle (of length 2; with two edges) through the committed transactions T1 and T2, this schedule (history) is not Conflict serializable.

The drawing sequence for the precedence graph:-

- For each transaction T_i participating in schedule S , create a node labelled T_i in the precedence graph. So the precedence graph contains T_1, T_2, T_3
- For each case in S where T_i executes a `write_item(X)` then T_j executes a `read_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph. This occurs nowhere in the above example, as there is no read after write.
- 3. For each case in S where T_i executes a `read_item(X)` then T_j executes a `write_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph. This results in directed edge from T_1 to T_2 .
- 4. For each case in S where T_i executes a `write_item(X)` then T_j executes a `write_item(X)`, create an edge ($T_i \rightarrow T_j$) in the precedence graph. This results in directed edges from T_2 to T_1 , T_1 to T_3 , and T_2 to T_3 .
- 5. The schedule S is conflict serializable if the precedence graph has no cycles. As T_1 and T_2 constitute a cycle, then we cannot declare S as serializable or not and serializability has to be checked using other methods.



TESTING FOR CONFLICT SERIALIZABILITY

- 1 A schedule is conflict serializable if and only if its precedence graph is acyclic.
- 2 To test for conflict serializability, we need to construct the precedence graph and to invoke a cycle-detection algorithm. Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph.

(Better algorithms take order $n + e$ where e is the number of edges.)
- 3 If precedence graph is acyclic, the serializability order can be obtained by a topological sorting of the graph. That is, a linear order consistent with the partial order of the graph.

For example, a serializability order for the schedule (a) would be one of either (b) or (c)

- 4 A serializability order of the transactions can be obtained by finding a linear order consistent with the partial order of the precedence graph.

RECOVERABLE SCHEDULES

- Recoverable schedule — if a transaction T_j reads a data item previously written by a transaction T_i , then the commit operation of T_i must appear before the commit operation of T_j .
- The following schedule is not recoverable if T_9 commits immediately after the read(A) operation.

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

CASCADING ROLLBACKS

- Cascading rollback – a single transaction failure leads to a series of transaction rollbacks. Consider the following schedule where none of the transactions has yet committed (so the schedule is recoverable)

T_{10}	T_{11}	T_{12}
read (A)		
read (B)		
write (A)		
	read (A)	
	write (A)	
		read (A)
abort		

If T_{10} fails, T_{11} and T_{12} must also be rolled back.

- Can lead to the undoing of a significant amount of work

CASCADELESS SCHEDULES

- Cascadeless schedules — for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j .
- Every cascadeless schedule is also recoverable
- It is desirable to restrict the schedules to those that are cascadeless
- Example of a schedule that is NOT cascadeless

T_{10}	T_{11}	T_{12}
read (A) read (B) write (A) abort	 read (A) write (A)	 read (A)

CONCURRENCY SCHEDULE

- A database must provide a mechanism that will ensure that all possible schedules are both:
- Conflict serializable.
- Recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency

- ☐ Concurrency-control schemes tradeoff between the amount of concurrency they allow and the amount of overhead that they incur
- ☐ Testing a schedule for serializability after it has executed is a little too late!
- ☐ Tests for serializability help us understand why a concurrency control protocol is correct
- ☐ Goal – to develop concurrency control protocols that will assure serializability.

WEEK LEVELS OF CONSISTENCY

- ☐ Some applications are willing to live with weak levels of consistency, allowing schedules that are not serializable
- ☐ E.g., a read-only transaction that wants to get an approximate total balance of all accounts
- ☐ E.g., database statistics computed for query optimization can be approximate (why?)
- ☐ Such transactions need not be serializable with respect to other transactions
- ☐ Tradeoff accuracy for performance

LEVELS OF CONSISTENCY IN SQL

- ☐ Serializable — default
- ☐ Repeatable read — only committed records to be read, repeated reads of same record must return same value. However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.
- ☐ Read committed — only committed records can be read, but successive reads of record may return different (but committed) values.
- ☐ Read uncommitted — even uncommitted records may be read.
- ☐ Lower degrees of consistency useful for gathering approximate information about the database
- ☐ Warning: some database systems do not ensure serializable schedules by default
- ☐ E.g., Oracle and PostgreSQL by default support a level of consistency called snapshot isolation (not part of the SQL standard)

TRANSACTION DEFINITION IN SQL

- ☐ Data manipulation language must include a construct for specifying the set of actions that comprise a transaction.

- ☐ In SQL, a transaction begins implicitly.
- ☐ A transaction in SQL ends by:
- ☐ Commit work commits current transaction and begins a new one.
- ☐ Rollback work causes current transaction to abort.
- ☐ In almost all database systems, by default, every SQL statement also commits implicitly if it executes successfully
- ☐ Implicit commit can be turned off by a database directive
- ☐ E.g. in JDBC, `connection.setAutoCommit(false);`

RECOVERY SYSTEM

Failure Classification:

- ☐ Transaction failure :
- ☐ Logical errors: transaction cannot complete due to some internal error condition
- ☐ System errors: the database system must terminate an active transaction due to an error condition (e.g., deadlock)
- ☐ System crash: a power failure or other hardware or software failure causes the system to crash.
- ☐ Fail-stop assumption: non-volatile storage contents are assumed to not be corrupted as result of a system crash
- ☐ Database systems have numerous integrity checks to prevent corruption of disk data
- ☐ Disk failure: a head crash or similar disk failure destroys all or part of disk storage
- ☐ Destruction is assumed to be detectable: disk drives use checksums to detect failures

RECOVERY ALGORITHMS

- ☐ Consider transaction T_i that transfers \$50 from account A to account B
- ☐ Two updates: subtract 50 from A and add 50 to B

- ☐ Transaction T_i requires updates to A and B to be output to the database.
- ☐ A failure may occur after one of these modifications have been made but before both of them are made.
- ☐ Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state
- ☐ Not modifying the database may result in lost updates if failure occurs just after transaction commits
- ☐ Recovery algorithms have two parts
 1. Actions taken during normal transaction processing to ensure enough information exists to recover from failures
 2. Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

STORAGE STRUCTURE

- ☐ Volatile storage:
 - ☐ does not survive system crashes
 - ☐ examples: main memory, cache memory
 - ☐ Nonvolatile storage:
 - ☐ survives system crashes
 - ☐ examples: disk, tape, flash memory,
- non-volatile (battery backed up) RAM
- ☐ but may still fail, losing data
 - ☐ Stable storage:
 - ☐ a mythical form of storage that survives all failures
 - ☐ approximated by maintaining multiple copies on distinct nonvolatile media

Stable-Storage Implementation

- ☐ Maintain multiple copies of each block on separate disks
- ☐ copies can be at remote sites to protect against disasters such as fire or flooding.
- ☐ Failure during data transfer can still result in inconsistent copies.

Block transfer can result in

- ☐ Successful completion
- ☐ Partial failure: destination block has incorrect information
- ☐ Total failure: destination block was never updated
- ☐ Protecting storage media from failure during data transfer (one solution):
- ☐ Execute output operation as follows (assuming two copies of each block):
 1. Write the information onto the first physical block.
 2. When the first write successfully completes, write the same information onto the second physical block.
 3. The output is completed only after the second write successfully completes.
- ☐ Copies of a block may differ due to failure during output operation. To recover from failure:
 1. First find inconsistent blocks:
 1. Expensive solution: Compare the two copies of every disk block.
 2. Better solution:
 - ☐ Record in-progress disk writes on non-volatile storage (Non-volatile RAM or special area of disk).
 - ☐ Use this information during recovery to find blocks that may be inconsistent, and only compare copies of these.
- ☐ Used in hardware RAID systems

2. If either copy of an inconsistent block is detected to have an error (bad checksum), overwrite it by the other copy. If both have no error, but are different, overwrite the second block by the first block.

DATA ACCESS

- ☐ Physical blocks are those blocks residing on the disk.
- ☐ System buffer blocks are the blocks residing temporarily in main memory.
- ☐ Block movements between disk and main memory are initiated through the following two operations:
 - ☐ $\text{input}(B)$ transfers the physical block B to main memory.
 - ☐ $\text{output}(B)$ transfers the buffer block B to the disk, and replaces the appropriate physical block there.
- ☐ We assume, for simplicity, that each data item fits in, and is stored inside, a single block.
- ☐ Each transaction T_i has its private work-area in which local copies of all data items accessed and updated by it are kept.
- ☐ T_i 's local copy of a data item X is denoted by x_i .
- ☐ BX denotes block containing X
- ☐ Transferring data items between system buffer blocks and its private work-area done by:
 - ☐ $\text{read}(X)$ assigns the value of data item X to the local variable x_i .
 - ☐ $\text{write}(X)$ assigns the value of local variable x_i to data item $\{X\}$ in the buffer block.
- ☐ Transactions
 - ☐ Must perform $\text{read}(X)$ before accessing X for the first time (subsequent reads can be from local copy)
 - ☐ The $\text{write}(X)$ can be executed at any time before the transaction commits
 - ☐ Note that $\text{output}(BX)$ need not immediately follow $\text{write}(X)$. System can perform the output operation when it seems fit.

Lock-Based Protocols

A lock is a mechanism to control concurrent access to a data item

Data items can be locked in two modes :

1. exclusive (X) mode. Data item can be both read as well as written. X-lock is requested using lock-X instruction.
2. shared (S) mode. Data item can only be read. S-lock is requested using lock-S instruction.

Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.

Lock-compatibility matrix

	S	X
S	true	false
X	false	false

- 1) A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- 2) Any number of transactions can hold shared locks on an item,
but if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.
- 3) If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Example of a transaction performing locking:

```
T1: lock-S(A);  
    read (A);  
    unlock(A);  
    lock-S(B);  
    read (B);  
    unlock(B);  
    display(A+B)
```

Locking as above is not sufficient to guarantee serializability — if *A* and *B* get updated in-between the read of *A* and *B*, the displayed sum would be wrong.

A locking protocol is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.

Consider the partial schedule

T_3	T_4
lock-x (B) read (B) $B := B - 50$ write (B) lock-x (A)	 lock-s (A) read (A) lock-s (B)

Neither T_3 nor T_4 can make progress — executing **lock-S**(B) causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X**(A) causes T_3 to wait for T_4 to release its lock on A .

Such a situation is called a **deadlock**.

1 To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

2. The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.

3.Starvation is also possible if concurrency control manager is badly designed. For example:

- A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
- The same transaction is repeatedly rolled back due to deadlocks.

4. Concurrency control manager can be designed to prevent starvation.

THE TWO-PHASE LOCKING PROTOCOL

1. This is a protocol which ensures conflict-serializable schedules.

2. Phase 1: Growing Phase

- transaction may obtain locks
- transaction may not release locks

3. Phase 2: Shrinking Phase

- transaction may release locks
- transaction may not obtain locks

4. The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e. the point where a transaction acquired its final lock).

5. Two-phase locking *does not* ensure freedom from deadlocks

6. Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called **strict two-phase locking**. Here a transaction must hold all its exclusive locks till it commits/aborts.

7. Rigorous two-phase locking is even stricter: here *all* locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit.

8. There can be conflict serializable schedules that cannot be obtained if two-phase locking is used.

9. However, in the absence of extra information (e.g., ordering of access to data), two-phase locking is needed for conflict serializability in the following sense:

Given a transaction T_i that does not follow two-phase locking, we can find a transaction T_j that uses two-phase locking, and a schedule for T_i and T_j that is not conflict serializable.

TIMESTAMP-BASED PROTOCOLS

1. Each transaction is issued a timestamp when it enters the system. If an old transaction T_i has time-stamp $TS(T_i)$, a new transaction T_j is assigned time-stamp $TS(T_j)$ such that $TS(T_j) < TS(T_i)$.

2. The protocol manages concurrent execution such that the time-stamps determine the serializability order.

3. In order to assure such behavior, the protocol maintains for each data Q two timestamp values:

a. $W\text{-timestamp}(Q)$ is the largest time-stamp of any transaction that executed $\text{write}(Q)$ successfully.

b. $R\text{-timestamp}(Q)$ is the largest time-stamp of any transaction that executed $\text{read}(Q)$ successfully.

4. The timestamp ordering protocol ensures that any conflicting **read** and **write** operations are executed in timestamp order.

5. Suppose a transaction T_i issues a **read**(Q)

1. If $TS(T_i) \geq W\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.

n Hence, the **read** operation is rejected, and T_i is rolled back.

2. If $TS(T_i) < W\text{-timestamp}(Q)$, then the **read** operation is executed, and $R\text{-timestamp}(Q)$ is set to $\max(R\text{-timestamp}(Q), TS(T_i))$.

6. Suppose that transaction T_i issues **write**(Q).

1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed

previously, and the system assumed that that value would never be produced.

n Hence, the **write** operation is rejected, and T_i is rolled back.

2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q .

n Hence, this **write** operation is rejected, and T_i is rolled back.

3. Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$ is set to $TS(T_i)$.

Thomas' Write Rule

potential concurrency than does the protocol i.e., Timestamp ordering Protocol . Let us consider schedule 4 of Figure below, and apply the timestamp-ordering protocol. Since T_{27} starts before T_{28} , we shall assume that $TS(T_{27}) < TS(T_{28})$. The $read(Q)$ operation of T_{27} succeeds, as does the $write(Q)$ operation of T_{28} . When T_{27} attempts its $write(Q)$ operation, we find that $TS(T_{27}) < W\text{-timestamp}(Q)$, since $W\text{-timestamp}(Q) = TS(T_{28})$. Thus, the $write(Q)$ by T_{27} is rejected and transaction T_{27} must be rolled back.

2. Although the rollback of T_{27} is required by the timestamp-ordering protocol, it is unnecessary. Since T_{28} has already written Q , the value that T_{27} is attempting to write is one that will never need to be read. Any transaction T_i with $TS(T_i) < TS(T_{28})$ that attempts a $read(Q)$ will be rolled back, since $TS(T_i) < W\text{-timestamp}(Q)$.

3. Any transaction T_j with $TS(T_j) > TS(T_{28})$ must read the value of Q written by T_{28} , rather than the value that T_{27} is attempting to write. This observation leads to a modified version of the timestamp-ordering protocol in which obsolete write operations can be ignored under certain circumstances. The protocol rules for read operations remain unchanged.

The protocol rules for write operations, however, are slightly different from the timestamp ordering protocol.

T_{27}	T_{28}
$read(Q)$	$write(Q)$
$write(Q)$	

The modification to the timestamp-ordering protocol, called **Thomas' write rule**, is this:

Suppose that transaction T_i issues $write(Q)$.

1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was previously needed, and it had been assumed that the value would never be produced. Hence, the system rejects the write operation and rolls T_i back.

2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, this write operation can be ignored.

3. Otherwise, the system executes the write operation and sets $W\text{-timestamp}(Q)$ to $TS(T_i)$.

VALIDATION-BASED PROTOCOLS

Phases in Validation-Based Protocols

1) Read phase. During this phase, the system executes transaction T_i . It reads the

values of the various data items and stores them in variables local to T_i . It performs all write operations on temporary local variables, without updates of the actual database.

2) Validation phase. The validation test is applied to transaction T_i . This determines whether T_i is allowed to proceed to the write phase without causing a violation of serializability. If a transaction fails the validation test, the system aborts the transaction.

3) Write phase. If the validation test succeeds for transaction T_i , the temporary local variables that hold the results of any write operations performed by T_i are copied to the database. Read-only transactions omit this phase.

MODES IN VALIDATION-BASED PROTOCOLS

1. Start(T_i)
2. Validation(T_i)
3. Finish

MULTIPLE GRANULARITY.

multiple granularity locking (MGL) is a locking method used in database management systems (DBMS) and relational databases.

In MGL, locks are set on objects that contain other objects. MGL exploits the hierarchical nature of the contains relationship. For example, a database may have files, which contain pages, which further contain records. This can be thought of as a tree of objects, where each node contains its children. A lock on such as a shared or exclusive lock locks the targeted node as well as all of its descendants.

Multiple granularity locking is usually used with non-strict two-phase locking to guarantee serializability. The **multiple-granularity locking protocol** uses these lock modes to ensure serializability. It requires that a transaction T_i that attempts to lock a node Q must follow these rules:

- Transaction T_i must observe the lock-compatibility function of Figure above.
- Transaction T_i must lock the root of the tree first, and can lock it in anymode.
- Transaction T_i can lock a node Q in S or IS mode only if T_i currently has the parent of Q locked in either IX or IS mode.
- Transaction T_i can lock a node Q in X, SIX, or IX mode only if T_i currently has the parent of Q locked in either IX or SIX mode.
- Transaction T_i can lock a node only if T_i has not previously unlocked any node (that is, T_i is two phase).
- Transaction T_i can unlock a node Q only if T_i currently has none of the children of Q locked.

RECOVERY AND ATOMICITY**FAILURE WITH LOSS OF NON-VOLATILE STORAGE**

What would happen if the non-volatile storage like RAM abruptly crashes? All transaction, which are being executed are kept in main memory. All active logs, disk buffers and related data is stored in non-volatile storage.

When storage like RAM fails, it takes away all the logs and active copy of database. It makes recovery almost impossible as everything to help recover is also lost. Following techniques may be adopted in case of loss of non-volatile storage.

- A mechanism like checkpoint can be adopted which makes the entire content of database be saved periodically.
- State of active database in non-volatile memory can be dumped onto stable storage periodically, which may also contain logs and active transactions and buffer blocks.
- <dump> can be marked on log file whenever the database contents are dumped from nonvolatile memory to a stable one.

RECOVERY:

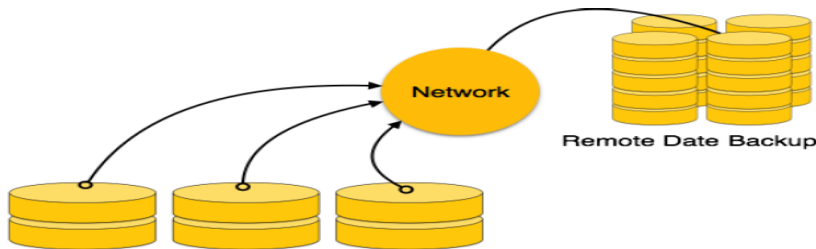
- When the system recovers from failure, it can restore the latest dump.
- It can maintain redo-list and undo-list as in checkpoints.
- It can recover the system by consulting undo-redo lists to restore the state of all transaction up to last checkpoint.

DATABASE BACKUP & RECOVERY FROM CATASTROPHIC FAILURE

Remote backup, described next, is one of the solutions to save life. Alternatively, whole database backups can be taken on magnetic tapes and stored at a safer place. This backup can later be restored on a freshly installed database and bring it to the state at least at the point of backup.

REMOTE BACKUP

Remote backup provides a sense of security and safety in case the primary location where the database is located gets destroyed. Remote backup can be offline or real-time and online. In case it is offline it is maintained manually.



DBMS DATA RECOVERY

CRASH RECOVERY

Though we are living in highly technologically advanced era where hundreds of satellite monitor the earth and at every second billions of people are connected through information technology, failure is expected but not every time acceptable.

FAILURE CLASSIFICATION

To see where the problem has occurred we generalize the failure into various categories, as follows:

Transaction failure When a transaction is failed to execute or it reaches a point after which it cannot be completed successfully it has to abort. This is called transaction failure. Where only few transaction or process are hurt. Reason for transaction failure could be:

- **Logical errors:** where a transaction cannot complete because of it has some code error or any internal error condition
- **System errors:** where the database system itself terminates an active transaction because DBMS is not able to execute it or it has to stop because of some system condition. For example, in case of deadlock or resource unavailability systems aborts an active transaction.

SYSTEM CRASH

There are problems, which are external to the system, which may cause the system to stop abruptly and cause the system to crash. For example interruption in power supply, failure of underlying hardware or software failure. Examples may include operating system errors.

DISK FAILURE: In early days of technology evolution, it was a common problem where hard disk drives or storage drives used to fail frequently. Disk failures include formation of bad sectors, unreachability to the disk, disk head crash or any other failure, which destroys all or part of disk storage

STORAGE STRUCTURE

We have already described storage system here. In brief, the storage structure can be divided in various categories:

- **Volatile storage:** As name suggests, this storage does not survive system crashes and mostly placed very closed to CPU by embedding them onto the chipset itself for examples: main memory, cache memory. They are fast but can store a small amount of information.
- **Nonvolatile storage:** These memories are made to survive system crashes. They are huge in data storage capacity but slower in accessibility. Examples may include, hard disks, magnetic tapes, flash memory, non-volatile (battery backed up) RAM.

RECOVERY AND ATOMICITY

When a system crashes, it may have several transactions being executed and various files opened for them to modifying data items. As we know that transactions are made of various operations, which are atomic in nature.

- It should check the states of all transactions, which were being executed.
- A transaction may be in the middle of some operation; DBMS must ensure the atomicity of transaction in this case.
- It should check whether the transaction can be completed now or needs to be rolled back.
- No transactions would be allowed to left DBMS in inconsistent state. There are two types of techniques, which can help DBMS in recovering as well as maintaining the atomicity of transaction:

actually modifying the database.

- Maintaining shadow paging, where the changes are done on a volatile memory and later the actual database is updated.

LOG-BASED RECOVERY

Log is a sequence of records, which maintains the records of actions performed by a transaction. It is important that the logs are written prior to actual modification and stored on a stable storage media, which is failsafe.

Log based recovery works as follows:

- The log file is kept on stable storage media
- When a transaction enters the system and starts execution, it writes a log about it $\langle T_n, \text{Start} \rangle$
- When the transaction modifies an item X, it writes logs as follows: $\langle T_n, X, V_1, V_2 \rangle$

It reads T_n has changed the value of X, from V_1 to V_2 .

When transaction finishes, it logs: $\langle T_n, \text{commit} \rangle$

Database can be modified using two approaches:

1. **Deferred database modification:** All logs are written on to the stable storage and database is

updated when transaction commits.

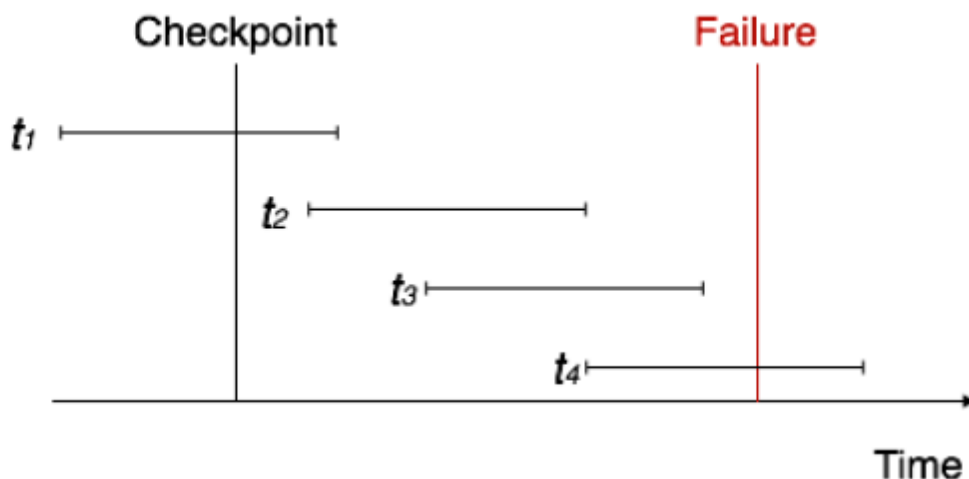
2. **Immediate database modification:** Each log follows an actual database modification. That is, database is modified immediately after every operation.

RECOVERY WITH CONCURRENT TRANSACTIONS

When more than one transactions are being executed in parallel, the logs are interleaved. At the time of recovery it would become hard for recovery system to backtrack all logs, and then start recovering. To ease this situation most modern DBMS use the concept of 'checkpoints'.

Checkpoint Keeping and maintaining logs in real time and in real environment may fill out all the memory space available in the system. At time passes log file may be too big to be handled at all. Checkpoint is a mechanism where all the previous logs are removed from the system and stored permanently in storage disk. Checkpoint declares a point before which the DBMS was in consistent state and all the transactions were committed.

Recovery When system with concurrent transaction crashes and recovers, it does behave in the following manner:



BUFFER MANAGEMENT

1. Database buffers are generally implemented in virtual memory in spite of some drawbacks:

a. When operating system needs to evict a page that has been modified, the page is written to swap space on disk.

b. When database decides to write buffer page to disk, buffer page may be in swap space, and may have to be read from swap space on disk and output to the database on disk, resulting in extra I/O!

► Known as **dual paging** problem.

c. Ideally when OS needs to evict a page from the buffer, it should pass control to

database, which in turn should

- ▶ Output the page to database instead of to swap space (making sure to output log records first), if it is modified
- ▶ Release the page from the buffer, for the OS to use

Dual paging can thus be avoided, but common operating systems do not support such functionality.

FUZZY CHECKPOINTING

a. To avoid long interruption of normal processing during checkpointing, allow updates to happen during checkpointing

b. Fuzzy checkpointing is done as follows:

1. Temporarily stop all updates by transactions
2. Write a **<checkpoint L>** log record and force log to stable storage
3. Note list *M* of modified buffer blocks
4. Now permit transactions to proceed with their actions
5. Output to disk all modified buffer blocks in list *M*

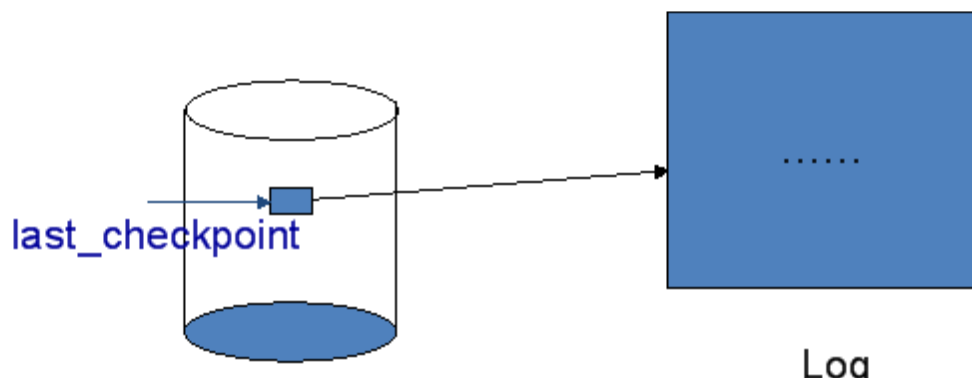
H blocks should not be updated while being output

H Follow WAL: all log records pertaining to a block must be output before the block is output

6. Store a pointer to the **checkpoint** record in a fixed position **last_checkpoint** on disk

7. When recovering using a fuzzy checkpoint, start scan from the **checkpoint** record pointed to by **last_checkpoint**

- a. Log records before **last_checkpoint** have their updates reflected in database on disk, and need not be redone.



FAILURE WITH LOSS OF NONVOLATILE STORAGE

a. So far we assumed no loss of non-volatile storage

b. Technique similar to checkpointing used to deal with loss of non-volatile storage

1. Periodically **dump** the entire content of the database to stable storage
2. No transaction may be active during the dump procedure; a procedure similar to

checkpointing must take place

- ▶ Output all log records currently residing in main memory onto stable storage.
- ▶ Output all buffer blocks onto the disk.
- ▶ Copy the contents of the database to stable storage.
- ▶ Output a record **<dump>** to log on stable storage.

RECOVERING FROM FAILURE OF NON-VOLATILE STORAGE

a.To recover from disk failure

- 1.restore database from most recent dump.
 - 2.Consult the log and redo all transactions that committed after the dump
- b.Can be extended to allow transactions to be active during dump;

known as **fuzzy dump** or **online dump**

- 1.Similar to fuzzy checkpointing

ARIES RECOVERY ALGORITHM

a.ARIES is a state of the art recovery method

- 1.Incorporates numerous optimizations to reduce overheads during normal processing and to speed up recovery
- 2.The recovery algorithm we studied earlier is modeled after ARIES, but greatly simplified by removing optimizations

b.Unlike the recovery algorithm described earlier, ARIES

- 1.Uses **log sequence number (LSN)** to identify log records

- ▶ Stores LSNs in pages to identify what updates have already been applied to a database page

- 2.Physiological redo
- 3.Dirty page table to avoid unnecessary redos during recovery
- 4.Fuzzy checkpointing that only records information about dirty pages, and does not require dirty pages to be written out at checkpoint time

ARIES Recovery Algorithm

ARIES recovery involves three passes

a.Analysis pass: Determines

- I Which transactions to undo

- I Which pages were dirty (disk version not up to date) at time of crash

- I RedoLSN: LSN from which redo should start

b.Redo pass:

- I Repeats history, redoing all actions from RedoLSN

- ▶ RecLSN and PageLSNs are used to avoid redoing actions already reflected on page

c. Undo pass:

l Rolls back all incomplete transactions

- ▶ Transactions whose abort was complete earlier are not undone

– Key idea: no need to undo these transactions: earlier undo actions were logged, and are redone as required

UNIT 5

Data on External Storage

☐ Disks: Can retrieve random page at fixed cost

But reading several consecutive pages is much cheaper than reading them in random order

□ Tapes: Can only read pages in sequence

Cheaper than disks; used for archival storage

□ File organization: Method of arranging a file of records on external storage.

Record id (rid) is sufficient to physically locate record

Indexes are data structures that allow us to find the record ids of records with given values in index search key fields

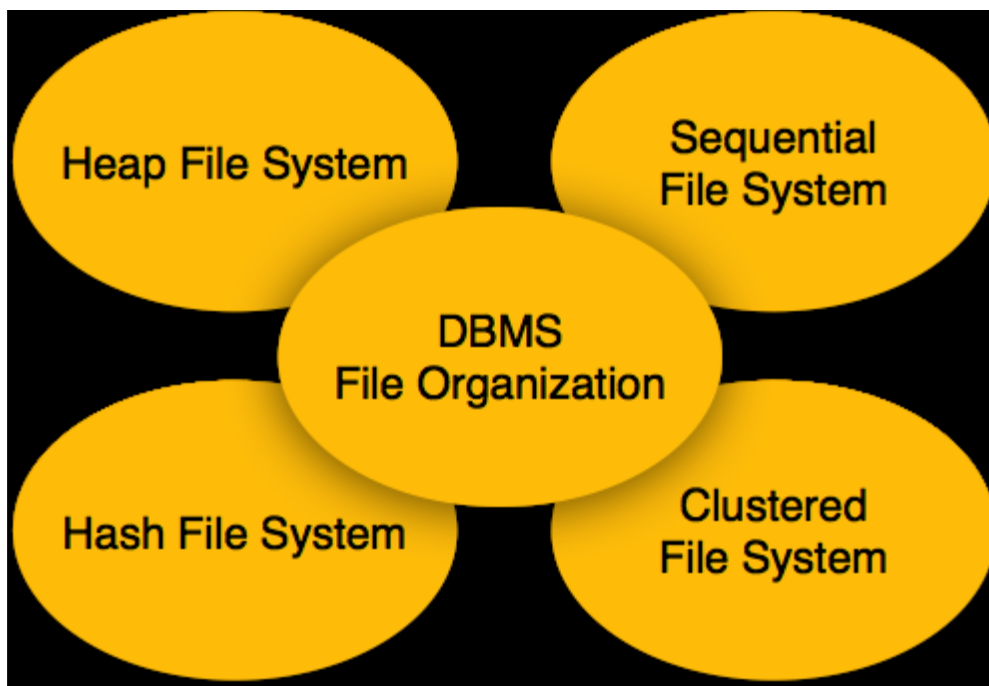
□ Architecture: Buffer manager stages pages from external storage to main memory buffer pool. File and index layers make calls to the buffer manager.

ALTERNATIVE FILE ORGANIZATIONS:

Relative data and information is stored collectively in file formats. A file is sequence of records stored in binary format.

FILE ORGANIZATION

The method of mapping file records to disk blocks defines file organization, i.e. how the file records are organized. The following are the types of file organization



Operating Systems allocates memory area to that file without any further accounting details. File records can be placed anywhere in that memory area.

• **Sequential File Organization:** Every file record contains a data field (attribute) to uniquely identify that record. In sequential file organization mechanism, records are placed in the file in the some sequential order based on the unique key field or search key. Practically, it is not possible to store all the records sequentially in physical form.

Hash File Organization: This mechanism uses a Hash function computation on some field of the records. As we know, that file is a collection of records, which has to be mapped on some block of the disk space allocated to it.

- **Clustered File Organization:** Clustered file organization is not considered good for large databases. In this mechanism, related records from one or more relations are kept in a same disk block, that is, the ordering of records is not based on primary key or search key.

FILE OPERATIONS

Operations on database files can be classified into two categories broadly.

- **Update Operations**
- **Retrieval Operations**

Update operations change the data values by insertion, deletion or update. Retrieval operations on the other hand do not alter the data but retrieve them after optional conditional filtering. In both types of operations, selection plays significant role. Other than creation and deletion of a file, there could be several operations, which can be done on files

- **Open:** A file can be opened in one of two modes, read mode or write mode. In read mode, operating system does not allow anyone to alter data it is solely for reading purpose. Files opened in read mode can be shared among several entities. The other mode is write mode, in which, data modification is allowed. Files opened in write mode can be read also but cannot be shared.

read or written. This pointer can be adjusted accordingly. Using find (seek) operation it can be moved forward or backward.

- **Read:** By default, when files are opened in read mode the file pointer points to the beginning of file. There are options where the user can tell the operating system to where the file pointer to be located at the time of file opening. The very next data to the file pointer is read.

- **Write:** User can select to open files in write mode, which enables them to edit the content of file. It can be deletion, insertion or modification. The file pointer can be located at the time of opening or can be dynamically changed if the operating system allowed doing so.

- **Close:** This also is most important operation from operating system point of view. When a request to close a file is generated, the operating system removes all the locks (if in shared mode) and saves the content of data (if altered) to the secondary storage media and release all the buffers and file handlers associated with the file.

Indexing in DBMS

- Indexing is used to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed.
- The index is a type of data structure. It is used to locate and access the data in a database table quickly.

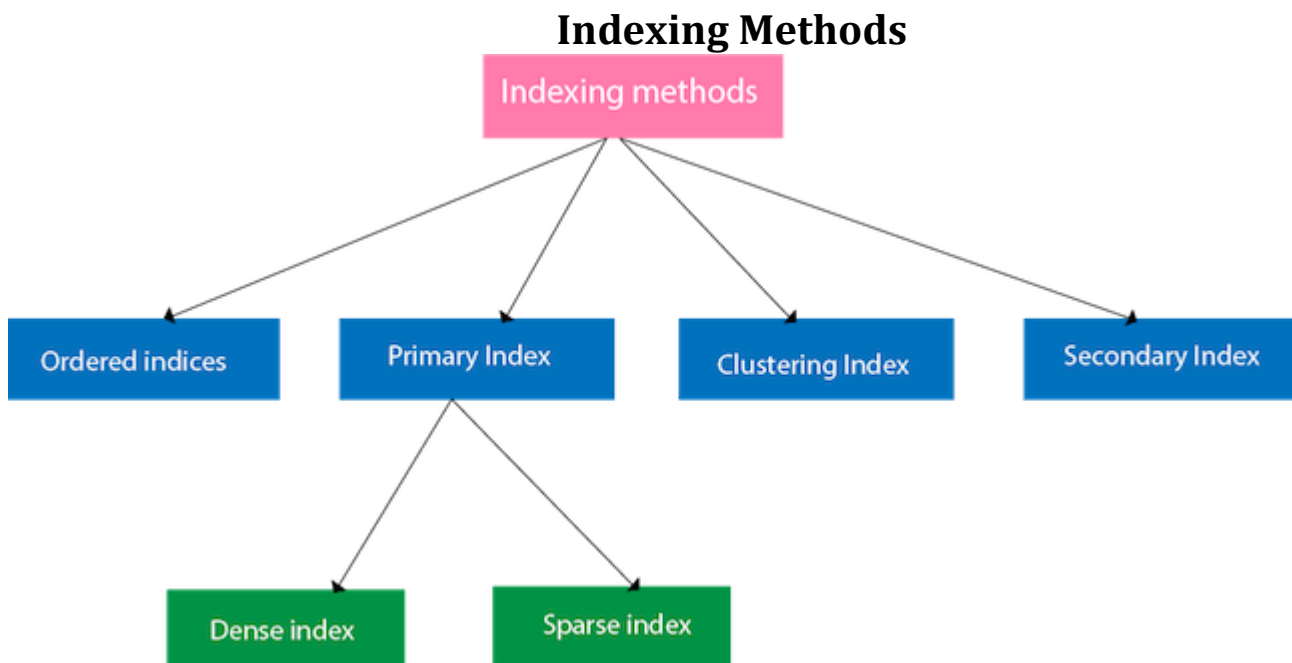
Index structure:

Indexes can be created using some database columns.

Search key	Data Reference
------------	-------------------

Fig: Structure of Index

- The first column of the database is the search key that contains a copy of the primary key or candidate key of the table. The values of the primary key are stored in sorted order so that the corresponding data can be accessed easily.
- The second column of the database is the data reference. It contains a set of pointers holding the address of the disk block where the value of the particular key can be found.



Ordered indices

The indices are usually sorted to make searching faster. The indices which are sorted are known as ordered indices.

Example: Suppose we have an employee table with thousands of record and each of which is 10 bytes long. If their IDs start with 1, 2, 3....and so on and we have to search student with ID-543.

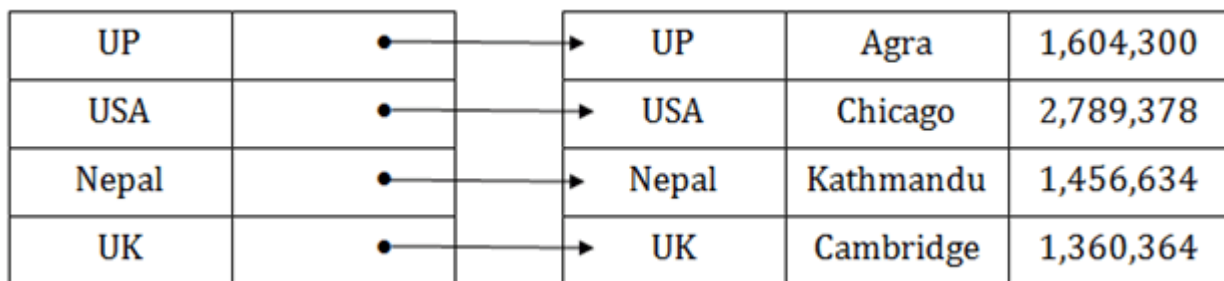
- In the case of a database with no index, we have to search the disk block from starting till it reaches 543. The DBMS will read the record after reading $543 \times 10 = 5430$ bytes.
- In the case of an index, we will search using indexes and the DBMS will read the record after reading $542 \times 2 = 1084$ bytes which are very less compared to the previous case.

Primary Index

- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types: Dense index and Sparse index.

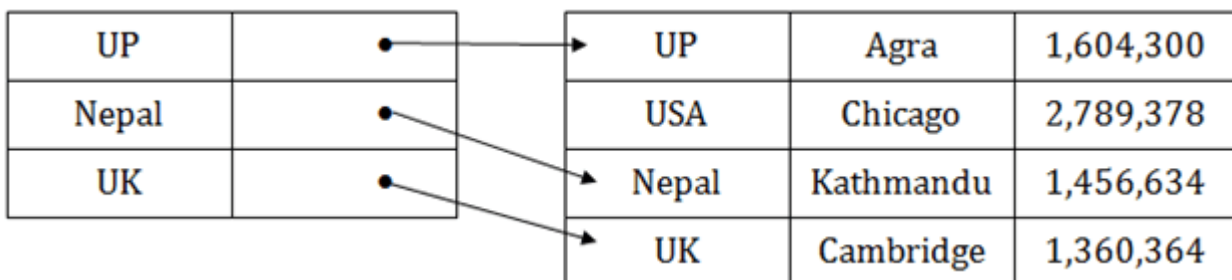
Dense index

- The dense index contains an index record for every search key value in the data file. It makes searching faster.
- In this, the number of records in the index table is same as the number of records in the main table.
- It needs more space to store index record itself. The index records have the search key and a pointer to the actual record on the disk.



Sparse index

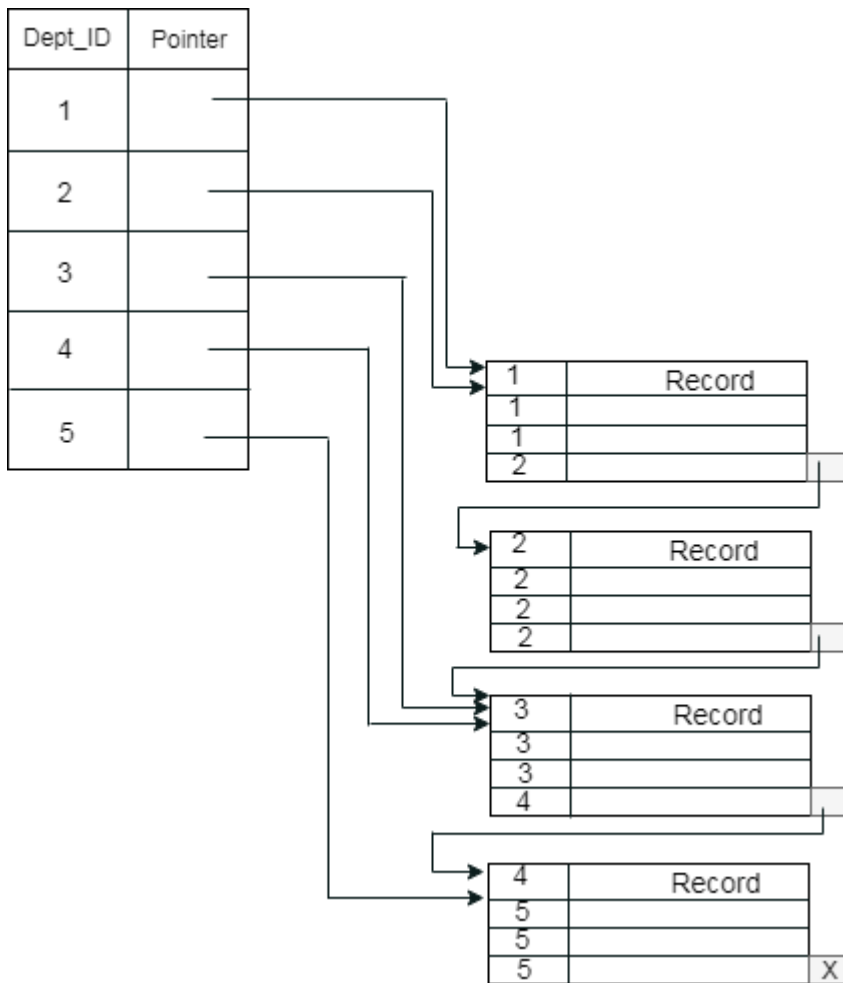
- In the data file, index record appears only for a few items. Each item points to a block.
- In this, instead of pointing to each record in the main table, the index points to the records in the main table in a gap.



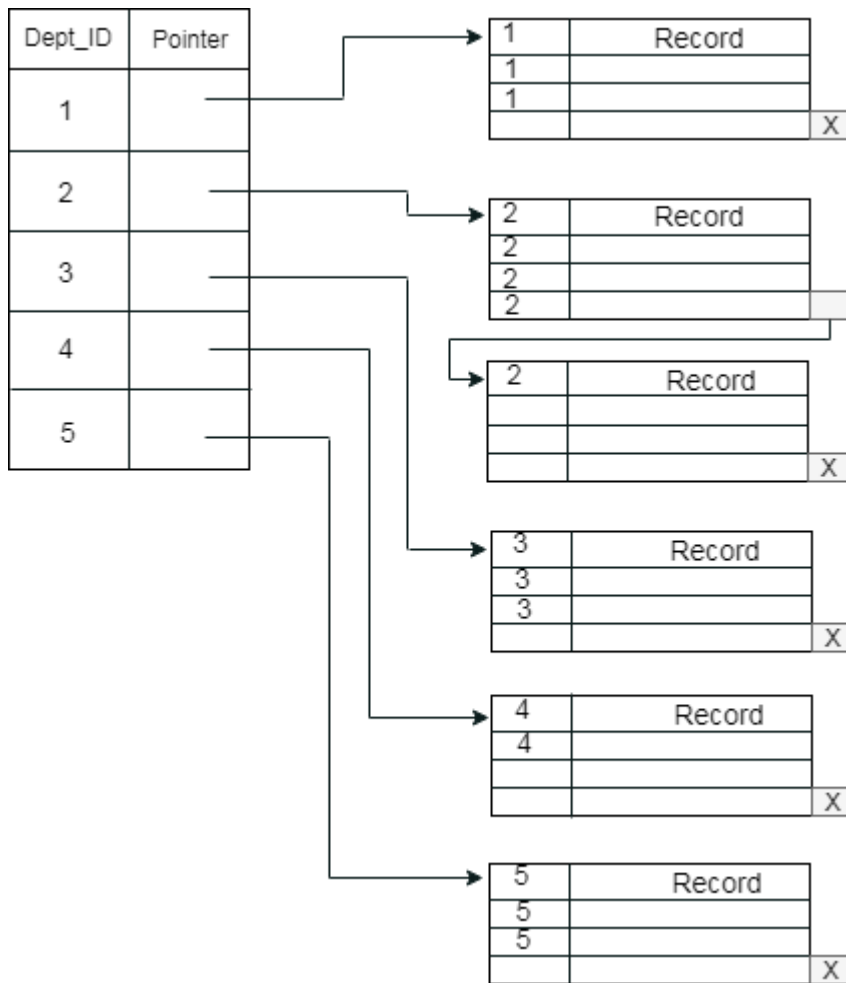
Clustering Index

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called a clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these group.

Example: suppose a company contains several employees in each department. Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole. Here Dept_Id is a non-unique key.



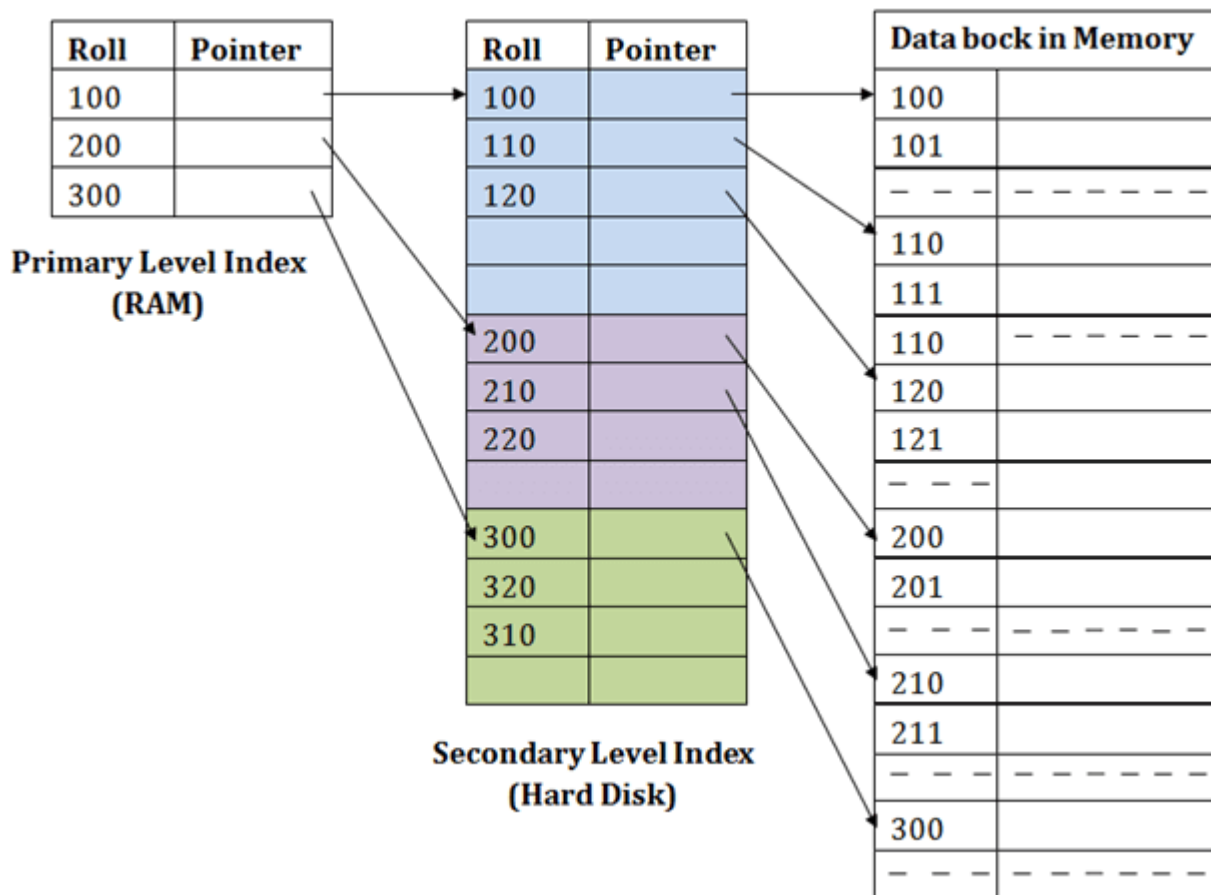
The previous schema is little confusing because one disk block is shared by records which belong to the different cluster. If we use separate disk block for separate clusters, then it is called better technique.



Secondary Index

In the sparse indexing, as the size of the table grows, the size of mapping also grows. These mappings are usually kept in the primary memory so that address fetch should be faster. Then the secondary memory searches the actual data based on the address got from mapping. If the mapping size grows then fetching the address itself becomes slower. In this case, the sparse index will not be efficient. To overcome this problem, secondary indexing is introduced.

In secondary indexing, to reduce the size of mapping, another level of indexing is introduced. In this method, the huge range for the columns is selected initially so that the mapping size of the first level becomes small. Then each range is further divided into smaller ranges. The mapping of the first level is stored in the primary memory, so that address fetch is faster. The mapping of the second level and actual data are stored in the secondary memory (hard disk).

**For example:**

- If you want to find the record of roll 111 in the diagram, then it will search the highest entry which is smaller than or equal to 111 in the first level index. It will get 100 at this level.
- Then in the second index level, again it does $\max(111) \leq 111$ and gets 110. Now using the address 110, it goes to the data block and starts searching each record till it gets 111.
- This is how a search is performed in this method. Inserting, updating or deleting is also done in the same manner.

DBMS INDEXING

We know that information in the DBMS files is stored in form of records. Every record is equipped with some key field, which helps it to be recognized uniquely.

Indexing is defined based on its indexing attributes. Indexing can be one of the following types:

- **Primary Index:** If index is built on ordering 'key-field' of file it is called Primary Index.

Generally it is the primary key of the relation.

- **Secondary Index:** If index is built on non-ordering field of file it is called Secondary Index.

• **Clustering Index:** If index is built on ordering non-key field of file it is called Clustering Index.

Ordering field is the field on which the records of file are ordered. It can be different from primary or candidate key of a file.

Ordered Indexing is of two types:

- Dense Index
- Sparse Index

Dense Index

In dense index, there is an index record for every search key value in the database. This makes searching faster but requires more space to store index records itself. Index record contains search key value and a pointer to the actual record on the disk.



[Image: Dense Index]

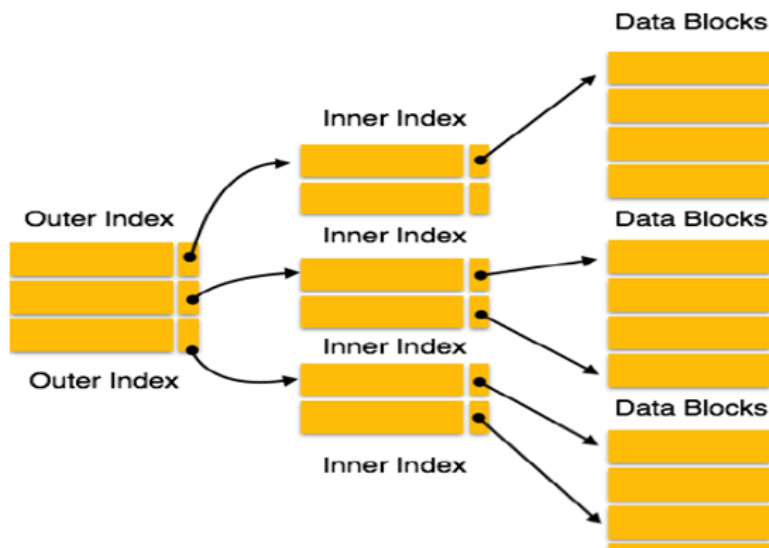
Sparse Index

In sparse index, index records are not created for every search key. An index record here contains search key and actual pointer to the data on the disk. To search a record we first proceed by index record and reach at the actual location of the data.



[Image: Sparse Index]

Index records are comprised of search-key value and data pointers. This index itself is stored on the disk along with the actual database files. As the size of database grows so does the size of indices.



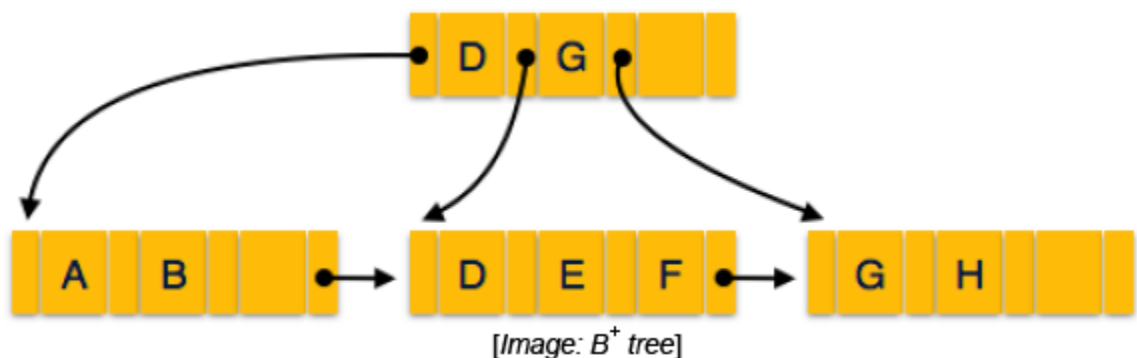
Multi-level Index helps breaking down the index into several smaller indices in order to make the outer most level so small that it can be saved in single disk block which can easily be accommodated anywhere in the main memory.

B+ TREE

B+ tree is multi-level index format, which is balanced binary search trees. As mentioned earlier single level index records becomes large as the database size grows, which also degrades performance. All leaf nodes of B+ tree denote actual data pointers. B+ tree ensures that all leaf nodes remain at the same height, thus balanced. Additionally, all leaf nodes are linked using link list, which makes B+ tree to support random access as well as sequential access.

STRUCTURE OF B+ TREE

Every leaf node is at equal distance from the root node. A B+ tree is of order n where n is fixed for every B+ tree.



Internal nodes:

- Internal (non-leaf) nodes contains at least $\lceil n/2 \rceil$ pointers, except the root node.
- At most, internal nodes contain n pointers.

Leaf nodes:

- Leaf nodes contain at least $\lceil n/2 \rceil$ record pointers and $\lceil n/2 \rceil$ key values
- At most, leaf nodes contain n record pointers and n key values
- Every leaf node contains one block pointer P to point to next leaf node and forms a linked list.

B+ tree insertion

- B+ tree are filled from bottom. And each node is inserted at leaf node.

- **If leaf node overflows:**

- o Split node into two parts
- o Partition at $i = \lfloor (m+1)/2 \rfloor$
- o First i entries are stored in one node
- o Rest of the entries ($i+1$ onwards) are moved to a new node
- i th key is duplicated in the parent of the leaf

- **If non-leaf node overflows:**

- o Split node into two parts
- o Partition the node at $i = \lfloor (m+1)/2 \rfloor$
- o Entries upto i are kept in one node
- o Rest of the entries are moved to a new node

B+ tree deletion

- B+ tree entries are deleted leaf nodes.
- The target entry is searched and deleted.
- o If it is in internal node, delete and replace with the entry from the left position.
- After deletion underflow is tested
- o If underflow occurs
 - Distribute entries from nodes left to it.
- o If distribution from left is not possible
 - Distribute from nodes right to it
- o If distribution from left and right is not possible
 - Merge the node with left and right to it.

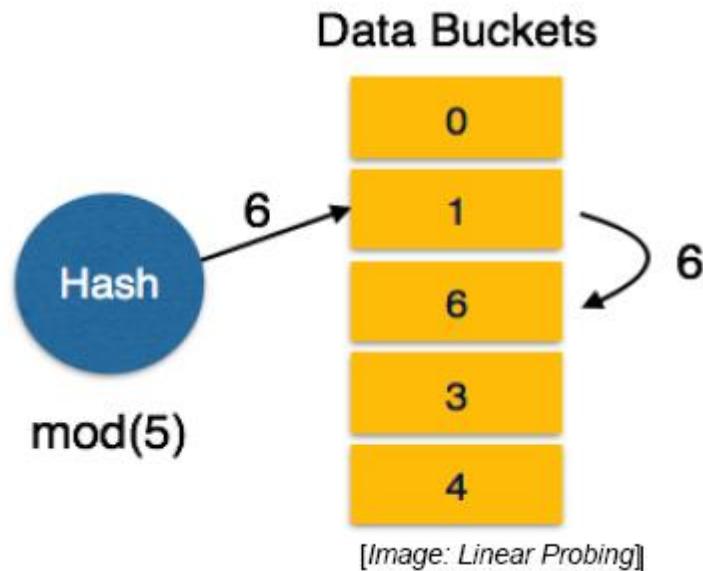
DBMS HASHING

For a huge database structure it is not sometime feasible to search index through all its level and then reach the destination data block to retrieve the desired data. Hashing is an effective

technique to calculate direct location of data record on the disk without using index structure.

Hash Organization

· **Bucket:** Hash file stores data in bucket format. Bucket is considered a unit of storage. Bucket typically stores one complete disk block, which in turn can store one or more records.

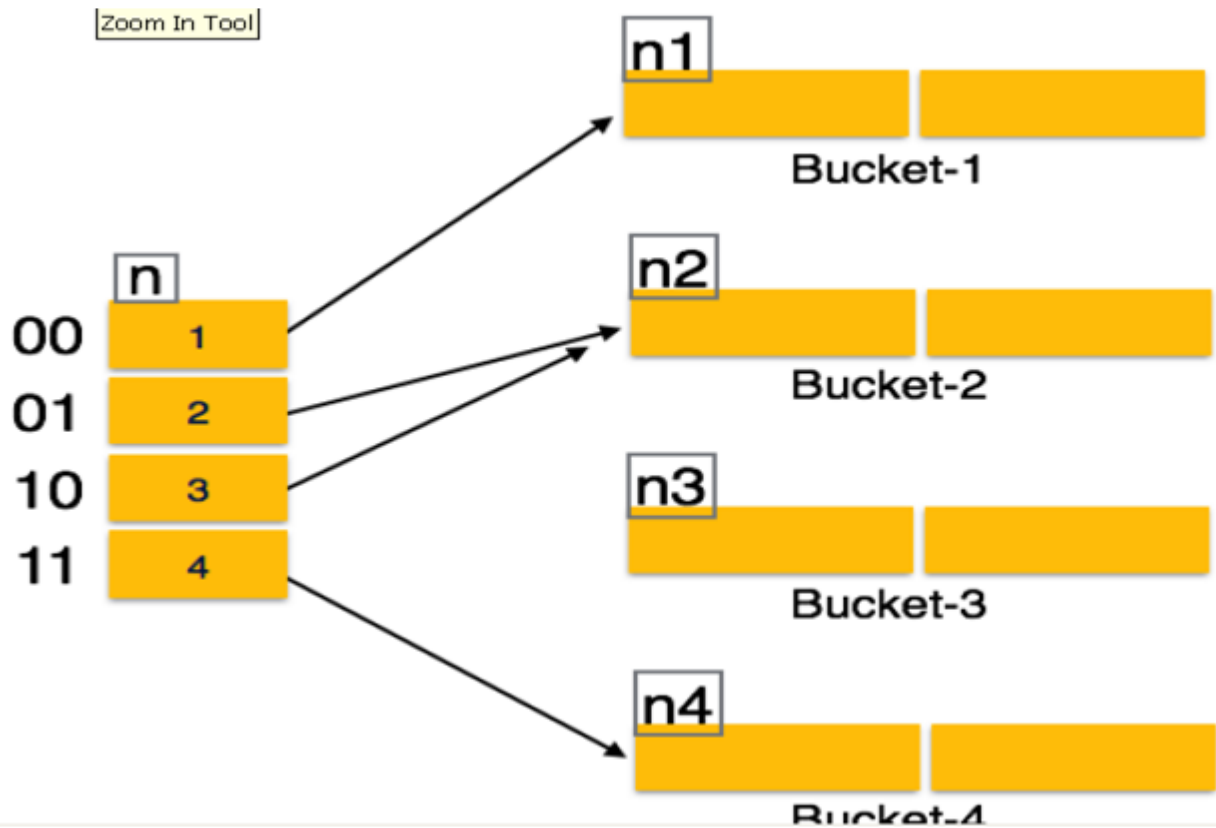


For a hash function to work efficiently and effectively the following must match:

- Distribution of records should be uniform
- Distribution should be random instead of any ordering

Dynamic Hashing

Problem with static hashing is that it does not expand or shrink dynamically as the size of database grows or shrinks. Dynamic hashing provides a mechanism in which data buckets are added and removed dynamically and on-demand. Dynamic hashing is also known as extended hashing.



· **Querying:** Look at the depth value of hash index and use those bits to compute the bucket address.

- **Update:** Perform a query as above and update data.
- **Deletion:** Perform a query to locate desired data and delete data.
- **Insertion:** compute the address of bucket o If the bucket is already full
 - Add more buckets
 - Add additional bit to hash value
 - Re-compute the hash function o Else
 - Add data to the bucket o If all buckets are full, perform the remedies of static hashing.

Hashing is not favorable when the data is organized in some ordering and queries require range of data. When data is discrete and random, hash performs the best. Hashing algorithm and implementation have high complexity than indexing. All hash operations are done in constant time.

QUERY OPTIMIZATION

Query Optimization works in a similar way:

There can be many different ways to get an answer from a given query. The result would be same in all scenarios.

DBMS strive to process the query in the most efficient way (in terms of 'Time') to produce the answer.

Cost = Time needed to get all answers

- Query optimization is the process of selecting the most efficient query-evaluation plan from

among the many strategies usually possible for processing a given query, especially if the query is complex.

- One aspect of optimization occurs at the relational-algebra level, where the system attempts to find an expression that is equivalent to the given expression, but more efficient to execute.

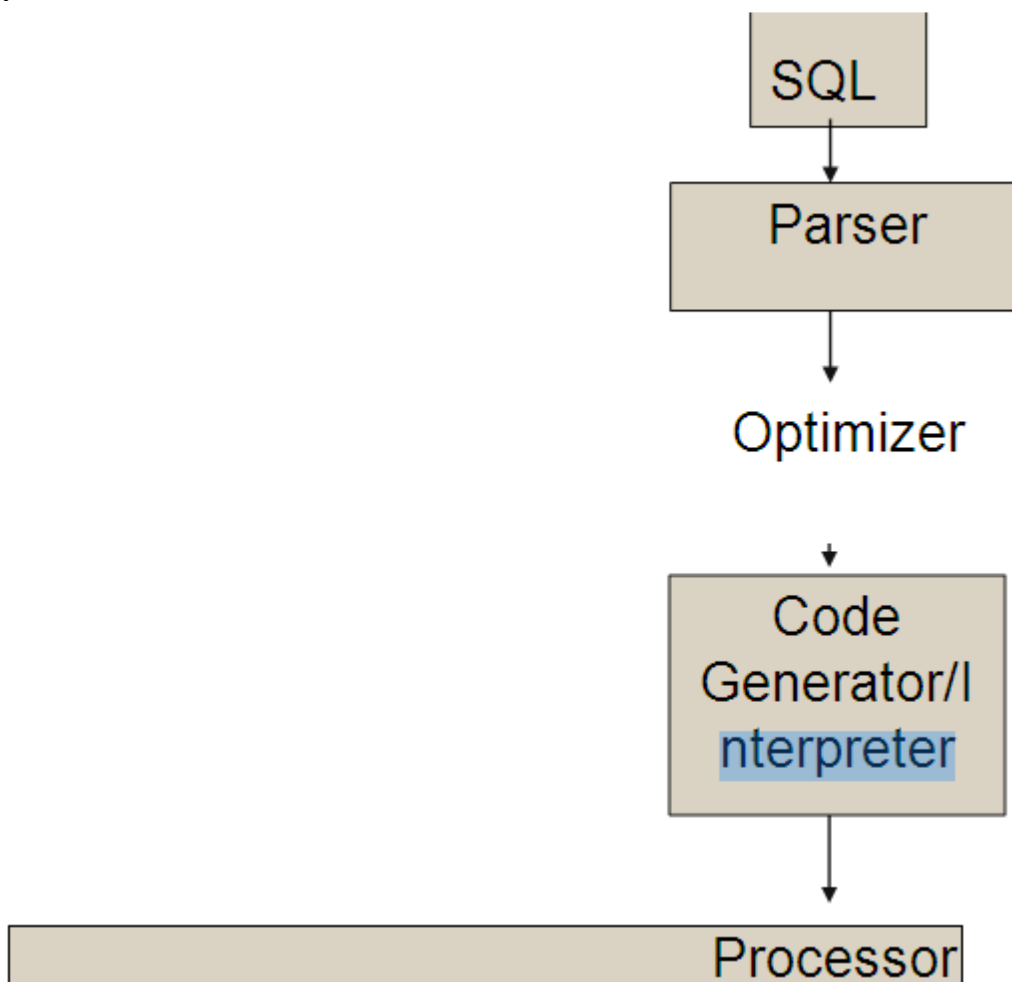
- Another aspect is selecting a detailed strategy for processing the query, such as choosing the algorithm to use for executing an operation, choosing the specific indices to use, and so on.

The estimation should be accurate and easy. Another important point is the need for being logically consistent because the least cost plan will always be consistently low.

Steps in a Cost-based query optimization

1. Parsing
2. Transformation
3. Implementation
4. Plan selection based on cost estimates

QUERY FLOW



- Query Parser – Verify validity of the SQL statement. Translate query into an internal structure using relational calculus.

- Query Optimizer – Find the best expression from various different algebraic expressions.

Criteria used is 'Cheapness'

- Code Generator/Interpreter – Make calls for the Query processor as a result of the work done by the optimizer.
- Query Processor – Execute the calls obtained from the code generator.

MEASURES OF QUERY COST

There are multiple possible evaluation plans for a query, and it is important to be able to compare the alternatives in terms of their (estimated) cost, and choose the best plan. To do so, we must estimate the cost of individual operations, and combine them to get the cost of a query evaluation plan. Thus, as we study evaluation algorithms for each operation later in this chapter, we also outline how to estimate the cost of the operation.

We use the *number of block transfers* from disk and *the number of disk seeks* to estimate the cost of a query-evaluation plan. If the disk subsystem takes an average of t_r seconds to transfer a block of data, and has an average block-access time (disk seek time plus rotational latency) of t_s seconds, then an operation that transfers b blocks and performs S seeks would take $b * t_r + S * t_s$ seconds. The values of t_r and t_s must be calibrated for the disk system used, but typical values for high-end disks today would be $t_s = 4$ milliseconds and $t_r = 0.1$ milliseconds, assuming a 4-kilobyte block size and a transfer rate of 40 megabytes per second.

t_r – time to transfer one block

t_s – time for one seek

Cost for b block transfers plus S seeks

$$b * t_r + S * t_s$$

Comparing File Organizations

- ☐ Heap files (random order; insert at eof)
- ☐ Sorted files, sorted on $\langle age, sal \rangle$
- ☐ Clustered B+ tree file, Alternative (1), searchkey $\langle age, sal \rangle$
- ☐ Heap file with unclustered B + tree index on search key $\langle age, sal \rangle$
- ☐ Heap file with unclustered hash index on search key $\langle age, sal \rangle$

What is Index tuning?

Index tuning is part of database tuning for selecting and creating indexes. The index tuning goal is to reduce the query processing time. Potential use of indexes in dynamic environments with several ad-hoc queries in advance is a difficult task. Index tuning involves the queries based on indexes and the indexes are created automatically on-the-fly. No explicit actions are needed by the database users for index tuning.

How is index tuning used to improve query performance?

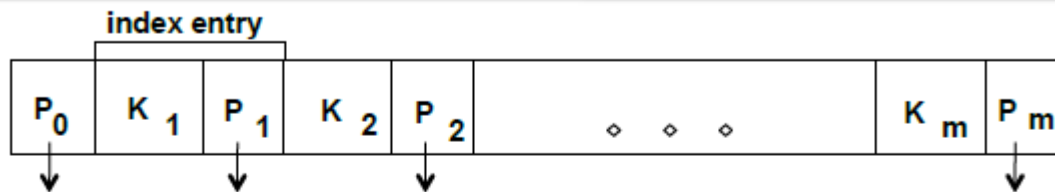
The Index tuning wizard can be used to improve the performance of queries and databases. It uses the following measures to do so:

- It uses the query optimizer to perform the analysis of queries with respect to the workload and based on this knowledge, it recommends the best usage of indexes.
- The changes in the usage of index, query distribution and their performance are analysed for checking the effect.
- It also recommends ways of tuning the database for a small set of problem queries.

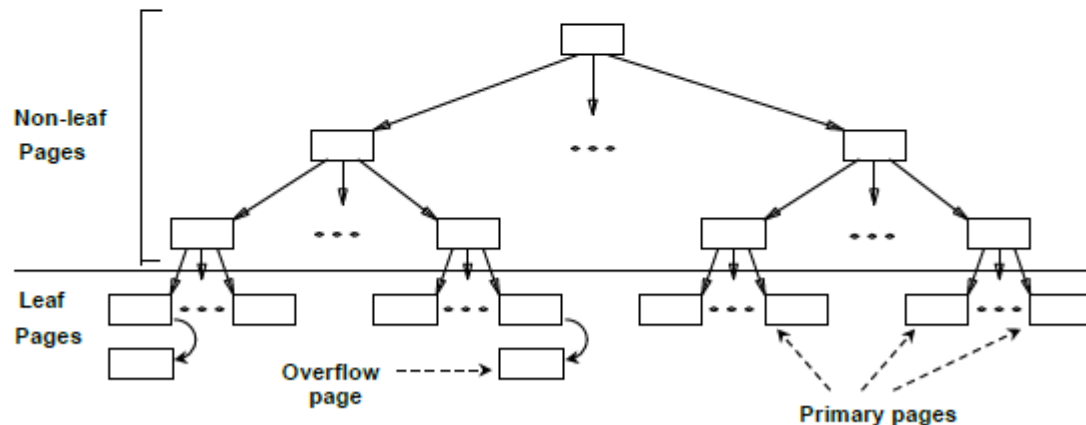
Following are the reasons for the poor performance of a query:

- No indexes.
- Excess recompilations of stored procedures.
- Procedures and triggers without SET NOCOUNT ON.
- Poorly written query with unnecessarily complicated joins.
- Highly normalized database design.
- Excess usage of cursors and temporary tables.
- Queries with predicates that use comparison operators between different columns of the same table.
- Queries with predicates that use operators, and any one of the following are true:
 1. There are no statistics on the columns involved on either side of the operators.
 2. The distribution of values in the statistics is not uniform, but the query seeks a highly selective value set. This situation can be especially true if the operator is anything other than the equality (=) operator.
 3. The predicate uses the not equal to (!=) comparison operator or the NOT logical operator.
- Queries that use any of the SQL Server built-in functions or a scalar-valued, user-defined function whose argument is not a constant value.
- Queries that involve joining columns through arithmetic or string concatenation operators.
- Queries that compare variables whose values are not known when the query is compiled and optimized.

ISAM – Indexed Sequential Access Method

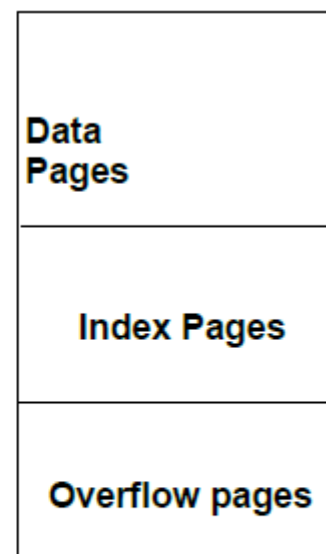


- ❖ Index file may be quite large.
- ❖ Can be applied hierarchically!



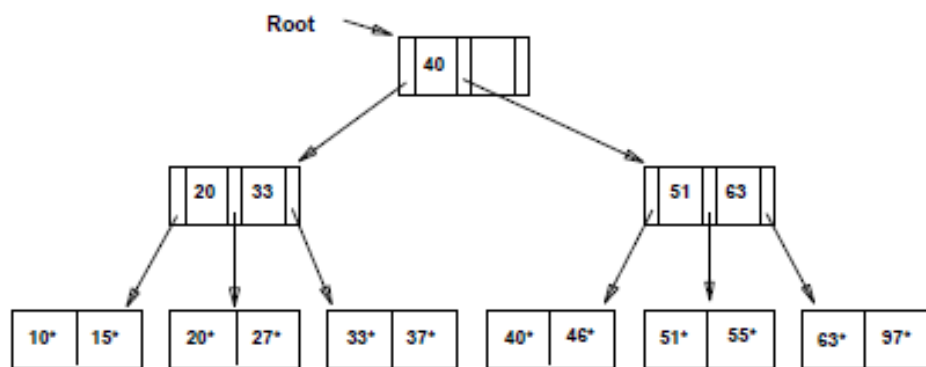
➤ Leaf pages contain *data entries* (i.e. actual records or $\langle \text{key}, \text{rid} \rangle$ pairs).

- ❖ File creation: Leaf (data) pages allocated sequentially, sorted by search key; then index pages allocated, then space for overflow pages.
- ❖ Index entries: $\langle \text{search key value}, \text{page id} \rangle$; they 'direct' search for *data entries*, which are in leaf pages.
- ❖ Search: Start at root; use key comparisons to go to leaf. Cost $\log_F N$
 $F = \# \text{ entries/index pg}$, $N = \# \text{ leaf pgs}$
- ❖ Insert: Find leaf data entry belongs to, put it there if space is available, else allocate an overflow page, put it there, and link it in.
- ❖ Delete: Find and remove from leaf; if empty overflow page, de-allocate.

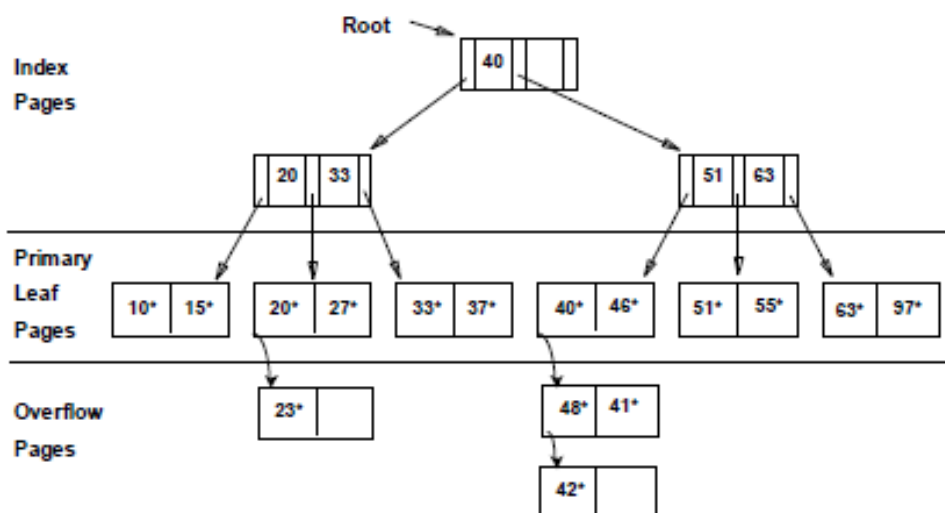


Example ISAM Tree

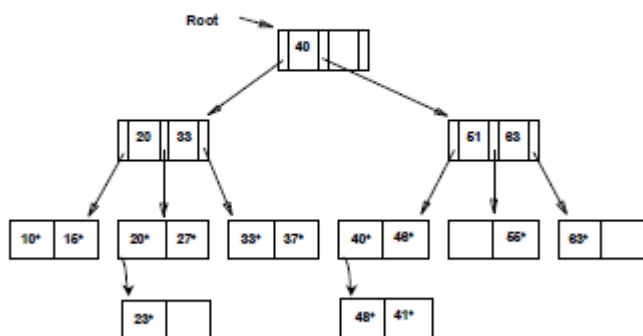
- ❖ Each node can hold 2 entries; no need for 'next-leaf-page' pointers. (Why?)



After Inserting 23, 48*, 41*, 42* ...*



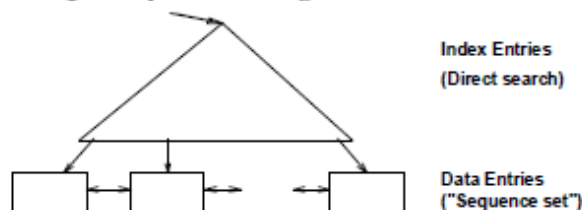
... Then Deleting 42, 51*, 97**



* Note that 51* appears in index levels, but not in leaf!

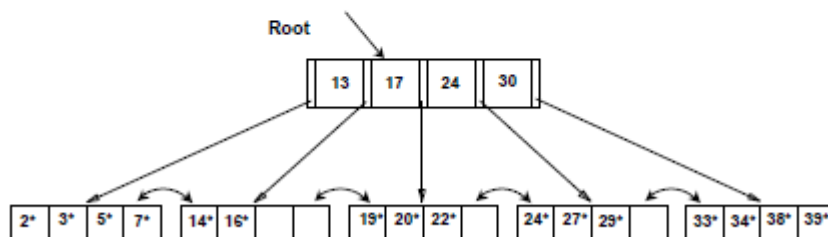
B+ Tree: Most Widely Used Index

- ❖ Insert/delete at $\log_F N$ cost; keep tree *height-balanced*. (F = fanout, N = # leaf pages)
- ❖ Minimum 50% occupancy (except for root). Each node contains $d \leq \underline{m} \leq 2d$ entries. The parameter d is called the *order* of the tree.
- ❖ Supports equality and range-searches efficiently.



Example B+ Tree

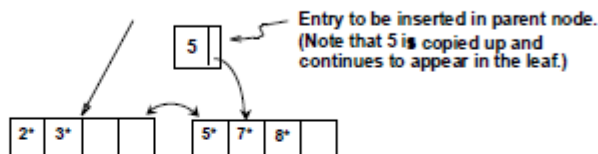
- ❖ Search begins at root, and key comparisons direct it to a leaf (as in ISAM).
- ❖ Search for 5^* , 15^* , all data entries $\geq 24^*$...



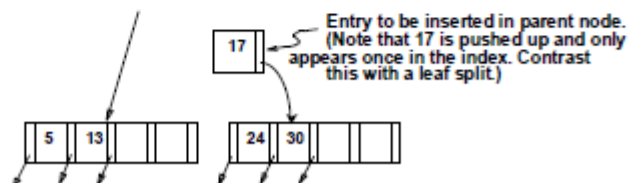
* Based on the search for 15^* , we know it is not in the tree!

Inserting 8^* into Example B+ Tree

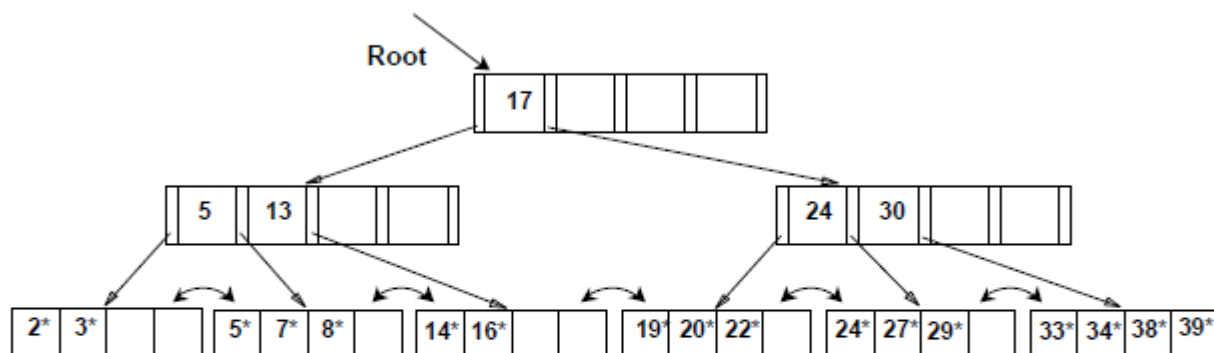
- ❖ Observe how minimum occupancy is guaranteed in both leaf and index pg splits.



- ❖ Note difference between *copy-up* and *push-up*; be sure you understand the reasons for this.



Example B+ Tree After Inserting 8*



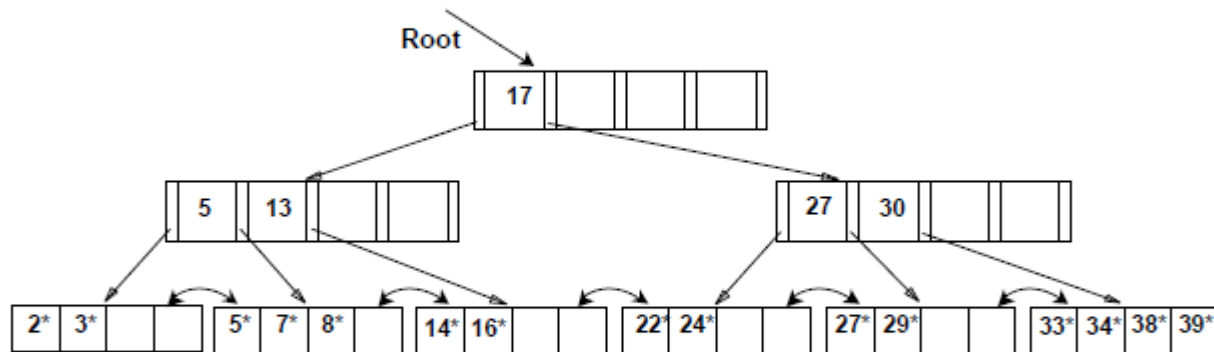
v Notice that root was split, leading to increase in height.

v In this example, we can avoid split by re-distributing entries; however, this is usually not done in practice.

Deleting a Data Entry from a B+ Tree

- ❖ Start at root, find leaf L where entry belongs.
- ❖ Remove the entry.
 - If L is at least half-full, *done!*
 - If L has only $d-1$ entries,
 - Try to re-distribute, borrowing from sibling (*adjacent node with same parent as L*).
 - If re-distribution fails, merge L and sibling.
- ❖ If merge occurred, must delete entry (pointing to L or sibling) from parent of L .
- ❖ Merge could propagate to root, decreasing height.

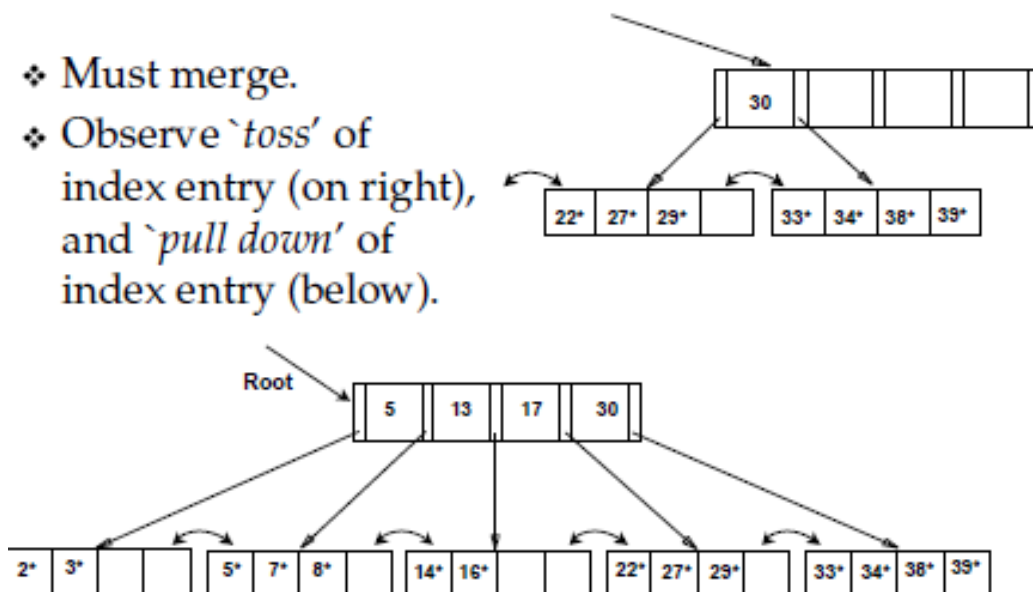
Example Tree After (Inserting 8* Then) Deleting 19* and 20* ...



- ❖ Deleting 19* is easy.
- ❖ Deleting 20* is done with re-distribution.
Notice how middle key is *copied up*.

... And Then Deleting 24*

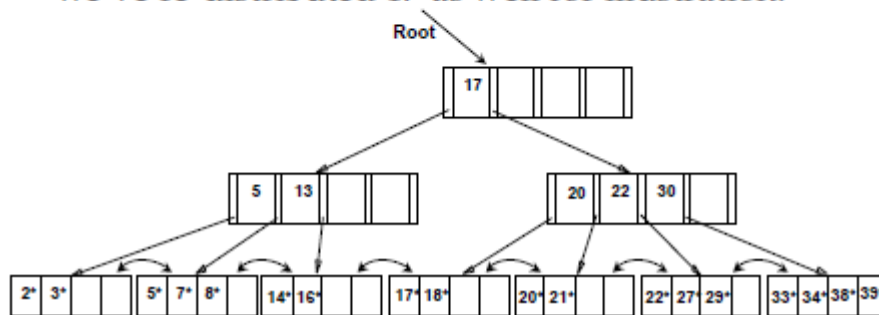
- ❖ Must merge.
- ❖ Observe 'toss' of index entry (on right), and 'pull down' of index entry (below).



After Re-distribution



- ❖ Intuitively, entries are re-distributed by *'pushing through'* the splitting entry in the parent node.
- ❖ It suffices to re-distribute index entry with key 20; we've re-distributed 17 as well for illustration.



LECTURE NOTES FOR GAPS

Embedded SQL

As we have seen in our previous tutorials, SQL is known as the Structured Query Language. It is the language that we use to perform operations and transactions on the databases.

When we talk about industry-level applications we need properly connected systems which could draw data from the database and present to the user. In such cases, the embedded SQL comes to our rescue.

We embed SQL queries into high-level languages such that they can easily perform the logic part of our analysis.

Some of the prominent examples of languages with which we embed SQL are as follows:

- C++
- Java
- Python etc.

Why do we need Embedded SQL?

Embedded SQL gives us the freedom to use databases as and when required. Once the application we develop goes into the production mode several things need to be taken care of.

We need to take care of a thousand things out of which one major aspect is the problem of authorization and fetching and feeding of data into/from the database.

With the help of the embedding of queries, we can easily use the database without creating any bulky code. With the embedded SQL, we can create API's which can easily fetch and feed data as and when required.

How to Embed SQL in High-Level Languages?

For using embedded SQL, we need some tools in each high-level language. In some cases, we have inbuilt libraries which provide us with the basic building block.

While in some cases we need to import or use some packages to perform the desired tasks.

For example, in Java, we need a connection class. We first create a connection by using the connection class and further we open the connection bypassing the required parameters to connect with the database.

Example: How to connect to a database (using JAVA).

Code[with embedded SQL]:

```
Class.forName("com.mysql.jdbc.Driver");  
Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/DataFlair","user","root");  
Statement statement = connection.createStatement();
```

Here, DataFlair is the name of the database and user is the username and root is the password.

Advantages of Embedded SQL

Some of the advantages of using SQL embedded in high-level languages are as follows:

- Helps to access databases from anywhere.
- Allows integrating authentication service for large scale applications.
- Provides extra security to database transactions.
- Avoids logical errors while performing transactions on our database.
- Makes it easy to integrate the frontend and the backend of our application.

QUERY EVALUATION:

- A query evaluation plan (Oracle calls it an “execution plan”) is a program for an abstract machine (interpreter) inside the DBMS. It is produced by the query optimizer.

Another name is “access plan” (the DBMS has to decide how to access the rows, e.g. whether to use an index).

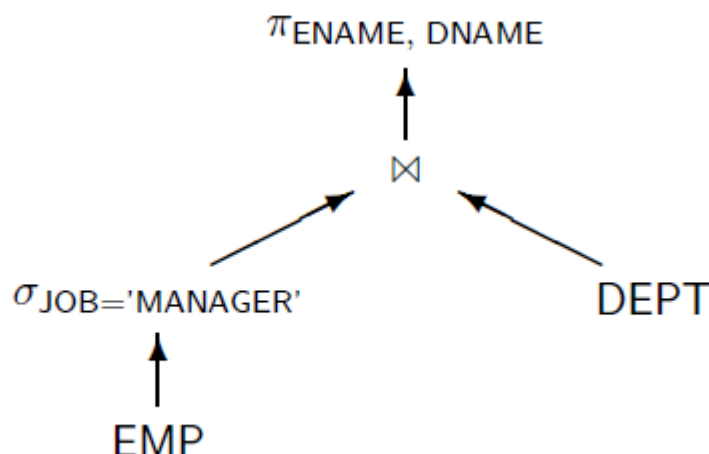
- In most systems, query evaluation plans (QEPs) are similar to relational algebra expressions (very system dependent).
- E.g. consider the following SQL query:

```
SELECT ENAME, DNAME
FROM   EMP, DEPT
WHERE  EMP.DEPTNO = DEPT.DEPTNO
AND    JOB = 'MANAGER'
```

- In relational algebra, the example query would be written as:

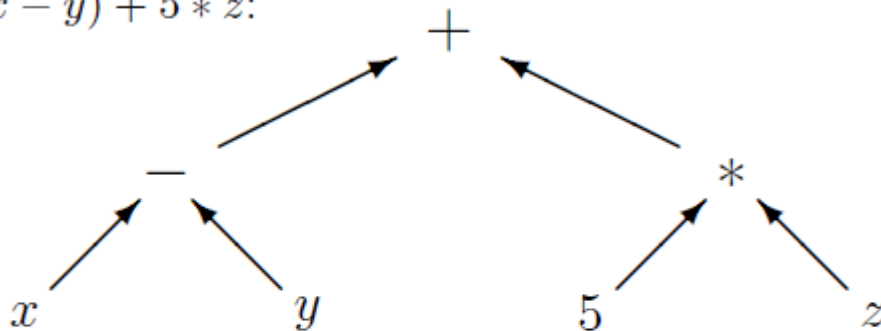
$$\pi_{ENAME, DNAME}(\sigma_{JOB='MANAGER'}(EMP) \bowtie DEPT)$$

- Complex relational algebra expressions are best displayed as “operator trees” (showing the flow of data):



Processing of Evaluation Plans (1)

- So a query evaluation plan is a tree with an operation attached to every node.
- Every node computes a relation which is passed as input to its parent node (up in the tree).
- The relation computed by the root node (the tree top) is returned to the user as the answer to the given SQL query.
- The leaf nodes access tables or indexes to compute their relation.
- Operations in other nodes process or combine relations which they get from their child nodes.
- Also arithmetic expressions can be displayed as a tree, e.g. $(x - y) + 5 * z$:



- Arithmetic expressions are usually compiled by using registers as temporary storage:
R1 := x + y;
R2 := 5 * z;
R1 := R1 + R2; // R1 contains now result

- It would be possible to compute every operation at once completely, store the result in a temporary relation, and let the parent operation read this temporary relation:

```
R1 :=  $\sigma_{\text{JOB}=\text{'MANAGER'}}$ (EMP);  
R2 := R1  $\bowtie$  DEPT;  
R3 :=  $\pi_{\text{ENAME, DNAME}}$ (R2);  
print R3;
```

- But then a lot of memory is needed for the intermediate results, and sometimes intermediate results are so large that they have to be written to disk and then read again.
- E.g. when the join has computed a tuple, one can immediately compute the projection result for that tuple.
- One can eliminate nearly all temporary storage since most operations work “tuple by tuple” (sorting is an exception).
- Most operations compute tuples only on demand (when the parent node needs them), and only one tuple at a time.

E.g. the join node requests a tuple from the selection node. In order to satisfy the request, it requests a tuple from the relation EMP and checks the condition $\text{JOB}=\text{'MANAGER'}$. If the condition is satisfied, it returns the tuple and is done. If not, it requests another tuple from the relation EMP.

- Thus, tuples flow immediately from the child to the parent, even before the child has computed the complete result.
- This is called “Pipelined Execution” (or lazy evaluation).